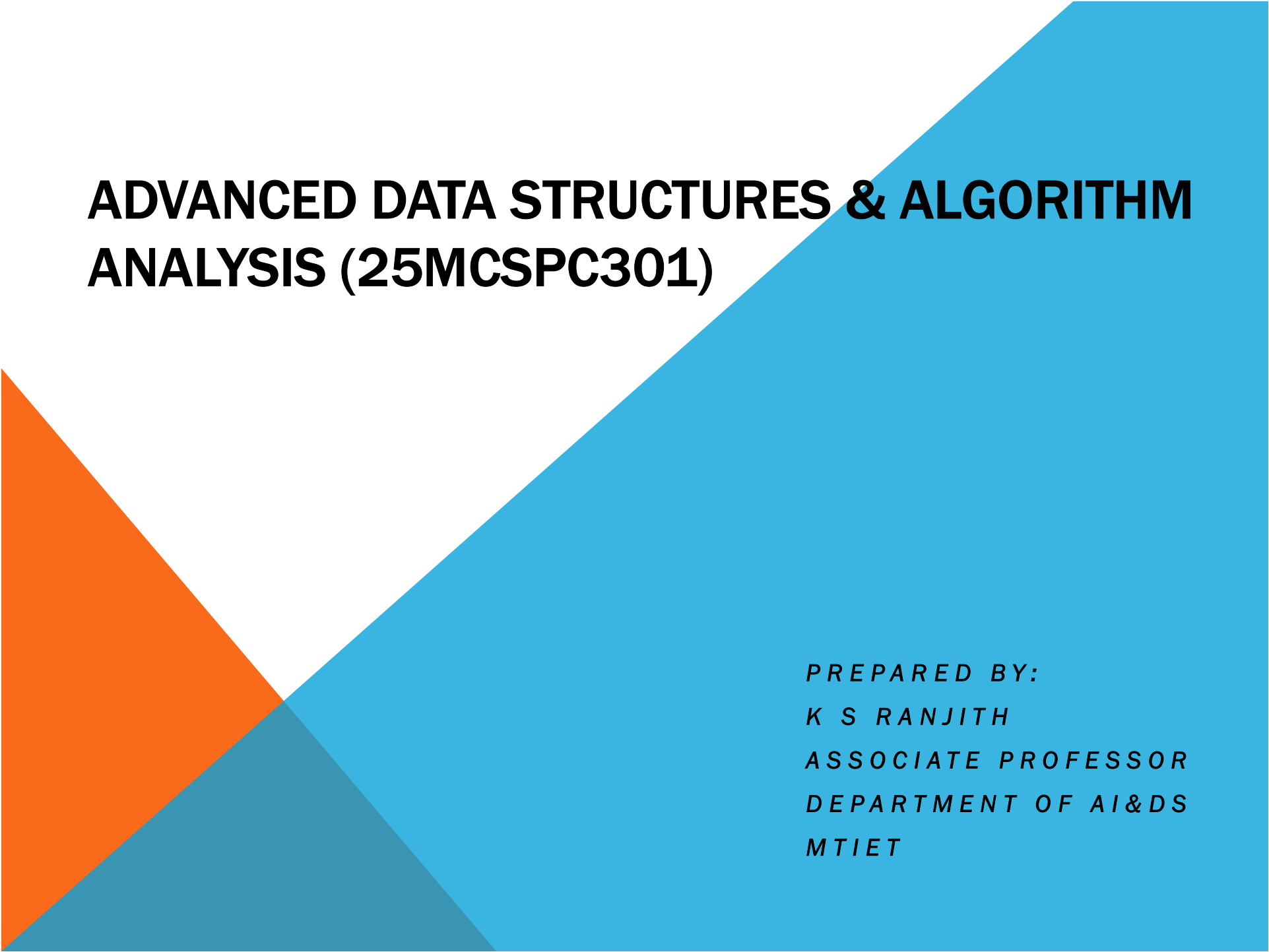




ADVANCED DATA STRUCTURES & ALGORITHM ANALYSIS (25MCSPC301)

*PREPARED BY:
K S RANJITH
ASSOCIATE PROFESSOR
DEPARTMENT OF AI&DS
MTIET*

UNIT 1

Introduction to Algorithm Analysis, Space and Time Complexity analysis, Asymptotic Notations.

AVL Trees – Creation, Insertion, Deletion operations and Applications

B-Trees – Creation, Insertion, Deletion operations and Applications

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

Topics

What is an Algorithm?

Syllabus

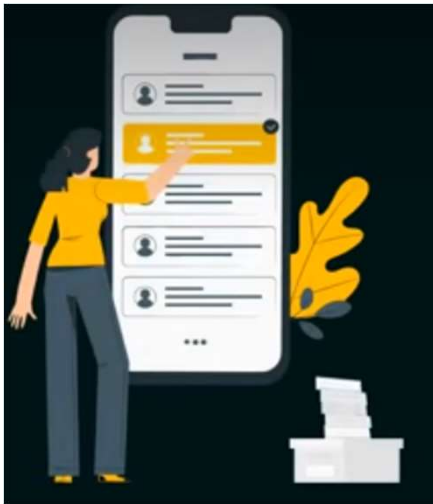
Target Audience

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

WHAT IS AN ALGORITHM?

- Step-by-step procedure to solve a specific problem.



Problem Statement

Write an algorithm to scan the phone's contact list.

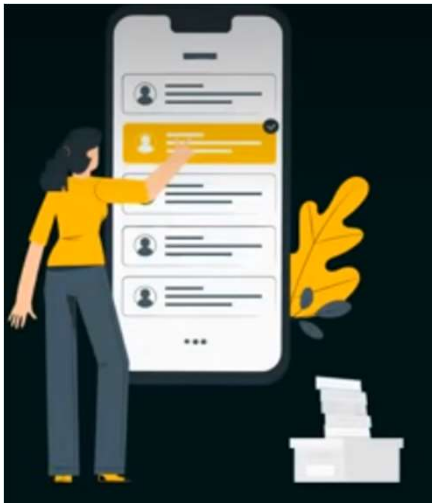
Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

WHAT IS AN ALGORITHM?

- Step-by-step procedure to solve a specific problem.

Steps



1. Unlock your phone.
2. Open the Contacts app.
3. Start scanning the list.
4. Compare the contact name with the desired contact name.
5. If the names match, then match is found.
6. If the names do not match, then move to the next contact in the list.
7. Repeat until the desired contact is found.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

CONTENTS IN UNIT 1

- Introduction to Algorithm Analysis
- Space and Time Complexity analysis
- Asymptotic Notations.
- AVL Trees – Creation, Insertion, Deletion operations, Applications
- B-Trees – Creation, Insertion, Deletion operations and Applications

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

TARGET AUDIENCE

- Students preparing for interviews and competitive exams (GATE, NET, etc.).
- Software developers, engineers, and tech enthusiasts.
- Anyone interested in solving computational problems.



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

ALGORITHM VS. PROGRAM

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

INTRODUCTION

- An algorithm is a step-by-step procedure to solve a specific problem.
- A program is also a step-by-step procedure to solve a specific problem.

So, what's the difference?



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

ALGORITHM VS. PROGRAM

1. An algorithm is an **abstract concept**. Whereas, a program is a **concrete implementation** of the algorithm.

Algorithm	Program
1. Start 2. Initialize a variable 'result' to 1. 3. Read the input number 'n'. 4. Repeat the following until 'n' becomes 0: a. Multiply 'result' by 'n'. b. Decrement 'n' by 1. 5. Print the value of 'result' as the factorial. 6. Stop.	<pre>int fact(int n) { int result = 1; while (n > 0) { result *= n; n--; } return result; }</pre>

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

ALGORITHM VS. PROGRAM

2. An algorithm can be written in **any language**, while a program must be written using a **programming language** only.

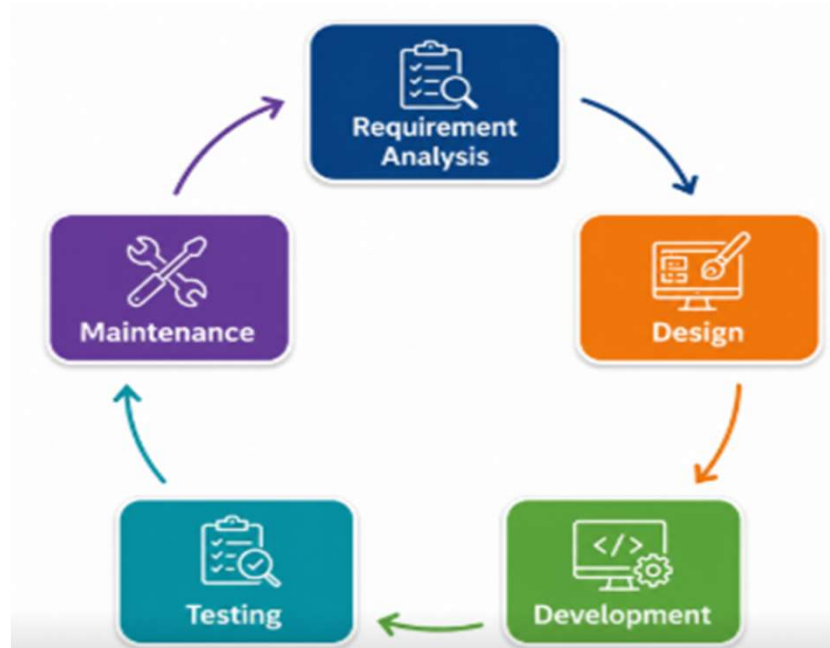
Algorithm	Program
1. Start 2. Initialize a variable 'result' to 1. 3. Read the input number 'n'. 4. Repeat the following until 'n' becomes 0: a. Multiply 'result' by 'n'. b. Decrement 'n' by 1. 5. Print the value of 'result' as the factorial. 6. Stop.	<pre>int fact(int n) { int result = 1; while (n > 0) { result *= n; n--; } return result; }</pre>

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

ALGORITHM VS. PROGRAM

3. An algorithm is developed during the **design phase**, and a program is developed during the **development phase**.



Software Development life Cycle

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

ALGORITHM VS. PROGRAM

4. An algorithm does not depend on the **hardware** and **operating system** while a program depends upon them.



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

ALGORITHM VS. PROGRAM

5. An algorithm is always **analyzed**, while a program is **tested**.



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

SUMMARY

Algorithm	Program
1. It is an abstract concept.	1. It is a concrete implementation.
2. Can be written in any language.	2. Can be written using a programming language.
3. Developed during the design phase.	3. Developed during the development phase.
4. Independent of the hardware and operating system.	4. Depends on the hardware and operating system.
5. It is analyzed.	5. It is tested.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

CHARACTERISTICS OF AN ALGORITHM

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

CHARACTERISTICS OF AN ALGORITHM

1. **Input:** An algorithm can take 0 or more inputs.

Algorithm with no input

1. Start.
2. Generate a random number.
3. Output the random number.
4. Stop

Algorithm with two inputs

1. Start.
2. Input the first number (let's call it num1).
3. Input the second number (let's call it num2).
4. Add num1 and num2.
5. Output the result of the addition.
6. Stop.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

CHARACTERISTICS OF AN ALGORITHM

2. **Output:** An algorithm must produce **at least 1 output**

Algorithm with one output

1. Start.
2. Generate a random number.
3. Output the random number.
4. Stop.

Algorithm with two outputs

1. Start.
2. Input a number (let's call it num).
3. Calculate the square of num (let's call it SquareResult).
4. Calculate the cube of num (let's call it CubeResult).
5. Output SquareResult.
6. Output CubeResult.
7. Stop

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

CHARACTERISTICS OF AN ALGORITHM

3. **Finiteness**: An algorithm must **terminate** in **finite time**.

Algorithm that runs forever

1. Start.
2. Initialize count to 0.
3. Enter the loop.
4. Within the loop, increment the count by 1.
5. Repeat steps 3 and 4.
6. Stop.

Algorithm that terminates in finite time

1. Start.
2. Initialize count to 0.
3. Set a limit for the number of counts (let's say it is 10).
4. Enter the loop that iterates up to the specified limit.
5. Within the loop, increment the count by 1.
6. Repeat steps 4 and 5 until the specified limit is reached.
7. Stop.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

CHARACTERISTICS OF AN ALGORITHM

4. **Definiteness**: An algorithm must be **clear** and **unambiguous**.

Algorithm with ambiguity

1. Start.
2. Input two numbers a and b.
3. Perform some operation on a and b.
4. Output the result.
5. Stop

Algorithm without ambiguity

1. Start.
2. Input two numbers a and b.
3. Calculate the sum of a and b and store it in the variable sum.
4. Output the value of sum.
5. Stop.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

CHARACTERISTICS OF AN ALGORITHM

5. **Effectiveness**: An algorithm should take less time and space to execute.

Less effective algorithm

1. Start.
2. Input a positive integer n.
3. Initialize the variable sum to 0.
4. Enter the loop that iterates from 1 to n.
5. Add the current iteration value to sum.
6. Repeat steps 4 and 5.
7. Output the value of sum.
8. Stop.

Time complexity is $O(n)$.

More effective algorithm

1. Start.
2. Input a positive integer n.
3. Calculate the sum using the formula
$$sum = \frac{n(n + 1)}{2}$$
4. Output the value of sum.
5. Stop.

Time complexity is $O(1)$.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

SUMMARY OF CHARACTERISTICS OF AN ALGORITHM

1. 0 or more Input
2. At least one Output
3. Finiteness
4. Definiteness
5. Effectiveness

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

QUESTION

Determine what characteristic(s) is/are missing in the following algorithm:

1. Start
2. Ask the user to input two numbers.
3. Check if both inputs are valid numeric values.
4. If valid, perform some operation of your choice on two numbers.
5. Stop.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

Missing Characteristics

The algorithm is missing the following characteristics:

Definiteness (Precision) ✕

Step 4 says "**perform some operation of your choice**".

It does not clearly specify **which operation** (addition, subtraction, multiplication, etc.) should be performed.

Every step of an algorithm must be **clear and unambiguous**.

Output ✕

The algorithm performs an operation but **never displays or returns the result**.

Every algorithm should produce at least one output.

Correct Answer

Missing characteristics:

Definiteness (Precision) – The operation is not clearly specified.

Output – The algorithm does not display or return the result.

Improved Algorithm

1. Start
2. Input two numbers.
3. Check if both inputs are valid numeric values.
4. If valid, **add the two numbers**.
5. Display the sum.
6. Stop.

PREPARED BY:

K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

SIGNIFICANCE OF ALGORITHMS

- A bad algorithm can lead to longer execution time..

Bad Algorithm

1. Start.
2. Initialize the variable sum to 0.
3. Enter the loop that iterates from 1 to 10^{12} .
4. Add the current iteration value to sum.
5. Repeat steps 4 and 5.
6. Return the value of sum.
7. Stop.

Equivalent Program

```
long int calculate_sum()
{
    long int sum = 0, i;
    for(i=1; i<=1000000000000; i++)
    {
        sum += i;
    }
    return sum;
}
```

Number of instructions = $2 + 1 + 1000000000000 \times (1+1+1) + 1 = 3000000000004$

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

SIGNIFICANCE OF ALGORITHMS

- A bad algorithm can lead to **longer execution time**.

$$\begin{aligned}\text{Number of instructions} &= 1 + 1 + 1000000000000 \times (1+1+1) + 1 \\ &= 3000000000004\end{aligned}$$

Assume that a computer takes 1 second to execute y instructions, and $y = 100000000$.

Time to run y instructions = 1 second

Time to run 1 instruction = $1/y$ seconds

Time to run x instructions = x/y seconds

$$\begin{aligned}\text{Time to run } 3000000000004 \text{ instructions} &= \frac{3000000000004}{100000000} \approx 30000 \text{ seconds} \approx 500 \text{ minutes} \\ &\approx 8 \text{ hours}\end{aligned}$$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

SIGNIFICANCE OF ALGORITHMS

- A Good algorithm can lead to **shorter execution time..**

Good Algorithm

1. Start.
2. Initialize n to 10^{12} .
3. Calculate the sum using the formula
$$sum = \frac{n(n+1)}{2}$$
4. Output the value of sum.
5. Stop

Equivalent Program

```
long int calculate_sum() {  
    long int n = 1000000000000;  
    long int sum;  
    sum = (n * (n + 1)) / 2;  
    return sum;  
}
```

Number of instructions = 2 + 4 + 1
 = 7

SIGNIFICANCE OF ALGORITHMS

$$\begin{aligned}\text{Number of instructions} &= 2 + 4 + 1 \\ &= 7\end{aligned}$$

Assume that a computer takes 1 second to execute y instructions. Let's say $y = 100000000$.

Time to run y instructions = 1 second

Time to run 1 instruction = $1/y$ seconds

Time to run x instructions = x/y seconds

$$\text{Time to run 7 instructions} = \frac{7}{100000000} = 0.00000007 \text{ seconds}$$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

DECIDABLE VS. UNDECIDABLE PROBLEMS

- Decidable Problems
- Undecidable Problems
- Polynomial vs. Exponential Time

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

DECIDABLE PROBLEMS

- The problem for which **efficient algorithm** exists.
- Takes **polynomial time** or **less** to execute.



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

UNDECIDABLE PROBLEMS

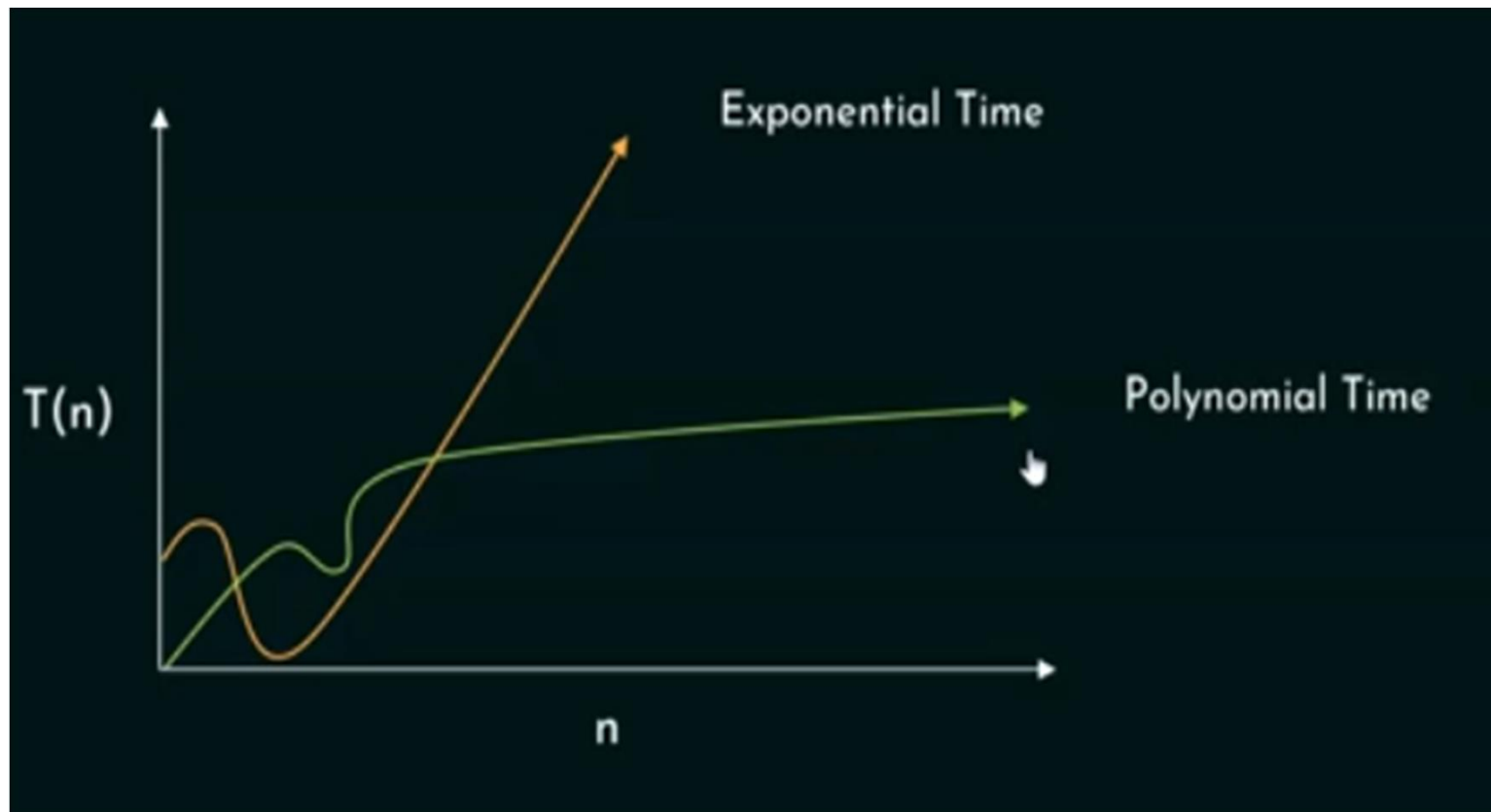
- The problem for which **no efficient algorithm** exists.
- Takes **exponential time** to execute.



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

POLYNOMIAL VS. EXPONENTIAL TIME

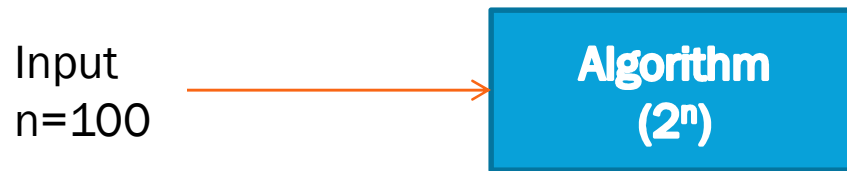


Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

THE NATURE OF UNDECIDABLE PROBLEMS

Let's say that the input size is 100.



The number of instructions = 2^{100}

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

THE NATURE OF UNDECIDABLE PROBLEMS

Consider the world's fastest computer capable of executing 2^{20} instructions per second.

In 1 year, the computer can execute

$$2^{20} * 60 * 60 * 24 * 365$$

$$= 2^{20} * 2^6 * 2^6 * 2^5 * 2^8$$

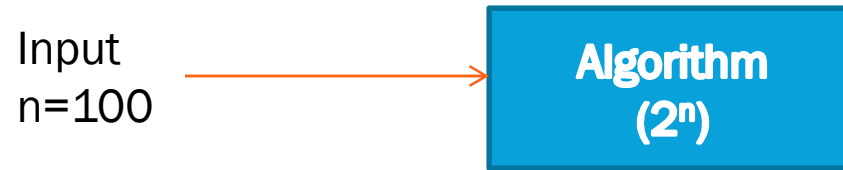
$$\approx 2^{45} \text{ instructions}$$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

THE NATURE OF UNDECIDABLE PROBLEMS

Let's say that the input size is 100.



$$2^{45} \longrightarrow 1 \text{ year}$$

$$2^{100} \frac{2^{100}}{2^{45}} = 2^{55} \text{ years} \longrightarrow$$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

POSTERIORI VS. PRIORI ANALYSIS

Introduction to Analysis of Algorithms

- Study of an algorithm's performance based on time and memory space consumption.
- The algorithm taking least time and memory space is preferred.

Two ways to analyze an algorithm:

1. Priori Analysis
2. Posteriori Analysis

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

PRIORI VS. POSTERIORI ANALYSIS

Priori Analysis	Posteriori Analysis
○ Estimation of time and memory space required by an algorithm before executing it on the system	○ Calculation of time and memory space required by an algorithm after executing it on the system.
○ Independent of the programming language.	○ Dependent on the programming language
○ Independent of the hardware.	○ Dependent on the hardware.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

QUIZ 1

Q1: The definiteness property of an algorithm states that

- a) an algorithm should take less time and memory.
- b) an algorithm must terminate in finite time.
- c) an algorithm must be clear and unambiguous.
- d) an algorithm must produce at least one output.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

QUIZ 1

Q1: The definiteness property of an algorithm states that

- a) an algorithm should take less time and memory.
- b) an algorithm must terminate in finite time.
- c) an algorithm must be clear and unambiguous.
- d) an algorithm must produce at least one output.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

QUIZ 1

Q2: An algorithm is an _____ concept, whereas a program is a _____ implementation.

- a) concrete, abstract
- b) abstract, concrete
- c) difficult, simple
- d) simple, difficult

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

QUIZ 1

Q2: An algorithm is an _____ concept, whereas a program is a _____ implementation.

- a) concrete, abstract
- b) abstract, concrete
- c) difficult, simple
- d) simple, difficult

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

QUIZ 1

Q3: Is it true that an algorithm depends upon the size of the RAM?

- a) Yes
- b) No

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

QUIZ 1

Q3: Is it true that an algorithm depends upon the size of the RAM?

a) Yes

b) No

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

QUIZ 1

Q4: Assume that the time to execute 1 operation by a computer is 10 ms. How much time the computer will take to execute 3 operations where $n = 5$?

- a) 243 ms
- b) 243 ms
- c) 9 seconds
- d) 2.43 seconds

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

QUIZ 1

Q4: Assume that the time to execute 1 operation by a computer is 10 ms. How much time the computer will take to execute 3 operations where $n = 5$?

- a) 243 ms
- b) 243 ms
- c) 9 seconds
- d) 2.43 seconds

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

QUIZ 1

Q5: In priori analysis, we _____ time and memory space.

- a) estimate
- b) calculate
- c) measure
- d) ignore

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

QUIZ 1

Q5: In priori analysis, we _____ time and memory space.

- a) estimate
- b) calculate
- c) measure
- d) ignore

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

QUIZ 1

Q6: A decidable problem is a problem for which _____ exists.

- a) no efficient algorithm
- b) efficient algorithm
- c) no algorithm
- d) an algorithm

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

QUIZ 1

Q6: A decidable problem is a problem for which _____ exists.

- a) no efficient algorithm
- b) efficient algorithm
- c) no algorithm
- d) an algorithm

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

QUIZ 1

Q7: Which of the following statements best describes posteriori analysis?

- a) It is based on theoretical predictions.
- b) It involves analyzing the algorithm's performance before implementation.
- c) It relies on actual performance data collected by running the algorithm.
- d) It is independent of the algorithm's actual performance.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

QUIZ 1

Q7: Which of the following statements best describes posteriori analysis?

- a) It is based on theoretical predictions.
- b) It involves analyzing the algorithm's performance before implementation.
- c) It relies on actual performance data collected by running the algorithm.
- d) It is independent of the algorithm's actual performance.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

QUIZ 1

Q8: An undecidable problem terminates in

- a) polynomial time.
- b) logarithmic time.
- c) constant time.
- d) exponential time.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

QUIZ 1

Q8: An undecidable problem terminates in

- a) polynomial time.
- b) logarithmic time.
- c) constant time.
- d) exponential time.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

QUIZ 1

Q9: How many operations does the following statement have?

$$x = y * z - 3 / p + 5$$

- a) 5
- b) 7
- c) 4
- d) 11

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

QUIZ 1

Q9: How many operations does the following statement have?

$$x = y * z - 3 / p + 5$$

- a) 5
- b) 7
- c) 4
- d) 11

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

QUIZ 1

Q10: Which of the following are the characteristics of an algorithm?

- a) Finiteness, ambiguity, and speed.
- b) Finiteness, definiteness, and effectiveness.
- c) Definiteness, ten inputs, and no output.
- d) None of the above.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

QUIZ 1

Q10: Which of the following are the characteristics of an algorithm?

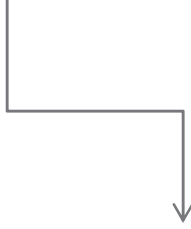
- a) Finiteness, ambiguity, and speed.
- b) Finiteness, definiteness, and effectiveness.
- c) Definiteness, ten inputs, and no output.
- d) None of the above.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

IMPORTANCE OF ALGORITHM ANALYSIS

- There are **many algorithms** to solve a problem.
- Requirement is to find the **most efficient algorithm** to solve the problem.



Efficiency in terms of
time and memory space
consumption.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

EXAMPLE – FINDING THE KEY

Approach 1

1	15	28	35	42	56	78	84	90
---	----	----	----	----	----	----	----	----



Key: 84

Total comparisons: 8

Approach 1

1	15	28	35	42	56	78	84	90
---	----	----	----	----	----	----	----	----



Total comparisons: 3

Tuesday, August 18, 2026

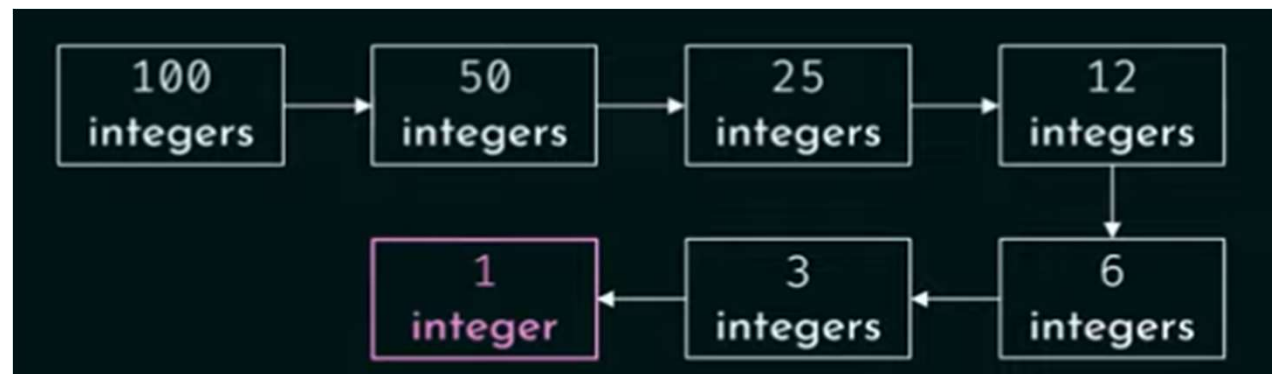
PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

EXAMPLE – FINDING THE KEY

For 100 integers

Linear Search: 100 Comparisons

Binary Search: 7 comparisons



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

IMPORTANCE OF GROWTH RATE

Linear Search	Binary Search
100 items ↓ 100 comparisons (100 ms)	100 items ↓ 7 comparisons (7 ms)
1 billion items ↓ 1 billion comparisons (11 days)	1 billion items ↓ 30 comparisons (30 ms)

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

IMPORTANCE OF GROWTH RATE

- As the size of the input grows, speed of the **binary search grows drastically.**
- One example is not enough to analyze algorithms.
- Important to know **how fast an algorithm grows** w.r.t the **input size.**

Definition: Rate at which **running time** increases as a **function of input** is called the **growth rate.**

Can be measured using
the Big O Notation

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

INTRODUCTION TO BIG O NOTATION

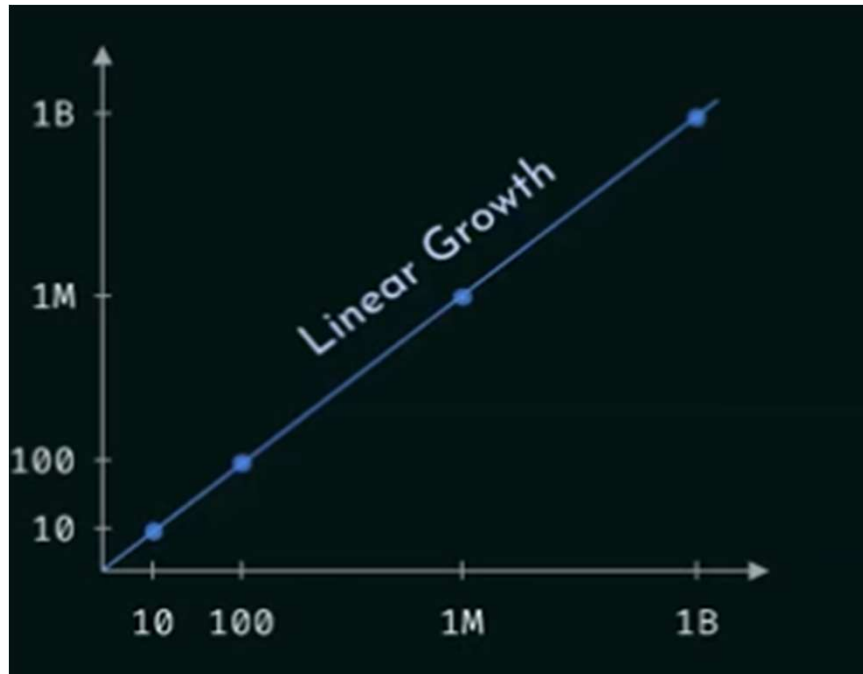
- Way to describe the performance of an algorithm w.r.t to the input size.
- In linear search, the number of comparisons is directly proportional to the input size.
- Linear Search:

Number of Items		Number of Comparisons
10		10
100		100
1 Million	$\propto$	1 Million
N		n

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

BIG O NOTATION



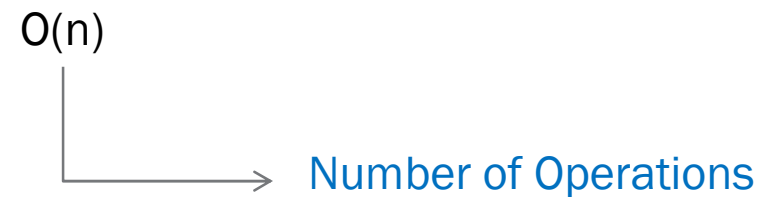
Number of Items		Number of Comparisons
10		10
100	$\propto$	100
1 Million		1 Million
N		n

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

BIG O NOTATION

- Big O notation doesn't tell speed in seconds.
- It helps in **comparing** the **number of operations**



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

BIG O NOTATION AND BINARY SEARCH

- In **binary search**, every time **half of the list is eliminated**.

Binary Search

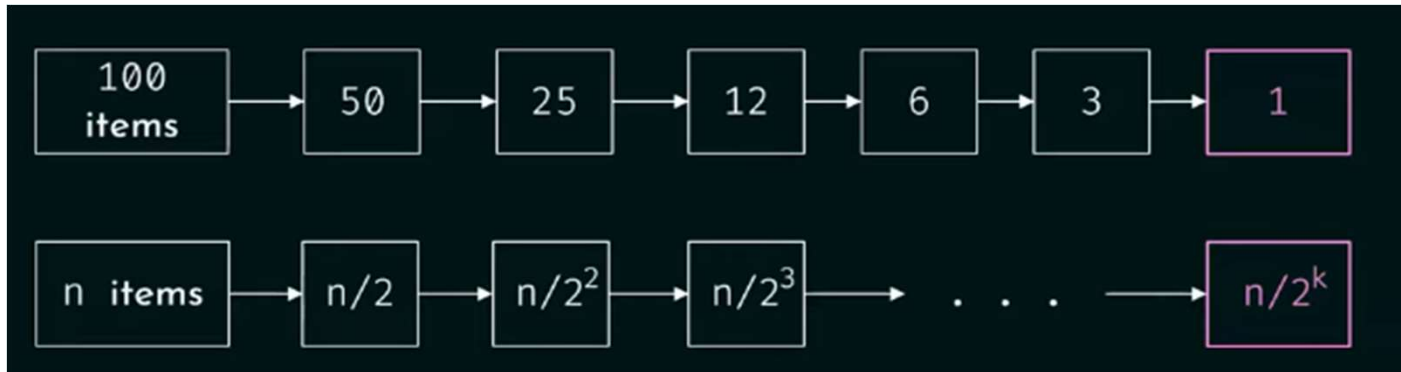
Number of Items	Number of Comparisons
10	4
100	7
1 Billion	30
n	?

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

BIG O NOTATION AND BINARY SEARCH

- In binary search, every time half of the list is eliminated.



$$\frac{n}{2^k} = 1 \Rightarrow 2^k = n \Rightarrow \log_2 2^k = \log_2 n \Rightarrow k = \log_2 n$$

Running time: $O(\log n)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

BIG O NOTATION AND BINARY SEARCH

- In **binary search**, every time **half of the list is eliminated**.

Binary Search

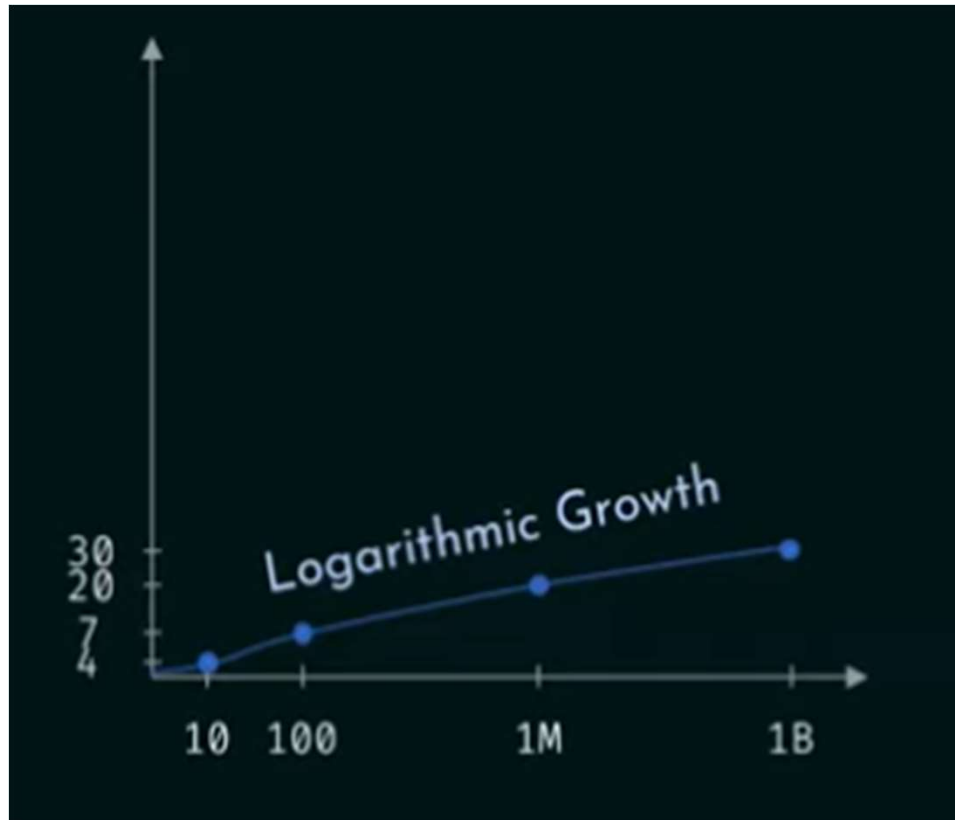
Number of Items	Number of Comparisons
10	4
100	7
1 Billion	30
n	$\log n$

Running time: $O(\log n)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

GROWTH RATE OF BINARY SEARCH

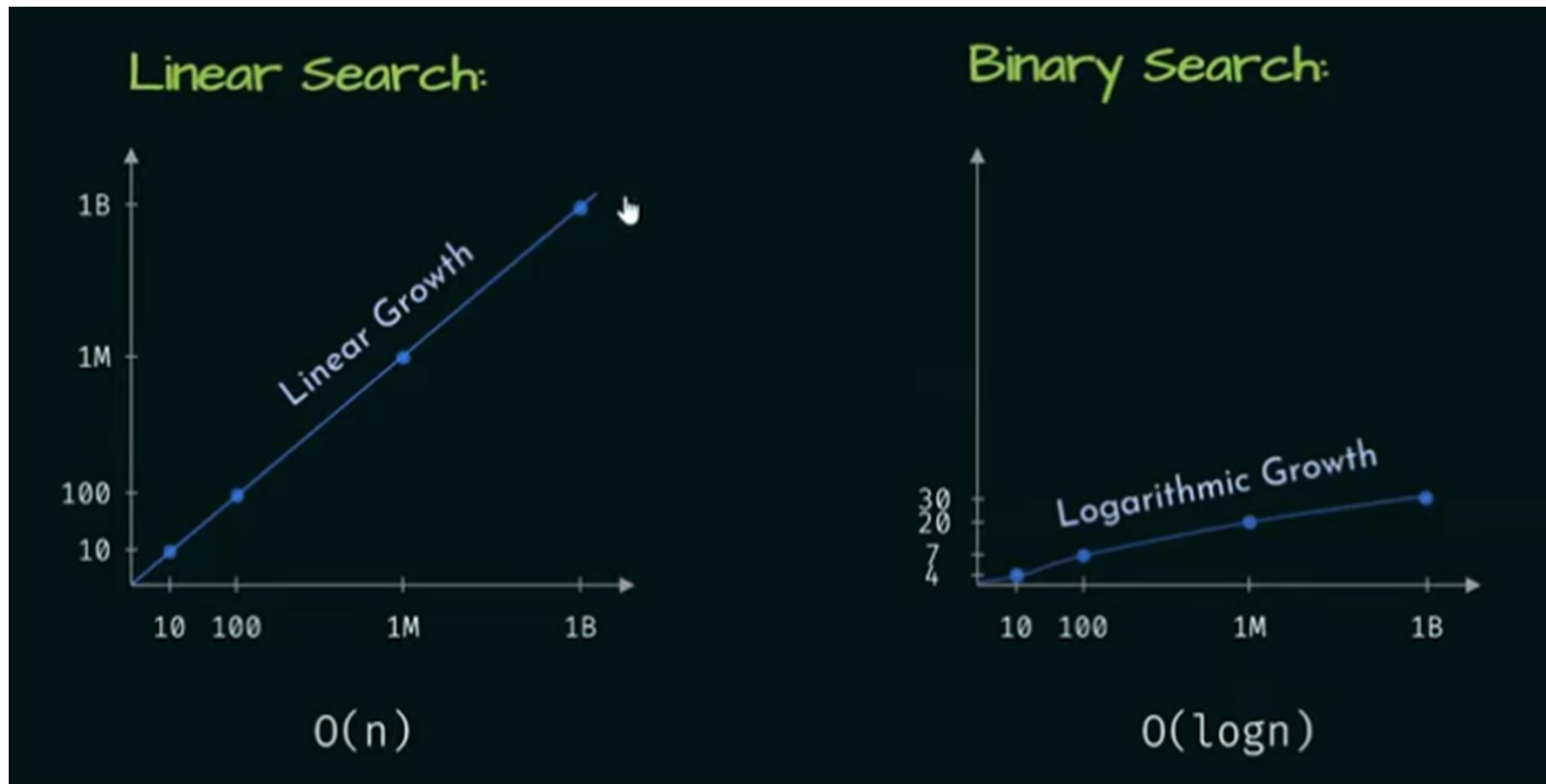


Number of Items	Number of Comparisons
10	4
100	7
1 Billion	30
n	log n

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

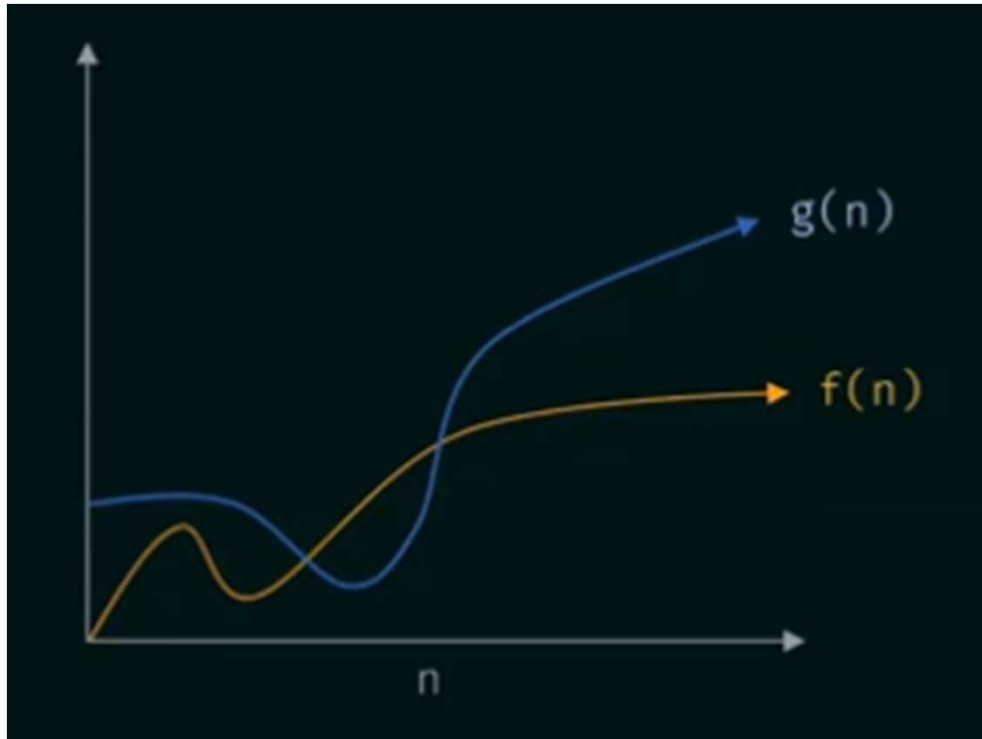
GROWTH RATE COMPARISON



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

FORMAL DEFINITION OF BIG O NOTATION



$g(n)$ is asymptotically bigger than $f(n)$

$$f(n) = O(g(n))$$

Tuesday, August 18, 2026

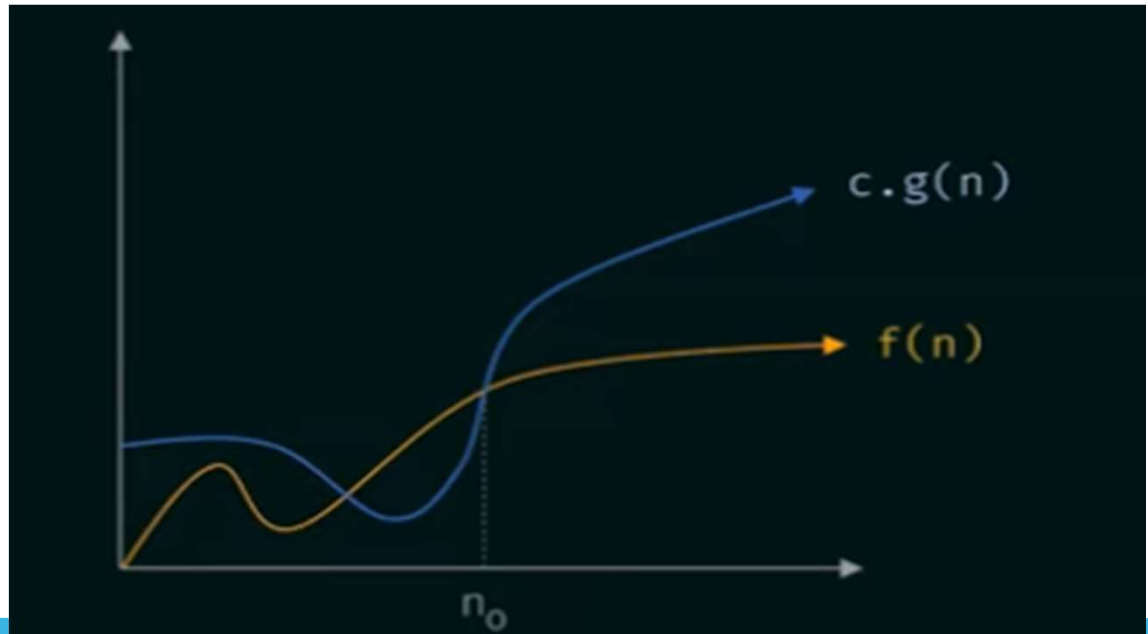
PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

FORMAL DEFINITION OF BIG O NOTATION

- Definition: Assuming $f(n)$ and $g(n)$ are **non-negative functions**.

$f(n) = O(g(n))$ iff $f(n) \leq c \cdot g(n)$ for all values of n where $n \geq n_0$ and c and n are constants.

$g(n)$ is the tight upper bound of $f(n)$



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

BIG O NOTATION (SOLVED PROBLEMS)

Problem 1: Assume that $f(n) = 5n + 50$ and $g(n) = n$. Is $f(n) = O(g(n))$?

Solution: According to the definition of Big O notation:

$f(n) = O(g(n))$ iff $f(n) \leq c.g(n)$ for all values of n where $n \geq n_0$
and c and n are constants.

Assuming $c = 6$

5n+50		6n
n=10	100	60
n=20	150	120
n=50	300	300
n=51	305	306

$5n + 50 = O(n)$
for $c = 6$ and $n \geq 50$

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

BIG O NOTATION (SOLVED PROBLEMS)

Problem 2: Assume that $f(n) = 5n + 50$ and $g(n) = \log_{10} n$.
Is $g(n)$ upper bound of $f(n)$?

Solution: According to the definition of Big O notation:

$f(n) = O(g(n))$ iff $f(n) \leq c.g(n)$ for all values of n where $n \geq n_0$
and c and n are constants

Assuming $c = 1000$

$5n+50$		$1000 \log_{10} n$
$n=10$	100	1000
$n=10^2$	550	2000
$n=10^3$	5050	3000
$n=10^4$	50050	4000

$g(n)$ is not an upper bound of $f(n)$.

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

BIG O NOTATION (SOLVED PROBLEMS)

Problem 3: Find the upper bound for $f(n) = 3n + 8$.

Solution:

Step 1: Identify the dominant term.

3n		8
n=10	30	8
n=100	300	8
n=1000	3000	8

$3n$ is the dominant term.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

BIG O NOTATION (SOLVED PROBLEMS)

Step 2: Choose $g(n)$ according to the dominant term.

$n, n \log n, n^2, n^3, \dots$



$g(n)$

Step 3: Apply the Big O definition.

According to the definition of Big O notation:

$$f(n) = O(g(n)) \text{ iff } f(n) \leq c \cdot g(n)$$

for all values of n where $n \geq n_0$ and c and n_0 are constants.

Assuming $c = 4$

$3n + 8 \leq 4n$ is true.

n_0

$\therefore 3n + 8 = O(n)$ for $c = 4$ and $n = 8$

	$3n+8$	$4n$
$n=7$	29	28
$n=8$	32	32
$n=10$	38	40
$n=100$	308	400

PREPARED BY:

K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

BIG O NOTATION (SOLVED PROBLEMS)

Problem 4: Find the upper bound for $f(n) = n^2 + 10$

Solution: Step 1: Identify the dominant term.

n^2		10
n=10	100	10
n=100	10000	10
n=1000	3000	10

n^2 is the dominant term.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

BIG O NOTATION (SOLVED PROBLEMS)

Solution:

Step 2: Choose $g(n)$ according to the dominant term.

$n^2, n^3, 2^n$

$g(n)$

Step 3: Apply the Big O definition.

According to the definition of Big O notation:

$f(n) = O(g(n))$ iff $f(n) \leq c \cdot g(n)$

for all values of n where $n \geq n_0$ and c and n_0 are constants.

Assuming $c = 2$

$n + 10 \leq 2n$ is true

$n_0 \rightarrow$

$\therefore n^2 + 10 = O(n^2)$ for $c = 2$ and $n_0 = 4$

	$n^2 + 10$	$2n^2$
$n=1$	11	2
$n=2$	14	8
$n=3$	19	18
$n=4$	26	32

ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

BIG O NOTATION (SOLVED PROBLEMS)

Problem 5: Find the upper bound for $f(n) = 2n^3 - 2n^2$.

Solution: Dominant term: $2n^3$ Assuming $g(n) = n^3$

According to the definition of Big O notation:

$$f(n) = O(g(n)) \text{ iff } f(n) \leq c \cdot g(n)$$

for all values of n where $n \geq n_0$ and c and n_0 are constants.

Assuming $c = 2$
 $2n^3 - 2n^2 \leq 2n^3$ is true $\longrightarrow$
 n_0

	$2n^3 - 2n^2$	$2n^3$
$n=0$	0	2
$n=2$	8	16
$n=3$	36	54
$n=4$	96	128

$\therefore 2n^3 - 2n^2 = O(n^3)$ for $c = 2$ and $n_0 = 1$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

BIG O NOTATION (SOLVED PROBLEMS)

Problem 6: Find the upper bound for $f(n) = n^4 + 100n^2 + 35$.

Solution: Dominant term: n^4 Assuming $g(n) = n^4$

According to the definition of Big O notation:

$$f(n) = O(g(n)) \text{ iff } f(n) \leq c \cdot g(n)$$

for all values of n where $n \geq n_0$ and c and n_0 are constants.

Assuming $c = 2$

$$n^4 + 100n^2 + 35 \leq 2n^4 \quad ??$$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

BIG O NOTATION (SOLVED PROBLEMS)

Problem 7: Find the upper bound for $f(n) = 2^n + 3n^3$.

Solution: Dominant term: 2^n Assuming $g(n) = 2^n$

According to the definition of Big O notation:

$$f(n) = O(g(n)) \text{ iff } f(n) \leq c \cdot g(n)$$

for all values of n where $n \geq n_0$ and c and n_0 are constants.

Assuming $c = 2$

$$2^n + 3n^3 \leq 2^{n+1} \text{ is true}$$

$2^n + 3n^3$		2^{n+1}
$n=10$	4024	2048
$n=11$	6041	4096
$n=14$	24616	32768

$\therefore 2^n + 3n^3 = O(2^n)$ for $c = 2$ and $n_0 = 14$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

BIG O NOTATION (SOLVED PROBLEMS)

Problem 7: Find the upper bound for $f(n) = 300$

Solution: Dominant term: 300

Assuming $g(n) = 1$

According to the definition of Big O notation:

$$f(n) = O(g(n)) \text{ iff } f(n) \leq c \cdot g(n)$$

for all values of n where $n \geq n_0$ and c and n_0 are constants.

Assuming $c = 300$

$300 \leq 300$ is true

	$2^n + 3n^3$		2^{n+1}
	$n=10$	4024	2048
	$n=11$	6041	4096
$\xrightarrow{n_0}$	$n=14$	24616	32768

$\therefore 300 = O(1)$ for $c = 300$ and $n_0 = 1$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

COMMON BIG O RUNTIMES

- $O(\log n)$, also known as **log time**. Example: binary search.
- $O(n)$, also known as **linear time**. Example: linear search.
- $O(n \log n)$. Example: quicksort.
- $O(n^2)$, also known as **polynomial time**. Example: selection sort.
- $O(n!)$, also known as **exponential time**. Example: traveling salesman.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

VISUALIZATION OF RUNTIMES

n	logn	n	nlogn	n^2	n!
10	1	10	10	100	362800
20	1.3	20	26	400	2.432902×10^{18}
100	2	100	200	10000	9.332622×10^{157}

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

FUNCTIONS IN ASYMPTOTIC NOTATIONS

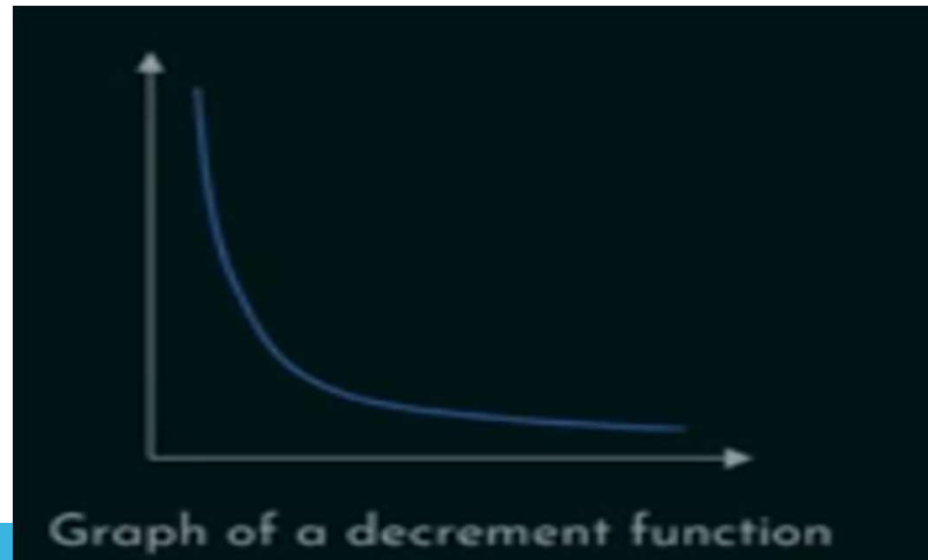
Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

DECREMENT FUNCTIONS

Definition: A function in which the denominator is bigger than the numerator.

Examples: $\frac{c}{n}, \frac{n}{n^2}, \frac{n}{2^n}, \frac{n^2}{3^n}, \dots$



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

DECREMENT FUNCTIONS

Problem: Arrange the following functions in the order of decrease from slowest to fastest: :

$$\frac{100}{n}, \frac{n}{2^n}, \frac{n}{n^2}, \frac{n^2}{3^n}, \frac{n^3}{3^n}$$

Solution: Step 1: Simplify the fractions.

$$\frac{100}{n}, \frac{n}{2^n}, \frac{1}{n}, \frac{n^2}{3^n}, \frac{n^3}{3^n}$$

Step 2: Compare the denominators and arrange. $n < 2^n < 3^n$

$$\frac{100}{n}, \frac{1}{n}, \frac{n}{2^n}, \frac{n^2}{3^n}, \frac{n^3}{3^n}$$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

DECREMENT FUNCTIONS

Step 3: Compare the numerators and arrange.

$$\frac{100}{n}, \frac{1}{n}, \frac{n}{2^n}, \frac{n^3}{3^n}, \frac{n^2}{3^n}$$

Slowest

Fastest

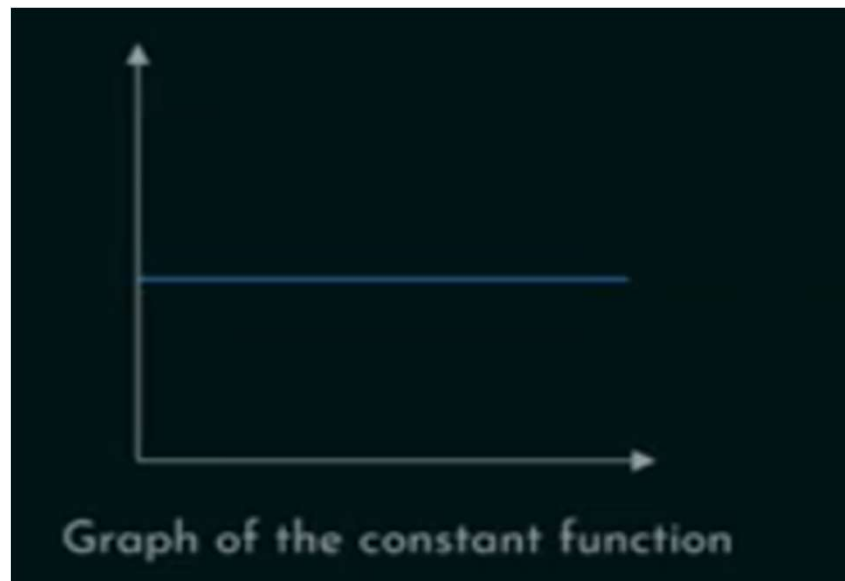
Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

CONSTANT FUNCTIONS

Definition: A function parallel to x-axis.

Examples: 100, 500392, 1 billion, etc.



$$f(n)=c$$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

COMPARISON WITH DECREMENT FUNCTION

Constant function is asymptotically bigger than the decrement function.

Problem: $f(n) = \frac{100}{n}$ and $g(n) = 10000$. is $f(n) = O(g(n))$?

Solution:

According to the definition of Big O notation:

$$f(n) = O(g(n)) \text{ iff } f(n) \leq c.g(n)$$

for all values of n where $n \geq n_0$ and c and n are constants.

Assuming $c = 1/10000$

$$\frac{100}{n} \leq 1 \text{ is true}$$

$\therefore g(n)$ is asymptotically bigger than $f(n)$.

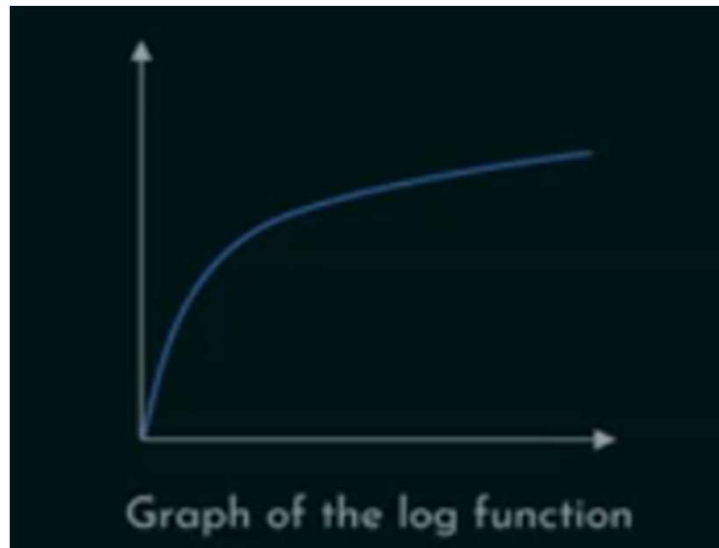
100/n		1
n=2	50	1
n=4	25	1
n=100	1	1
n=1000	0.1	1

K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

LOGARITHMIC FUNCTIONS

Definition: A function that grows positively, but with slower growth rate.

Examples: $\log n$, $(\log n)^{10}$, $\log \log n$, $\log \log n \cdot \log \log \log n$, etc.



$$f(n) = \log n$$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

LOGARITHMIC FUNCTIONS

Logarithmic function is asymptotically bigger than the constant function.

Problem: $f(n) = 100$ and $g(n) = \log n$. Is $f(n) = O(g(n))$?

Solution: According to the definition of Big O notation:

$$f(n) = O(g(n)) \text{ iff } f(n) \leq c \cdot g(n)$$

for all values of n where $n \geq n_0$ and c and n_0 are constants.

Assuming $c = 1$
 $100 \leq \log_{10} n$ is true.

100		$\log_{10} n$
$n=10$	100	1
$n=10^{100}$	100	100
$n=10^{1000}$	100	1000

$\therefore g(n)$ is asymptotically bigger than $f(n)$

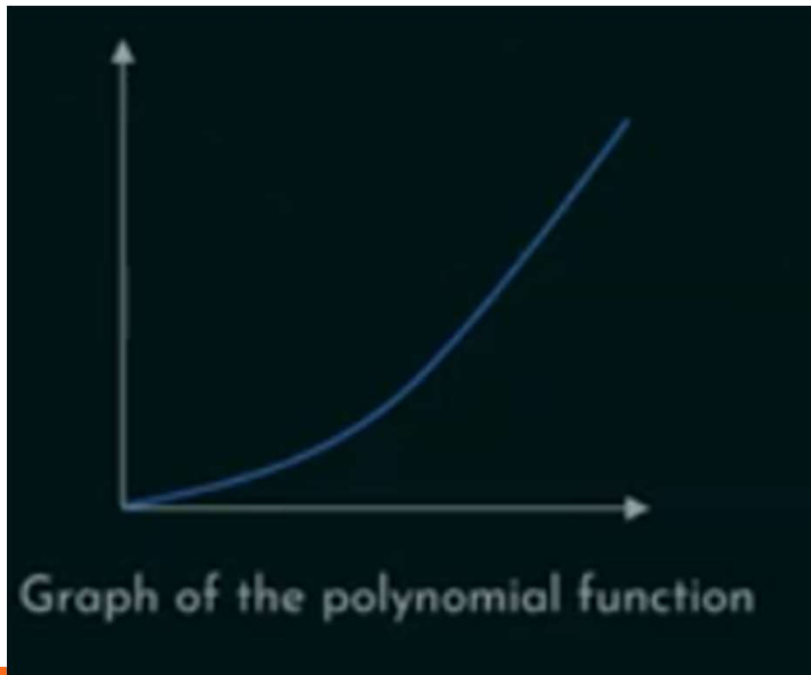
Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

POLYNOMIAL FUNCTIONS

Definition: A function whose **growth rate** increases **polynomially**.

Examples $n^{0.1}$, $\sqrt{n}$, n^2 , $n \log n$, n^k , etc



$$f(n) = n^k$$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

POLYNOMIAL FUNCTIONS

Polynomial function is asymptotically bigger than the logarithmic function.

Problem: $f(n) = \log_{10} n$ and $g(n) = \sqrt{n}$. Is $f(n) = O(g(n))$?

Solution: According to the definition of Big O notation:

$$f(n) = O(g(n)) \text{ iff } f(n) \leq c \cdot g(n)$$

for all values of n where $n \geq n_0$ and c and n_0 are constants.

Assuming $c = 1$

$$\log_{10} n \leq \sqrt{n} \text{ is true.}$$

$\therefore g(n)$ is asymptotically bigger than $f(n)$

100		$\log_{10} n$
$n=10$	1	3.16
$n=100$	2	10
$n=10000$	4	100

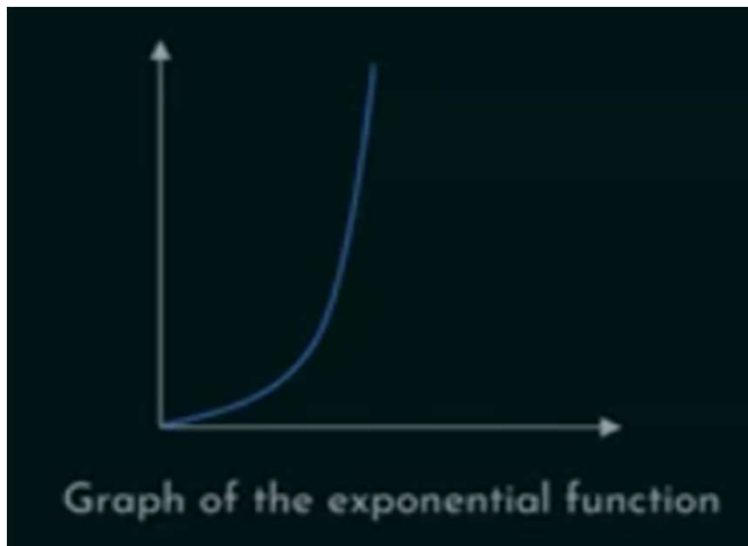
Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

EXPONENTIAL FUNCTIONS

Definition: A function whose growth rate increases rapidly.

Examples : 2^n , 3^n , $n!$, n^n , a^n , etc.



$$f(n) = a^n$$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

EXPONENTIAL FUNCTIONS

Exponential function is asymptotically bigger than the polynomial function.

Problem: $f(n) = n^2$ and $g(n) = 2^n$. Is $f(n) = O(g(n))$?

Solution: According to the definition of Big O notation:

$$f(n) = O(g(n)) \text{ iff } f(n) \leq c \cdot g(n)$$

for all values of n where $n \geq n_0$ and c and n_0 are constants.

Assuming $c = 1$

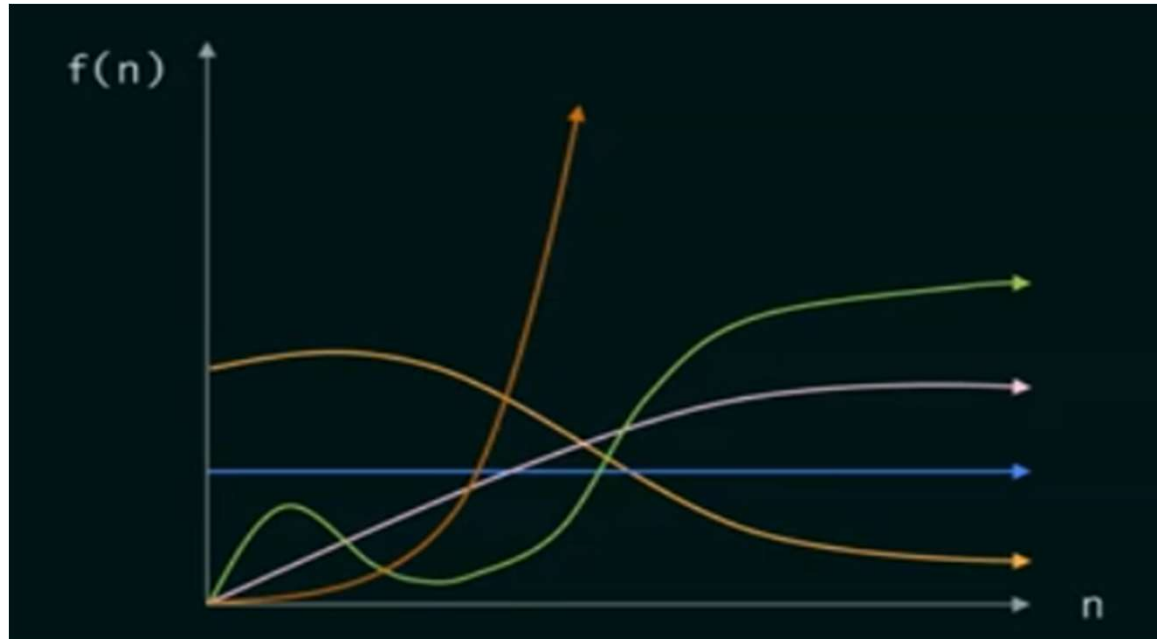
$n^2 \leq 2^n$ is true.

$\therefore g(n)$ is asymptotically bigger than $f(n)$.

n^2		2^n
$n=10$	100	1024
$n=20$	400	1048576
$n=30$	900	1073741824

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

COMPARISON



Decr functions < Const Functions < Log Functions < Poly Functions < Exp Functions

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

PROBLEM 1

Problem: Write the following functions in asymptotically increasing order:
 n^3 , $n^{3.1}$, $n^2 \log n$, $\log n$, $(\log n)^{10}$, 100, 1 billion, $(0.3)^n$, and $(1.5)^n$.

Solution:

Decrement Function:	$(0.3)^n$
Constant Functions:	100, 1 billion
Logarithmic Functions:	$\log n$, $(\log n)^{10}$
Polynomial Functions:	$n^2 \log n$, n^3 , $n^{3.1}$
Exponential Function:	$(1.5)^n$

Decr functions < Const Functions < Log Functions < Poly Functions < Exp Functions

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

PROBLEM 1

Problem: Write the following functions in asymptotically increasing order:
 n^3 , $n^{3.1}$, $n^2 \log n$, $\log n$, $(\log n)^{10}$, 100, 1 billion, $(0.3)^n$, and $(1.5)^n$.

Solution: $(0.3)^n < 100 < 1 \text{ billion} < \log n < (\log n)^{10} < n^2 \log n < n^3 < n^{3.1} < (1.5)^n$

Decr functions < Const Functions < Log Functions < Poly Functions < Exp Functions

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

PROBLEM 2

Problem: Which of the following function is asymptotically bigger?

$$f(n) = \log n \text{ and } g(n) = \log \log n$$

n	log n	Log log n
10^{10}	10	1
10^{100}	100	2
$10^{10^{10}}$	10^{10}	10

$f(n)$ is asymptotically bigger than $g(n)$.

$$g(n) = O(f(n))$$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

PROBLEM 3

Problem: Which of the following function is asymptotically bigger?

$$f(n) = \log n \text{ and } g(n) = (\log \log n)^{10}$$

n	log n	Log log n
10^{10}	10	1
10^{100}	100	1024
$10^{10^{10}}$	10^{10}	10^{10}
$10^{10^{100}}$	10^{100}	100^{10}

$f(n)$ is asymptotically bigger than $g(n)$.

$$g(n) = O(f(n))$$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

BIG OMEGA NOTATION

- Describes the lower bound of an algorithm.
- Used to express the best case running time of an algorithm.

Definition: Assuming $f(n)$ and $g(n)$ are non-negative functions.

$f(n) = \Omega(g(n))$ iff $f(n) \geq c \cdot g(n)$ for some $c > 0$ and for all values of n where $n \geq n_0$ and c and n_0 are constants.

$g(n)$ is the asymptotic tight lower bound of $f(n)$

Tuesday, August 18, 2026

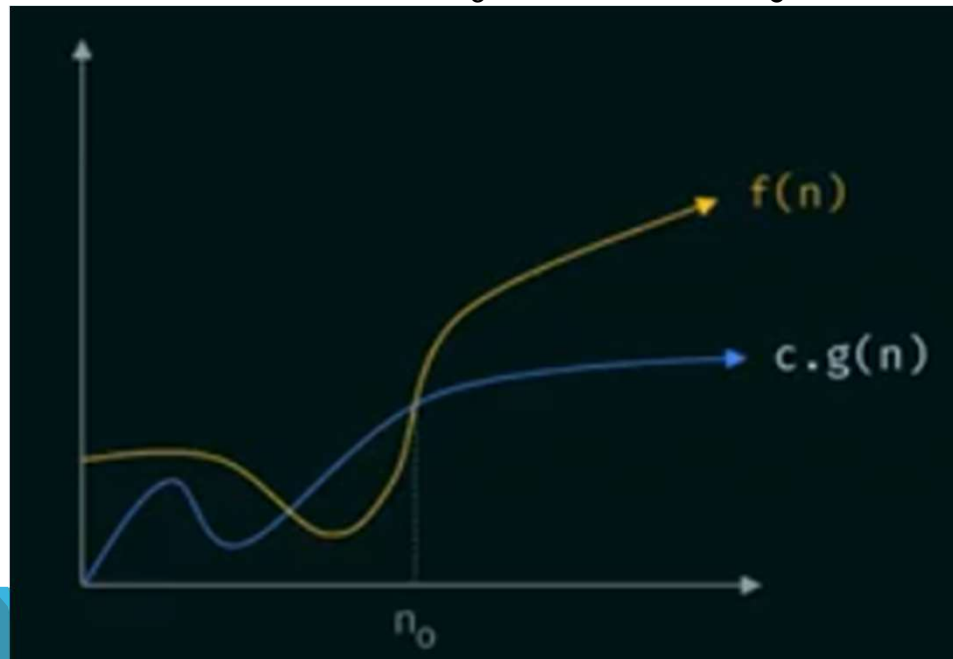
PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

102

BIG OMEGA NOTATION

Definition: Assuming $f(n)$ and $g(n)$ are non-negative functions.

$f(n) = \Omega(g(n))$ iff $f(n) \geq c \cdot g(n)$ for some $c > 0$ and for all values of n where $n \geq n_0$ and c and n_0 are constants.



Tuesday, August 18, 2026

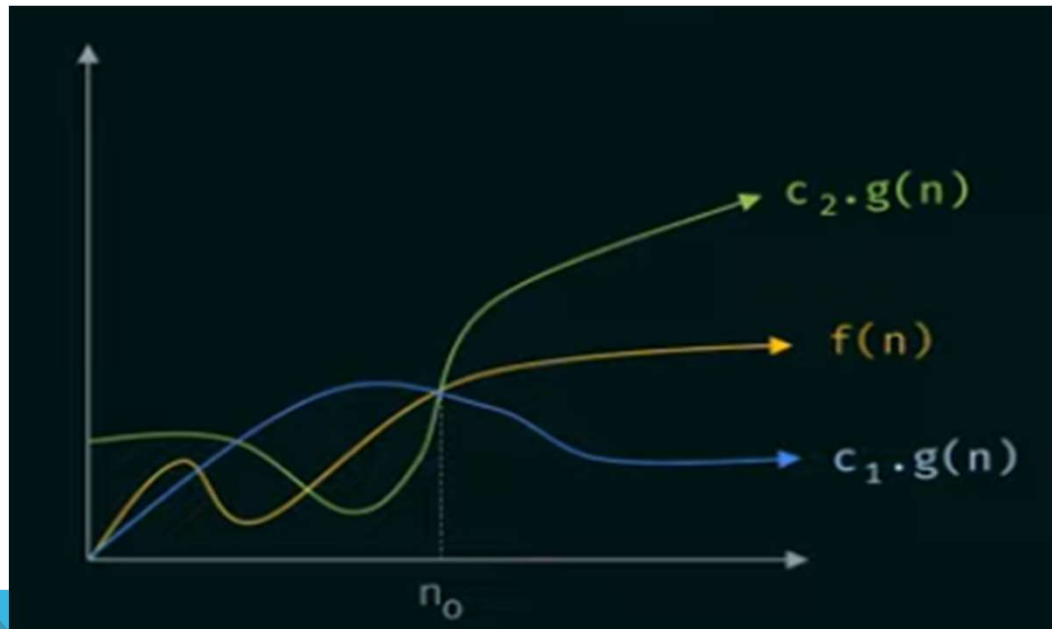
PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

BIG THETA NOTATION

Definition: Assuming $f(n)$ and $g(n)$ are non-negative functions.

$f(n) = \theta(g(n))$ iff $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all values of n where $n \geq n_0$ and c_1 , c_2 and n_0 are constants.

$g(n)$ is the asymptotic tight bound of $f(n)$



It's used when an algorithm's upper and lower bounds are the same, meaning the algorithm's time complexity is tightly bounded.

Dr. Ravi K. S. N. (Ph.D.),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

BIG OMEGA NOTATION PROBLEM 1

Problem: Find the lower bound for $f(n) = 10n^2 + 5$?

Solution:

Dominant Term: $10n^2$

Assuming $g(n) = n^2$

According to the definition of Big Omega notation:

$f(n) = \Omega(g(n))$ iff $f(n) \geq c \cdot g(n)$ for some $c > 0$ and for all values of n where $n \geq n_0$ and c and n_0 are constants

Assuming $c = 10$

$10n^2 + 5 \geq 10n^2$ is true for all $n \geq 1$.

$\therefore 10n^2 + 5 = \Omega(n^2)$ for $c = 10$ and $n_0 = 1$.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

BIG THETA NOTATION – SOLVED PROBLEM

Problem: Show that $f(n) = n^3 + 3n^2 = \theta(n^3)$

Solution: According to the definition of Big Theta notation:

$f(n) = \theta(g(n))$ iff $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all values of n where $n \geq n_0$ and c_1 , c_2 and n_0 are constants.

1. $f(n) \leq c_2 \cdot g(n)$

Assuming $c_2 = 2$

$$n^3 + 3n^2 \leq 2n^3 \quad \text{is true for } n \geq 3.$$

2. $f(n) \geq c_1 \cdot g(n)$

Assuming $c_1 = 1$

$$n^3 + 3n^2 \geq n^3 \text{ is true for } n \geq 1.$$

$$\therefore n^3 + 3n^2 = \theta(n^3) \text{ for } c_1 = 1, c_2 = 2 \text{ and } n_0 = 3$$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

THE LITTLE O NOTATION

Definition: Assuming $f(n)$ and $g(n)$ are **non-negative functions**.

$f(n) = o(g(n))$ iff $f(n) < c.g(n)$ for all values of $c > 0$

and for all values of n where $n \geq n_0$ and c and n_0 are constants.

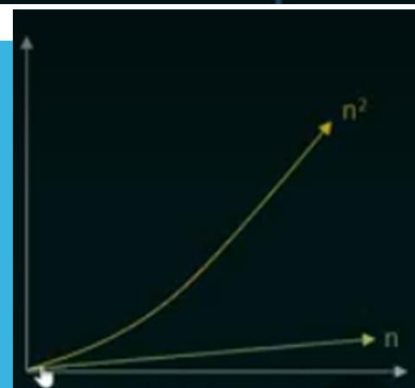
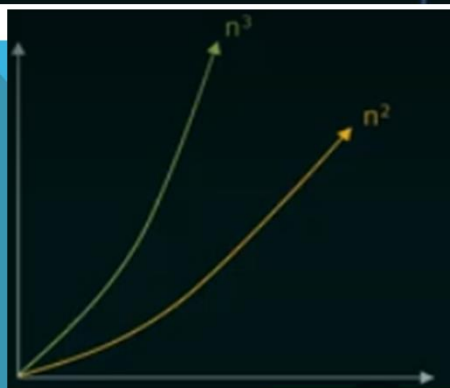
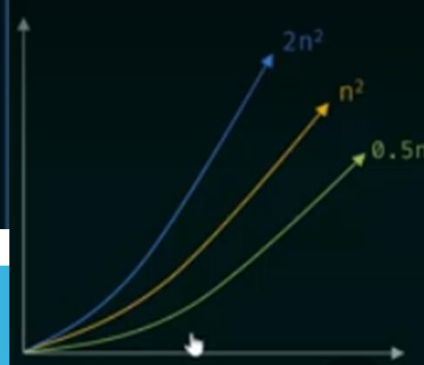
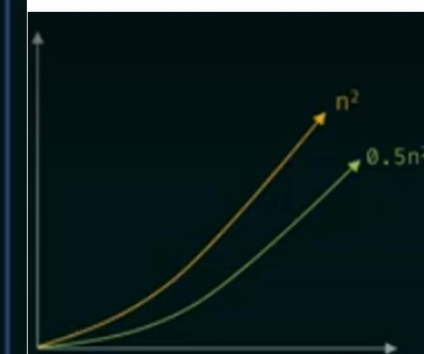
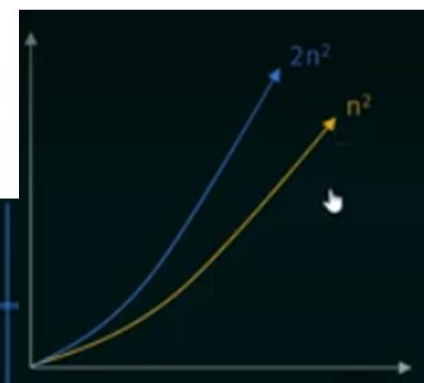
Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

107

SUMMARY

Notation	Definition	Meaning	Example
O	$f(n) \leq c \cdot g(n)$	Upper bound	$n^2 = O(n^2)$
Ω	$f(n) \geq c \cdot g(n)$	Lower bound	$n^2 = \Omega(n^2)$
θ	$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$	Tight bound	$n^2 = \theta(n^2)$
o	$f(n) < c \cdot g(n)$	Strict upper bound	$n^2 = o(n^3)$
ω	$f(n) > c \cdot g(n)$	Strict lower bound	$n^2 = \omega(n)$



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

PROBLEMS ON ASYMPTOTIC NOTATIONS

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

PROBLEM 1

Problem: Let $f(n) = n^2 \log n$ and $g(n) = n(\log n)^{10}$ be two positive functions of n . Which of the following statements is correct?

- (A) $f(n) = O(g(n))$ and $g(n) \neq O(f(n))$.
- (B) $g(n) = O(f(n))$ and $f(n) \neq O(g(n))$.
- (C) $f(n) \neq O(g(n))$ and $g(n) \neq O(f(n))$.
- (D) $f(n) = O(g(n))$ and $g(n) = O(f(n))$. [GATE 2001]

Ans: B

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

PROBLEM 2

Problem: Consider the following three claims:

i. $(n+k)^m = \theta(n^m)$ where k and m are constants.

ii. $2^{n+1} = O(2^n)$

iii. $2^{2n+1} = O(2^n)$

Which of the following claims are correct?

(A) i and ii (B) i and iii (C) ii and iii (D) i, ii, and iii

[GATE 2003]

Ans : A

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

PROBLEM 3

Problem: Let $f(n)$, $g(n)$, and $h(n)$ be functions defined for positive integers such that $f(n) = O(g(n))$, $g(n) \neq O(f(n))$, $g(n) = O(h(n))$, and $h(n) = O(g(n))$.

Which one of the following statements is false?

- (A) $f(n) + g(n) = O(h(n) + h(n))$ (B) $f(n) = O(h(n))$
(C) $h(n) \neq O(f(n))$ (D) $f(n)h(n) \neq O(g(n)h(n))$

[GATE IT 2004]

Ans : D

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

PROBLEM 4

Problem: Consider the following functions:

$$f(n) = 2^n \quad g(n) = n! \quad h(n) = n^{\log n}$$

Which of the following statements about the asymptotic behaviors of $f(n)$, $g(n)$ and $h(n)$ is true?

(A) $f(n) = O(g(n)); g(n) = O(h(n))$

(B) $f(n) = \Omega(g(n)); g(n) = O(h(n))$

(C) $g(n) = O(f(n)); h(n) = O(f(n))$

(D) $h(n) = O(f(n)); g(n) = \Omega(f(n))$

[GATE 2011]

Ans : D

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

PROBLEM 5

Problem: Consider the following functions:

$$f_1 = 10^n \quad f_2 = n^{\log n} \quad f_3 = n^{\sqrt{n}}$$

Which of the following options arranges the functions in the increasing order of asymptotic growth rate?

- (A) f_3, f_2, f_1
- (B) f_3, f_1, f_2
- (C) f_1, f_2, f_3
- (D) f_2, f_3, f_1

[GATE 2021]

Ans : D

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

PROBLEM 6

Problem: Let f and g be functions of natural numbers given by $f(n) = n$ and $g(n) = n^2$. Which of the following statements is/are true?

(A) $f \in O(g)$

(B) $f \in \Omega(g)$

(C) $f \in o(g)$

(D) $f \in \theta(g)$

[GATE 2023]

Ans: A,C

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

QUIZ

Q1: What role does algorithm analysis play in the early stages of software development?

- a) It helps in choosing the color scheme of the user interface.
- b) It guides the selection of the most efficient algorithm for key tasks.
- c) It simplifies the algorithm's implementation.
- d) It determines the programming language to be used.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

116

QUIZ

- Q1: What role does algorithm analysis play in the early stages of software development?
- a) It helps in choosing the color scheme of the user interface.
 - b) It guides the selection of the most efficient algorithm for key tasks.
 - c) It simplifies the algorithm's implementation.
 - d) It determines the programming language to be used.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

117

QUIZ

Q2: What does the growth rate of an algorithm represent?

- a) The time required for an algorithm to execute.
- b) The memory required for an algorithm to execute.
- c) The increase in resource usage as input size increases.
- d) The efficiency of an algorithm in terms of accuracy

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

118

QUIZ

Q2: What does the growth rate of an algorithm represent?

- a) The time required for an algorithm to execute.
- b) The memory required for an algorithm to execute.
- c) The increase in resource usage as input size increases.
- d) The efficiency of an algorithm in terms of accuracy

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

119

QUIZ

Q3: Which of the following has the worst time complexity?

- a) $O(\log n)$
- b) $O(n)$
- c) $O(n \log n)$
- d) $O(n^2)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

120

QUIZ

Q3: Which of the following has the worst time complexity?

- a) $O(\log n)$
- b) $O(n)$
- c) $O(n \log n)$
- d) $O(n^2)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

121

QUIZ

Q4: Which of the following represents a constant time complexity?

- a) $O(n)$
- b) $O(n^2)$
- c) $O(1)$
- d) $O(\log n)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

122

QUIZ

Q4: Which of the following represents a constant time complexity?

- a) $O(n)$
- b) $O(n^2)$
- c) $O(1)$
- d) $O(\log n)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

123

QUIZ

Q5: The time complexity of an algorithm that reduces the problem size by half each time is:

- a) $O(n)$
- b) $O(n^2)$
- c) $O(n \log n)$
- d) $O(\log n)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

124

QUIZ

Q5: The time complexity of an algorithm that reduces the problem size by half each time is:

- a) $O(n)$
- b) $O(n^2)$
- c) $O(n \log n)$
- d) $O(\log n)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

125

QUIZ

Q6: Which Big O notation best describes the function

$$f(n) = n^3 + 2n^2 + 5 ?$$

- a) $O(n^2)$
- b) $O(n)$
- c) $O(n^3)$
- d) $O(\log n)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

126

QUIZ

Q6: Which Big O notation best describes the function

$$f(n) = n^3 + 2n^2 + 5 ?$$

- a) $O(n^2)$
- b) $O(n)$
- c) $O(n^3)$
- d) $O(\log n)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

127

QUIZ

Q7: Which of the following best describes Big Omega notation?

- a) It represents the best-case time complexity.
- b) It represents the worst-case time complexity.
- c) It represents the average-case time complexity.
- d) It represents the tight upper bound of time complexity.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

128

QUIZ

Q7: Which of the following best describes Big Omega notation?

- a) It represents the best-case time complexity.
- b) It represents the worst-case time complexity.
- c) It represents the average-case time complexity.
- d) It represents the tight upper bound of time complexity.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

129

QUIZ

Q8: For function $f(n)$, which statement is true regarding the reflexive property?

- a) $f(n)$ is $O(f(n))$
- b) $f(n)$ is $\Theta(f(n))$
- c) $f(n)$ is $\Omega(f(n))$
- d) All of the above.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

130

QUIZ

Q8: For function $f(n)$, which statement is true regarding the reflexive property?

- a) $f(n)$ is $O(f(n))$
- b) $f(n)$ is $\Theta(f(n))$
- c) $f(n)$ is $\Omega(f(n))$
- d) All of the above.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

131

QUIZ

Q9: If $f(n) = 2n \log n$ and $g(n) = n \log n$, what is $f(n)$ in terms of $g(n)$?

- a) $f(n)$ is $O(g(n))$
- b) $f(n)$ is $\theta(g(n))$
- c) $f(n)$ is $\Omega(g(n))$
- d) None of the above

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

132

QUIZ

Q9: If $f(n) = 2n \log n$ and $g(n) = n \log n$, what is $f(n)$ in terms of $g(n)$?

- a) $f(n)$ is $O(g(n))$
- b) $f(n)$ is $\theta(g(n))$**
- c) $f(n)$ is $\Omega(g(n))$
- d) None of the above

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

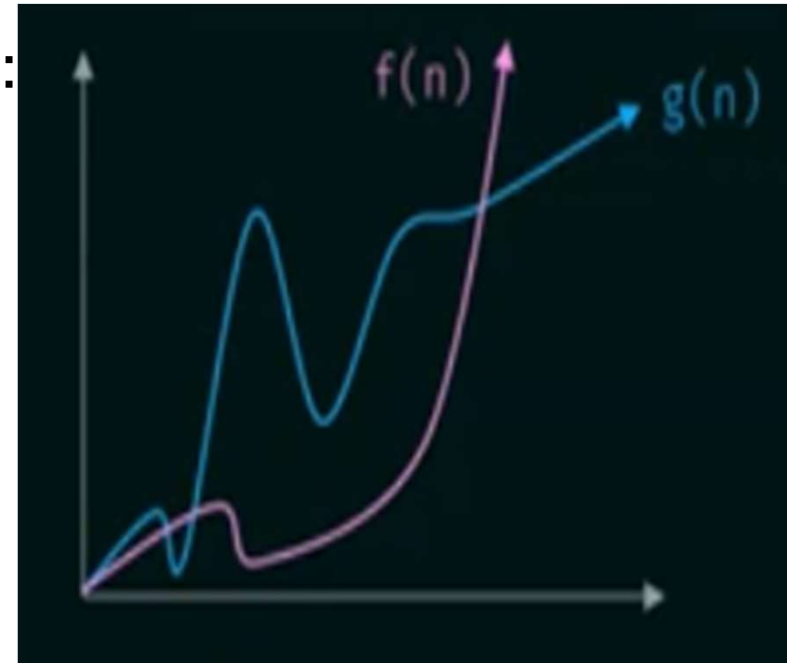
133

QUIZ

Q10: Consider the following graph:

- a) $f(n)$ is $O(g(n))$
- b) $g(n)$ is $O(f(n))$
- c) $f(n)$ is $\theta(g(n))$
- d) None of the above.

Which of the following is true?



Tuesday, August 18, 2026

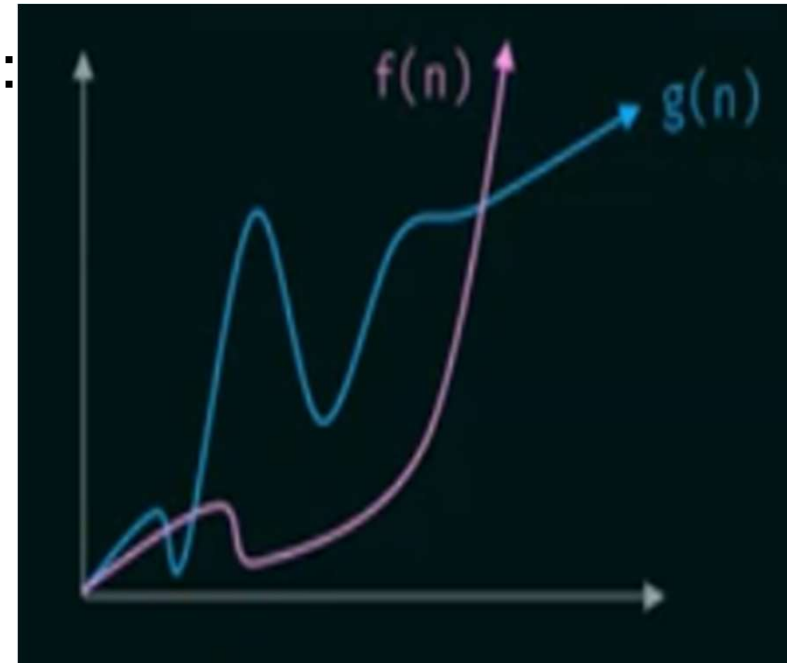
PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

QUIZ

Q10: Consider the following graph:

- a) $f(n)$ is $O(g(n))$
- b) $g(n)$ is $O(f(n))$
- c) $f(n)$ is $\theta(g(n))$
- d) None of the above.

Which of the following is true?



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

135

QUIZ

Q11: Which of the following correctly orders the functions by increasing growth rate?

- a) $O(\log n) < O(n \log n) < O(n^2) < O(2^n)$
- b) $O(n^2) < O(\log n) < O(2^n) < O(n \log n)$
- c) $O(2^n) < O(n^2) < O(n \log n) < O(\log n)$
- d) $O(n \log n) < O(\log n) < O(n^2) < O(2^n)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

136

QUIZ

Q11: Which of the following correctly orders the functions by increasing growth rate?

- a) $O(\log n) < O(n \log n) < O(n^2) < O(2^n)$
- b) $O(n^2) < O(\log n) < O(2^n) < O(n \log n)$
- c) $O(2^n) < O(n^2) < O(n \log n) < O(\log n)$
- d) $O(n \log n) < O(\log n) < O(n^2) < O(2^n)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

137

QUIZ

- Q12: Given the functions $f(n) = O(n^2 \log n)$ and $g(n) = O(n^2)$, which function has slower growth rate?
- a) $f(n)$
 - b) $g(n)$
 - c) They have the same growth rate.
 - d) $g(n)$ grows slowly only for smaller values of n .

QUIZ

Q12: Given the functions $f(n) = O(n^2 \log n)$ and $g(n) = O(n^2)$, which function has slower growth rate?

a) $f(n)$

b) $g(n)$

c) They have the same growth rate.

d) $g(n)$ grows slowly only for smaller values of n .

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

139

QUIZ

Q13: Which of the following relations is correct regarding little o notation?

- a) $n^2 = o(n)$
- b) $\log n = o(n)$
- c) $n^3 = o(n^2)$
- d) $n \log n = o(n)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

140

QUIZ

Q13: Which of the following relations is correct regarding little o notation?

a) $n^2 = o(n)$

b) $\log n = o(n)$

c) $n^3 = o(n^2)$

d) $n \log n = o(n)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

141

QUIZ

Q14: If $f(n) = n^2$ and $g(n) = n$, which of the following is true?

- a) $f(n) = o(g(n))$
- b) $f(n) = O(g(n))$
- c) $f(n) = \omega(g(n))$
- d) $f(n) = \theta(g(n))$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

142

QUIZ

Q14: If $f(n) = n^2$ and $g(n) = n$, which of the following is true?

- a) $f(n) = o(g(n))$
- b) $f(n) = O(g(n))$
- c) $f(n) = \omega(g(n))$
- d) $f(n) = \theta(g(n))$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

143

QUIZ

Q15: Match the following:

1. Linear time
2. Exponential time
3. Logarithmic time
4. Constant time
5. Decrement time

- A. $O(2^n)$
- B. $O(100000000)$
- C. $O(n!)$
- D. $O(n-2)$
- E. $O(\log \log n)$

- a) 1-C, 2-A, 3-E, 4-B, 5-D
- b) 1-A, 2-D, 3-E, 4-B, 5-A
- c) 1-C, 2-D, 3-E, 4-B, 5-A
- d) 1-A, 2-C, 3-E, 4-B, 5-D

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

QUIZ

Q15: Match the following:

1. Linear time
 2. Exponential time
 3. Logarithmic time
 4. Constant time
 5. Decrement time
- a) 1-C, 2-A, 3-E, 4-B, 5-D
 - b) 1-A, 2-D, 3-E, 4-B, 5-A
 - c) 1-C, 2-D, 3-E, 4-B, 5-A
 - d) 1-A, 2-C, 3-E, 4-B, 5-D

- A. $O(2^n)$
- B. $O(100000000)$
- C. $O(n!)$
- D. $O(n-2)$
- E. $O(\log \log n)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

UNDERSTANDING THE TIME COMPLEXITY

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

146

PRIORI VS. POSTERIORI ANALYSIS - RECAP

Priori Analysis	Posteriori Analysis
Estimation of time and memory space required by an algorithm before executing it on the system.	Calculation of time and memory space required by an algorithm after executing it on the system.

Tuesday, August 18, 2026

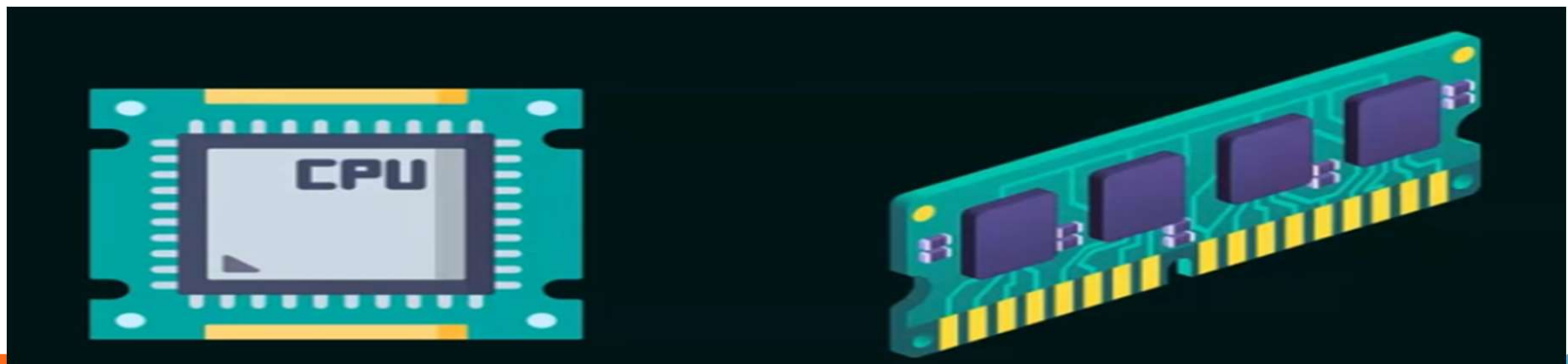
PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

147

CPU COMPUTATIONS AND MAIN MEMORY SPACE

CPU Computations: refers to a **task performed** by the CPU or **instruction executed** by the CPU.

Main Memory Space: is used to **temporarily store data** and **instructions** that CPU needs for **quick access** during program execution



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

UNDERSTANDING THE TIME COMPLEXITY

Estimation of total CPU computations required to execute an algorithm.

Time Complexity of an Algorithm = Total No. of CPU Computations = Sum of frequency count of each instruction of an algorithm.

Number of times an instruction is executed.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

EXAMPLE

```
Algo sum(a, n)
{
    sum = 0
    for (i = 1; i <= n; i++)
        sum = sum + a[i]
    return sum
}
```

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

150

EXAMPLE

Algo sum(a, n)

{

sum = 0

for (i = 1; i <= n; i++)

sum = sum + a[i]

return sum

}

Number of elements

List of n elements

Tuesday, August 18, 2026

PREPARED BY:

K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

151

EXAMPLE

Algo sum(a, n)

{

sum = 0

for (i = 1; i <= n; i++)

sum = sum + a[i]

return sum

}

1 unit

1 +

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

152

EXAMPLE

Algo sum(a, n)

{

sum = 0

for (i = 1; i <= n; i++)

sum = sum + a[i]

return sum

}

1 unit

1 + 1 +

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

153

EXAMPLE

Algo sum(a, n)

{

sum = 0

for (i = 1; i <= n; i++)

sum = sum + a[i]

return sum

}

n+ 1 units

1 <= n

2 <= n

3 <= n

.

.

.

n <= n

n + 1 <= n False

1 + 1 +

Tuesday, August 18, 2026

PREPARED BY:

K S RANJITH, M.E (Ph.D),

ASSOCIATE PROFESSOR,

DEPT. OF AI&DS,

MTIET

154

EXAMPLE

Algo sum(a, n)

```
{  
    sum = 0  
    for (i = 1; i <= n; i++)  
        sum = sum + a[i]  
    return sum  
}
```

n+1 units

$$\left\{ \begin{array}{l} 1 \leq n \\ 2 \leq n \\ 3 \leq n \\ \vdots \\ \cdot \\ n \leq n \\ n+1 \leq n \end{array} \right\}$$

n+1
Comparisons

1 + 1 + n+1 +

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

155

EXAMPLE

Algo sum(a, n)

{

sum = 0

for (i = 1; i <= n; i++)

sum = sum + a[i]

return sum

}

n units

1 + 1 + n + 1 + n

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

156

EXAMPLE

Algo sum(a, n)

{

sum = 0

for (i = 1; i <= n; i++)

sum = sum + a[i]

return sum

}

2n units

$1 + 1 + n + 1 + n + 2n$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

157

EXAMPLE

Algo sum(a, n)

{

sum = 0

for (i = 1; i <= n; i++)

sum = sum + a[i]

return sum



1 units

}

$$1 + 1 + n + 1 + n + 2n + 1$$

Tuesday, August 18, 2026

PREPARED BY:

K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

158

EXAMPLE

```
Algo sum(a, n)
{
    sum = 0
    for (i = 1; i <= n; i++)
        sum = sum + a[i]
    return sum
}
```

$$\begin{aligned} &1 + 1 + n + 1 + n + 2n + 1 \\ &= 4n + 4 \\ &= O(n) \end{aligned}$$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

UNDERSTANDING THE SPACE COMPLEXITY

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

160

Estimation of main memory space required to execute an algorithm.

$$\begin{aligned} \text{Space Complexity of an Algorithm} = & \left\{ \begin{aligned} & \text{Space required to store the source code} \\ & + \\ & \text{Space required for simple variables} \\ & + \\ & \text{Space for the data structures used} \\ & + \\ & \text{Space required for stack for recursive algorithms} \end{aligned} \right. \end{aligned}$$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

161

EXAMPLE

Algo sum(a, n)

{

sum = 0

for (i = 1; i <= n; i++)

sum = sum + a[i]

return sum

}

List of n elements

Space Complexity: $O(n)$

Space required to store the source code
+
Space required for simple variables
+
Space for the data structures used
+
Space required for stack for recursive algorithms

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

162

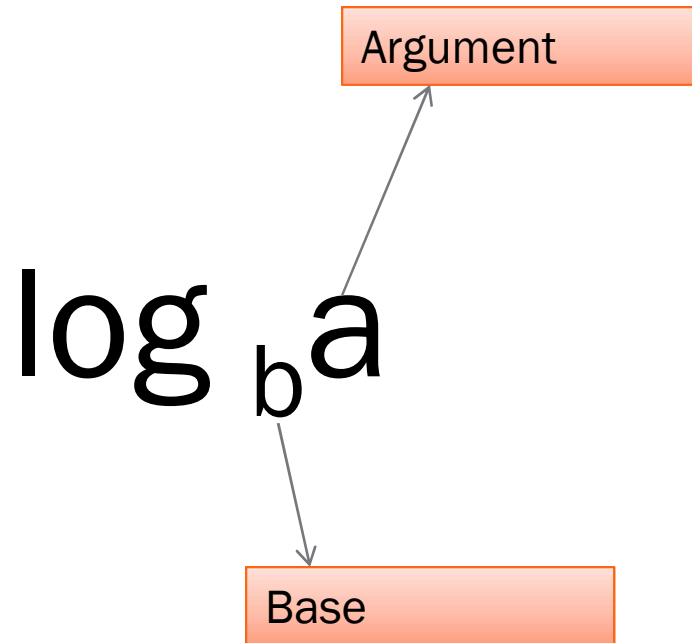
ANALYSIS OF ALGORITHMS

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

163

LOGARITHMS AND SUMMATIONS



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

LOGARITHMS AND SUMMATIONS

$$\log_2 128 = 7$$

Argument

Base

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

165

COMMONLY USED LOGARITHMS

1. $\log_a a = 1$

Example: $\log_{10} 10 = 1$

2. $\log_a 1 = 0$

Example: $\log_{10} 1 = 0$

3. $\log_c a^b = b * \log_c a$

Example: $\log_{10} 10^2 = 2\log_{10} 10 = 2 \times 1 = 2$

4. $\log_c ab = \log_c a + \log_c b$

Example: $\log_{10} 1000 = \log_{10} 100 + \log_{10} 10 = 2 + 1 = 3$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

COMMONLY USED LOGARITHMS

5. $\log_c a/b = \log_c a - \log_c b$

Example: $\log_{10} 100/10 = \log_{10} 100 - \log_{10} 10 = 2 - 1 = 1$

6. $\log_b a = 1 / \log_a b$

Example: $\log_{32} 2 = 1 / \log_2 32 = 1 / 5 = 0.2$

7. $\log_b a = \log_c a / \log_c b$

Example: $\log_{16} 256 = \log_2 256 / \log_2 16 = 8 / 4 = 2$

8. $a^{\log_c b} = b^{\log_c a}$

Example: $100^{\log_{10} 16} = 16^{\log_{10} 100} = 16^2 = 256$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

COMMONLY USED SUMMATIONS

1. $1 + 2 + 3 + 4 + \dots + n = \frac{n(n+1)}{2} = \theta(n^2)$
2. $1^2 + 2^2 + 3^2 + 4^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6} = \theta(n^3)$
3. $1^3 + 2^3 + 3^3 + 4^3 + \dots + n^3 = \frac{n^2(n+1)^2}{4} = \theta(n^4)$
4. $1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \dots + \frac{1}{n} = \log_e n = \theta(\log n)$
5. ${}^nC_0 + {}^nC_1 + {}^nC_2 + {}^nC_3 + \dots + {}^nC_n = 2^n = \theta(2^n)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

TIME COMPLEXITY OF SINGLE LOOPS

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

169

TIME COMPLEXITY - RECAP

Understanding the Time Complexity

- Estimation of total CPU computations required to execute an algorithm.

$$\text{Time Complexity of an Algorithm} = \text{Total No. of CPU Computations} = \text{Sum of frequency count of each instruction of an algorithm.}$$

Number of times an instruction is executed

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

TIME COMPLEXITY OF LOOPS

Estimation of total CPU computations required to execute a loop.

Time Complexity
of a Loop = Sum of Frequency count
of each instruction of the
loop.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

171

TIME COMPLEXITY OF LOOPS

Estimation of total CPU computations required to execute a loop.

Time Complexity
of a Loop = Frequency count of the
innermost instruction of
the loop.

Example:

```
for (i = 1; i <= n; i++)  
    printf("Algorithm Analysis Class");
```

n times = $O(n)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

TIME COMPLEXITY OF SINGLE LOOPS

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

173

INCREMENT BY A CONSTANT

1. for (i = 1; i <= n; i++)

printf("Algorithm Analysis Class"); ←

n times = $O(n)$

2. for (i = 1; i <= n; i+2)

printf("Algorithm Analysis Class"); ← ??

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

174

INCREMENT BY A CONSTANT

2. for (i = 1; i <= n; i+2)
 printf("Algorithm Analysis Class"); ← $\frac{n-1}{2} + 1$ times

i = 1, 1 + 2, 1 + 2 x 2, 1 + 3 x 2, ..., 1 + k x 2

$$1 + k \times 2 = n$$

$$k \times 2 = n - 1$$

$$k = \frac{n-1}{2}$$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

175

INCREMENT BY A CONSTANT

1. for (i = 1; i <= n; i++)

printf("Algorithm Analysis Class"); ←

n times = $O(n)$

2. for (i = 1; i <= n; i+2)

printf("Algorithm Analysis Class"); ←

n times = $O(n)$

3. for (i = 1; i <= n; i+2)

printf("Algorithm Analysis Class"); ←

$\frac{n-1}{10} + 1$ times = $O(n)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

176

DECREMENT BY A CONSTANT

1. for (i = n; i <= n; i - -)

printf("Algorithm Analysis Class"); ←

n times = $O(n)$

2. for (i = n; i >= 1; i - 2)

printf("Algorithm Analysis Class"); ← ??

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

177

DECREMENT BY A CONSTANT

2. for (i = n; i >= 1; i-2)
 printf("Algorithm Analysis Class"); ← $\frac{n-1}{2} + 1$ times

i = n, n - 2, n - 2 x 2, n - 3 x 2, ..., n - k x 2

$$n - k \times 2 = 1$$

$$k \times 2 = n - 1$$

$$k = \frac{n-1}{2}$$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

DECREMENT BY A CONSTANT

1. for (i = n; i >= 1; i - -)

printf("Algorithm Analysis Class"); ←

n times = $O(n)$

2. for (i = n; i >= 1; i=i-2)

printf("Algorithm Analysis Class"); ←

n times = $O(n)$

3. for (i = n; i >= 1; i=i-10)

printf("Algorithm Analysis Class"); ←

$\frac{n-1}{10} + 1$ times = $O(n)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

179

MULTIPLYING THE UPDATE EXPRESSION

Single Loop - Multiplying the Update Expression

```
for (i = 1; i <= n; i = i * 5)  
    printf("Welcome to ADSA Class");
```

Time complexity = $\theta(\log n)$

$i = 1, 5, 5^2, 5^3, \dots, 5^k$

$$5^k = n$$

Taking log on both sides

$$\log 5^k = \log_5 n$$

➔ $k = \log_5 n$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

MULTIPLYING THE UPDATE EXPRESSION

Determine the time complexity of the following loops:

1. `for (i = 1; i <= n; i = i * 10)`
 `printf("Welcome to ADSA Class");`
2. `for (i = 1; i <= n; i = i * 3)`
 `printf("Welcome to ADSA Class");`

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

181

DIVIDING THE UPDATE EXPRESSION

Single Loop - Dividing the Update Expression

```
for (i = n; i >= 1; i = i / 5)  
    printf("Welcome to ADSA Class");
```

$$i = n, \frac{n}{5}, \frac{n}{5^2}, \frac{n}{5^3}, \dots, \frac{n}{5^k}$$
$$\frac{n}{5^k} = 1 \Rightarrow 5^k = n$$

Taking log on both sides

$$\log_5 5^k = \log_5 n$$
$$\Rightarrow k = \log_5 n$$

DIVIDING THE UPDATE EXPRESSION

Determine the time complexity of the following loops:

1. `for (i = n; i >= 1; i = i / 10)`
 `printf("Welcome to ADSA Class");`
2. `for (i = n; i >= 1; i = i / 3)`
 `printf("Welcome to ADSA Class");`

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

183

UPDATE EXPRESSION WITH POWER

Single Loop - Update Expression with Power

```
for (i = 3; i <= n; i = i2 )  
    printf("Welcome to ADSA Class");
```

$i = 3, 3^2, 3^{2^2}, 3^{2^3}, \dots, 3^{2^k}$
 $3^{2^k} = n$

time complexity = $\theta(\log \log n)$

Taking log on both the sides

$$\log 3^{2^k} = \log_3 n$$

$$2^k = \log_3 n$$

Taking log on both side

$$\log 2^k = \log_2 \log_3 n$$

$$k = \log_2 \log_3 n$$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

UPDATE EXPRESSION WITH POWER

```
for (i = n; i >= 5; i =  $\sqrt{i}$ )  
    printf("Welcome to ADSA Class");  
i = n,  $n^{1/2}$ ,  $n^{1/2^2}$ ,  $n^{1/2^3}$ , ...,  $n^{1/2^k}$   
 $n^{1/2^k} = 5$ 
```

taking log on both sides

$$\log n^{1/2^k} = \log_5 5$$

$$(1/2^k) \log_5 n = 1$$

$$2^k = \log_5 n$$

taking log on both the sides

$$\log 2^k = \log \log_5 n$$

$$k = \log_2 \log_5 n$$

time complexity = $\theta(\log \log n)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

185

UPDATE EXPRESSION WITH POWER

Determine the time complexity of the following loops:

1. for ($i = 1; i \leq n; i = i^{2.5}$)
 printf("Welcome $\sqrt[3]{i}$ DSA Class");
2. for ($i = n; i \geq 1; i = \quad$)
 printf("Welcome to ADSA Class");

CONDITIONAL STATEMENT WITH A FUNCTION

Conditional Statement with a Function

```
for (i = 1; i <= 2n ; i++)  
    printf("Good Morning");  
    i = 1, 2, 3, 4, ..., k  
    k = 2n
```

Time complexity = $\theta(2^n)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

187

CONDITIONAL STATEMENT WITH A FUNCTION

```
for (i = 2; i <= 2n ; i = i2 )  
    printf("Good Morning");  
i = 2, 22 , 22^2 , 22^3 , ..., 22^k  
22^k = 2n
```

taking log on both sides

$$\log_2 2^{2^k} = \log_2 2^n$$

$$2^k = n$$

taking log on both sides

$$\log 2^k = \log n$$

$$k = \log_2 n$$

Time complexity = $\theta(\log n)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

CONDITIONAL STATEMENT WITH A FUNCTION

```
for (i = 1; i <=  $\sqrt{n}$ ; i++)  
    printf("Good Morning");  
i = 1, 2, 3, 4, ..., k  
k =  $\sqrt{n}$ 
```

Time complexity = $\theta(\sqrt{n})$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

189

CONDITIONAL STATEMENT WITH A FUNCTION

Determine the time complexity of the following loops:

1. for ($i = 1; i \leq \sqrt{n}; i++$)
 printf("Good Morning");
2. for ($i = 1; i^2 \leq n; i = i * 2$)
 printf("Good Morning");

INDEPENDENT LOOPS

Nested Loops – Introduction

Types of Nested Loops:

1. Independent Nested Loop
2. Dependent Nested Loop

Independent Nested Loop: Inner loop variable is independent of the outer loop variable.

Time Complexity of Independent Nested loop = Multiplication of frequency count of each loop

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

191

INDEPENDENT LOOPS

Example

```
for (i = 1; i <= n; i++)  
    for (j = 1; j <= n; j++)  
        x = x + 1
```



n times



$n \times n$ times

i=1
j=1 to n

i=2
j=1 to n

i=3
j=1 to n

.....

i=n
j=1 to n

Time Complexity = $\theta(n^2)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

192

TIME COMPLEXITY OF INDEPENDENT NESTED LOOPS

```
for (i = 1; i <= n; i++)  
    for (j = 1; j <= n; j++)  
        for (k = 1; k <= n; k++)  
            x = x + 1
```

n times

n times

n times

$n \times n \times n$
 $= n^3$ times

Time Complexity = $\theta(n^3)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

193

TIME COMPLEXITY OF INDEPENDENT NESTED LOOPS

```
for (i = 3; i <= n; i = i3)  
    for (j = n; j >= 1; j = j / 2)  
        for (k = 1; k <= n; k = k * 4)  
            x = x + 1
```

$\log_3 \log_3 n$ times

$\log_2 n$ times

$\log_4 n$ times

Time Complexity = $\theta(\log^2 n \cdot \log \log n)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

194

DEPENDENT LOOPS

Dependent Loops – Introduction

Types of Nested Loops:

1. Independent Nested Loop
2. Dependent Nested Loop

Dependent Nested Loop: Inner loop variable depends on the outer loop variable

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

195

DEPENDENT LOOPS

Example

```
for (i = 1; i <= n; i++)  
    for (j = 1; j <= n; j = j + i)  
        x = x + 1
```

i	j values	Iterations
1	1,2,3,...,n	n
2	1,3,5,...	n/2
3	1,4,7,...	n/3
4	1,5,9,...	n/4
...	...	...
n	1	1

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

DEPENDENT LOOPS

Total Number of Executions

Step	Calculation
Given Code	<pre>for(i = 1; i <= n; i++) for(j = 1; j <= n; j = j + i) x = x + 1;</pre>
Outer Loop	Executes n times.
Inner Loop	For each value of i , j increases by i , so the number of iterations is n/i .
Total Executions	$T(n) = n + n/2 + n/3 + n/4 + \dots + n/n$
Take n Common	$T(n) = n(1 + 1/2 + 1/3 + 1/4 + \dots + 1/n)$
Harmonic Series	$(1 + 1/2 + 1/3 + \dots + 1/n) = \Theta(\log n)$
Substitute	$T(n) = n \times \Theta(\log n)$
Final Result	$T(n) = \Theta(n \log n)$

Time Complexity = $\Theta(n \log n)$

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

DEPENDENT LOOPS

```
for (i = 1; i <= n; i++)  
    for (j = 1; j <= i; j++)  
        for (k = 1; k <= j; k++)  
            x = x + 1
```

i	j Values	k Iterations	Total for i
1	1	1	1
2	1, 2	1 + 2	3
3	1, 2, 3	1 + 2 + 3	6
4	1, 2, 3, 4	1 + 2 + 3 + 4	10
...	...	...	...
n	1 to n	1 + 2 + ... + n	$n(n+1)/2$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

DEPENDENT LOOPS

$T(n)$

$= 1$

$+ (1+2)$

$+ (1+2+3)$

$+ (1+2+3+4)$

$+ \dots$

$+ (1+2+3+\dots+n)$

Now it becomes

$T(n)$

$= 1$

$+ 3$

$+ 6$

$+ 10$

$+ \dots$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

199

DEPENDENT LOOPS

Every bracket is

$$1+2+\dots+i$$

$$= i(i+1)/2$$

Therefore

$$T(n)$$

$$= 1(2)/2$$

$$+ 2(3)/2$$

$$+ 3(4)/2$$

$$+ 4(5)/2$$

$$+ \dots$$

$$+ n(n+1)/2$$

$$1(2)/2 + 2(3)/2 + 3(4)/2 + \dots + n(n+1)/2$$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

200

DEPENDENT LOOPS

Expand

Take 1/2 common.

$$= (1^2+1)/2$$

$$= 1/2[(1^2+2^2+3^2+\dots+n^2) + (1+2+3+\dots+n)]$$

$$+ (2^2+2)/2$$

$$+ (3^2+3)/2$$

+ ...

$$+ (n^2+n)/2$$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

201

DEPENDENT LOOPS

We know

$$1+2+\dots+n = O(n^2)$$

and

$$1^2+2^2+3^2+\dots+n^2 = O(n^3)$$

Since

$$O(n^3) > O(n^2)$$

Therefore

$$\text{Time Complexity} = \theta(n^3)$$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

202

DEPENDENT LOOPS

H.W Problem

```
for (i = 1; i <= n; i++)  
    for (j = i; j <= n; j++)  
        for (k = j; k <= n; k++)  
            x = x + 1
```

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

203

EXAMPLE PROBLEMS

Problem: Consider the following C program fragment in which i , j , and n are integer variables.

```
for (i = n, j = 0; i > 0; i /= 2; j += i);
```

Let $\text{val}(j)$ denotes the value stored in the variable j after termination of the for loop. Which one of the following is true?

- | | |
|--------------------------------------|--|
| (A) $\text{val}(j) = \theta(\log n)$ | (B) $\text{val}(j) = \theta(\sqrt{n})$ |
| (C) $\text{val}(j) = \theta(n)$ | (D) $\text{val}(j) = \theta(n \log n)$ |

[GATE 2006]

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

204

EXAMPLE PROBLEMS

Problem: Consider the following C code segment:

```
int isPrime(n)
{
    int i, n;
    for (i = 2; i <= sqrt(n); i++) {
        if (n % i == 0) {
            printf("Not prime\n");
            return 0;
        }
    }
    return 1;
}
```

Let $T(n)$ denote the number of times the for loop is executed by the program on input n . Which of the following is true?

[GATE 2007]

- (A) $T(n) = O(\sqrt{n})$ and $T(n) = \Omega(\sqrt{n})$ (B) $T(n) = O(\sqrt{n})$ and $T(n) = \Omega(1)$
(C) $T(n) = O(n)$ and $T(n) = \Omega(\sqrt{n})$ (D) None of the above

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

205

EXAMPLE PROBLEMS

Problem: Consider the following C code segment:

```
int fun1(int n)
{
    int i, j, k, p, q = 0;
    for (i = 1; i < n; ++i) {
        p = 0;
        for (j = n; j > 1; j = j / 2)
            ++p;
        for (k = 1; k < p; k = k * 2)
            ++q;
    }
    return q;
}
```

Which of the following most closely approximates the return value of the function fun1?

[GATE 2015]

- (A) n^3 (B) $n(\log n)^2$ (C) $n \log n$ (D) $n \log(\log n)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

206

EXAMPLE PROBLEMS

Problem: Consider the following C function:

```
int fun(int n)
{
    int i, j;
    for (i = 1; i <= n; i++) {
        for (j = 1; j < n; j += i) {
            printf("%d %d", i, j);
        }
    }
}
```

Asymptotic notation of fun
in terms of θ notation is

[GATE 2017]

(A) $\theta(n\sqrt{n})$ (B) $\theta(n^2)$ (C) $\theta(n\log n)$ (D) $\theta(n^2\log n)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

207

EXAMPLE PROBLEMS

Problem: Consider the following function:

```
int unknown(int n)
{
    int i, j, k=0;
    for (i = n/2; i <= n; i++)
        for (j = 2; j <= n; j = j*2)
            k = k + n/2;
    return (k);
}
```

The return value of the function is

- (A) $\theta(n^2)$ (B) $\theta(n^2 \log n)$
(C) $\theta(n^3)$ (D) $\theta(n^3 \log n)$

[GATE 2013]

QUIZ

Q1: What is the time complexity of the following loop?

```
for (i = 1; i <= n; i++) {  
    printf("Hi!");  
}
```

- a) $O(n)$
- b) $O(1)$
- c) $O(n^2)$
- d) $O(\log n)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

209

QUIZ

Q2: How to find the time complexity of the independent nested loop?

- a) By adding the frequency count of each loop.
- b) By finding the maximum frequency count.
- c) By multiplying the frequency count of each loop.
- d) By dividing the frequency count of each loop by 2.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

210

QUIZ

Q3: What is the time complexity of the nested loop?

```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= n; j++) {  
        printf("Hi!");  
    }  
}
```

- a) $O(n)$
- b) $O(1)$
- c) $O(n^2)$
- d) $O(\log n)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

211

QUIZ

Q4: What is the time complexity of the following loop?

```
for (int i = 1; i <= n; i *= 2) {  
    printf("Hi!");  
}
```

- a) $O(n)$
- b) $O(\log n)$
- c) $O(n \log n)$
- d) $O(n^2)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

212

QUIZ

Q5: Determine the time complexity of the following loop:

```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= i; j++) {  
        printf("Hi!");  
    }  
}
```

- a) $O(n)$
- b) $O(n \log n)$
- c) $O(n^2)$
- d) $O(n^3)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

QUIZ

Q6: What is the space complexity of the following function?

```
void fun(int n) {  
    int arr[n];  
    for (int i = 0; i < n; i++) {  
        arr[i] = i;  
    }  
}
```

- a) $O(1)$
- b) $O(\log \log n)$
- c) $O(\log n)$
- d) $O(n)$

QUIZ

Q7: Which of the following loops has the time complexity of $O(\log n)$?

- a) `for(int i = 1; i <= n; i++) {}`
- b) `for(int i = 1; i <= n; i *= 2) {}`
- c) `for(int i = 1; i <= n; i += 2) {}`
- d) `for(int i = 1; i <= n; i /= 2) {}`

QUIZ

Q8: Which of the following is true about the time complexity of the single loop?

- a) Time complexity of a single loop is same as the frequency count of the first instruction of the program.
- b) Time complexity of a single loop is same as the frequency count of the innermost instruction of the loop.
- c) Time complexity of a single loop is obtained after multiplying the frequency count of each instruction.
- d) None of the above.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

216

QUIZ

Q9: Which of the following has the smallest time complexity?

- a) $O(n^2)$
- b) $O(\log n)$
- c) $O(\log \log n)$
- d) $O(n)$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

217

QUIZ

Q10: Determine the time complexity of the following loop:

```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= i; j *= 2) {  
        printf("Hi!");  
    }  
}
```

- a) $O(n \log n)$
- b) $O(n^2)$
- c) $O(n)$
- d) $O(n^3)$

QUIZ

Q11: What is the value of $(\log_{10} 100)^{\log_2 128}$

- a) 7
- b) 14
- c) 70
- d) 128

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

219

QUIZ

Q12: Which of the following is the valid reason for preferring an algorithm with a higher space complexity but a lower time complexity?

- a) The algorithm will run on a machine with limited memory.
- b) Space complexity and time complexity are independent to each other.
- c) Space complexity is never a metric for an efficient algorithm.
- d) Memory is cheap, and time is a critical factor for performance.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

220

SPACE COMPLEXITY OF RECURSIVE ALGORITHMS

- A specific memory space used to manage function calls.
- When a function is called, the activation record goes inside the stack.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

221

SPACE COMPLEXITY OF RECURSIVE ALGORITHMS

$$\begin{aligned} \text{Space Complexity of an Algorithm} = & \left\{ \begin{aligned} & \text{Space required to store the source code} \\ & + \\ & \text{Space required for simple variables} \\ & + \\ & \text{Space for the data structures used} \\ & + \\ & \text{Space required for stack for recursive algorithms} \end{aligned} \right. \end{aligned}$$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

222

SPACE COMPLEXITY OF RECURSIVE ALGORITHMS

$$\text{Space Complexity of an Algorithm} = \left\{ \begin{array}{l} \text{Space for the data structures used} \\ + \\ \text{Space required for stack for recursive algorithms} \end{array} \right.$$

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

223

QUESTIONS ON ALGORITHM ANALYSIS

1. Define asymptotic notations. Explain different types of notations with example.
2. Discuss various types of asymptotic notations used for best, average and worst case analysis of algorithm.
3. Give the time complexity of the following code snippet

```
intfunc(int n)
{
    for (int i=1;i<=n; i++)
    {
        for(int j=1; j<=n; j+=i)
        {
            sum=sum +i*j;
        }
    }
    return (sum);
}
```

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

QUESTIONS ON ALGORITHM ANALYSIS

4. Determine the time complexity of the following function:

```
intfunc(int n) {  
    for (int i = 1; i <= n; i++) {  
        for (int j = 1; j <= n; j *= 2) {  
            sum = sum + i + j;  
        }  
    }  
    return sum;  
}
```

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

225

QUESTIONS ON ALGORITHM ANALYSIS

2 Marks Questions

1. Distinguish between Algorithm and Pseudo code
2. Define i) Time Complexity ii) Space Complexity.
3. Describe & define any three Asymptotic Notations
4. Describe Different characteristics of an algorithm

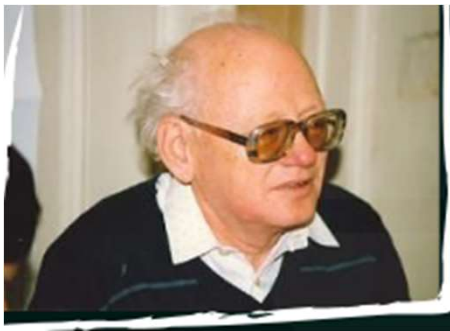
Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

226

INTRODUCTION TO AVL TREE

- The height balanced binary search tree is called the AVL Tree or Adelson-Velsky and Landis Tree.



G. M. Adelson-Velsky



E. M. Landis

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

227

RECALL

A height balanced binary tree or height balanced binary search tree has the following properties:

- ★ Difference between the left height and right height of a node is not more than one.
- ★ Left subtree of a node is itself height balanced.
- ★ Right subtree of a node is itself height balanced.

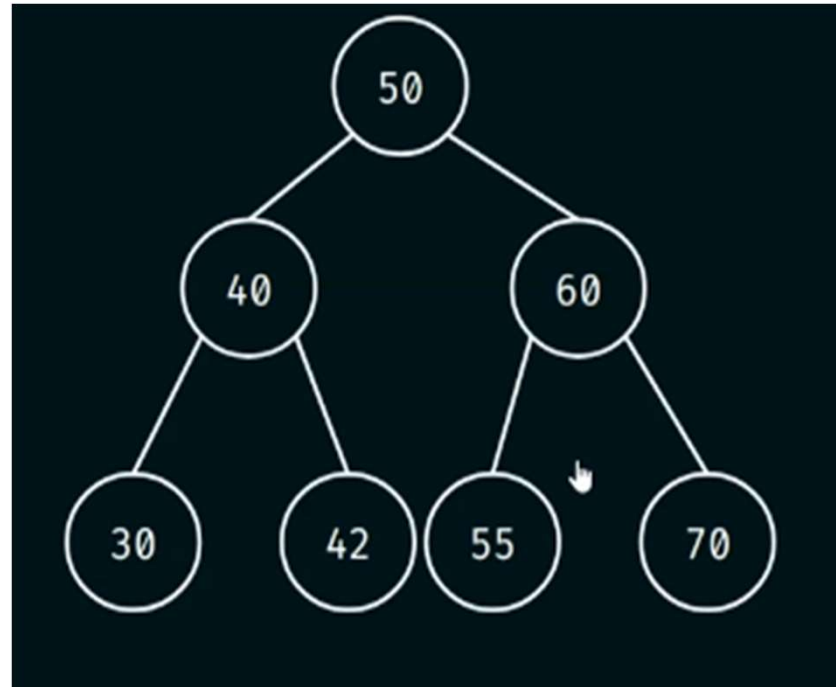
Therefore, an **AVL Tree or a height balanced binary search tree** is a tree where the difference between the left height and right height of any node can be at most 1.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

228

EXAMPLE

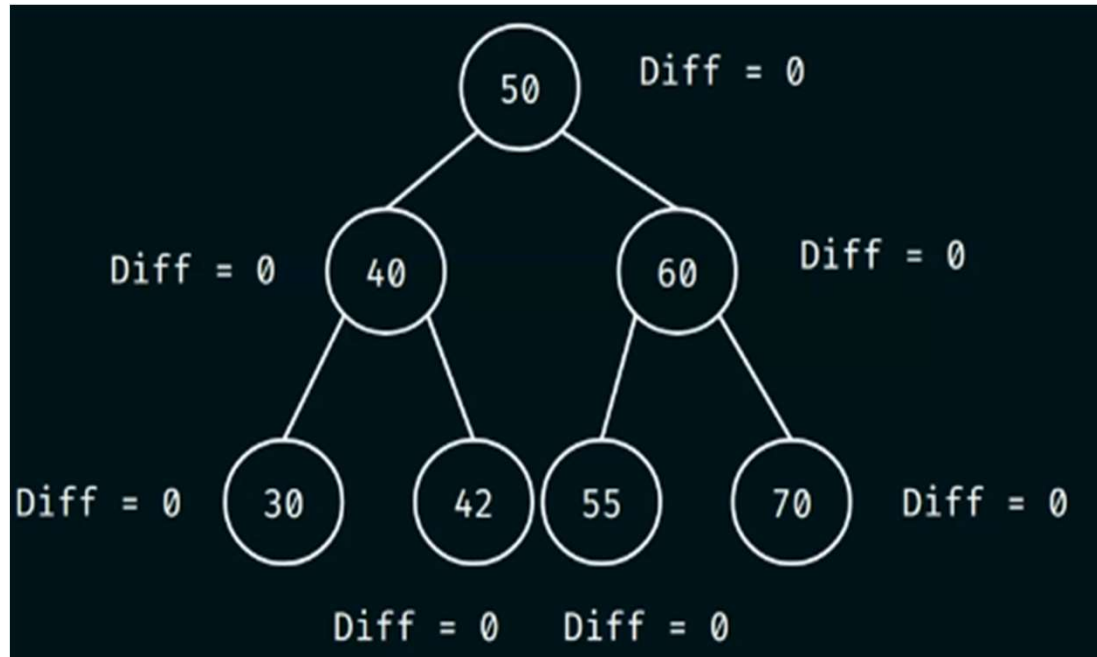


Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

229

EXAMPLE



Height Balanced BST or AVL Tree

$$\text{abs}(\text{leftHeight} - \text{rightHeight}) \leq 1$$

Tuesday, August 18, 2026

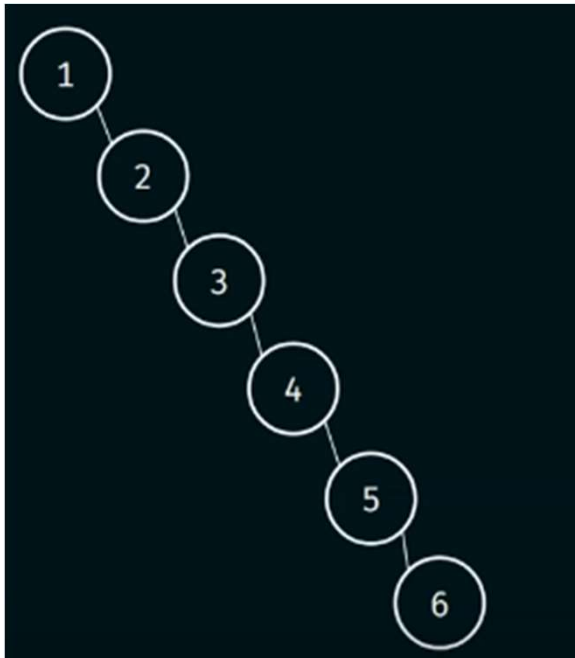
PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

230

WHY DO WE EVEN NEED AVL TREES?

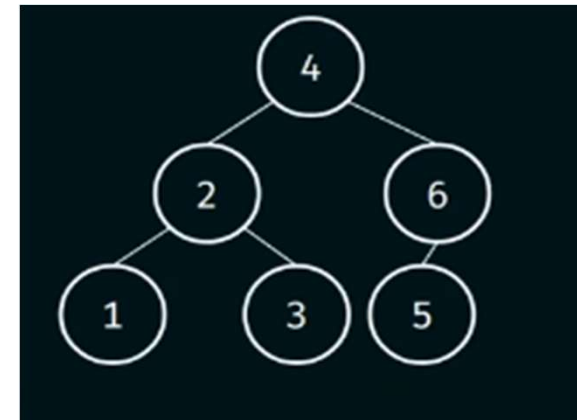
The sequence of the data does matter in the creation of binary search trees.

Example: 1, 2, 3, 4, 5, 6



Average number of comparisons
 $= (1 + 2 + 3 + 4 + 5 + 6)/6 = 3.5$

4, 2, 1, 3, 6, 5



Average number of comparisons
 $= (1 + 2 + 2 + 3 + 3 + 3)/6 = 2.3$

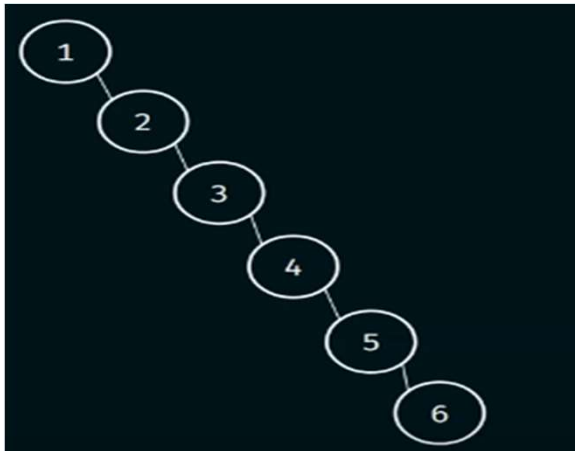
Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

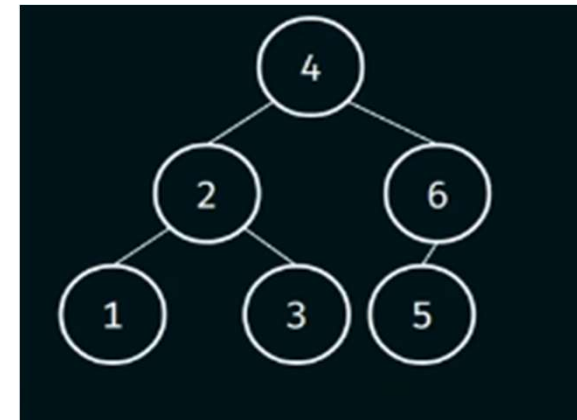
WHY DO WE EVEN NEED AVL TREES?

The sequence of the data does matter in the creation of binary search trees.

Example: 1, 2, 3, 4, 5, 6



4, 2, 1, 3, 6, 5



Height Balanced

The main advantage of AVL tree or height balanced binary search tree is to perform efficient search, insertion and deletion operations.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

232

LEFT HEAVY NODE VS RIGHT HEAVY NODE

Formula for calculating the difference = $\text{abs}(\text{leftheight} - \text{rightheight})$

Two possible values: 0 or 1.

Balance Factor = $\text{leftHeight} - \text{rightHeight}$

Three possible values: -1, 0 or 1.

A node is called **right heavy or right high** if the right height of the node is one more than the left height of the node.

A node is called left heavy or left high if the left height of the node is one more than the right height of the node.

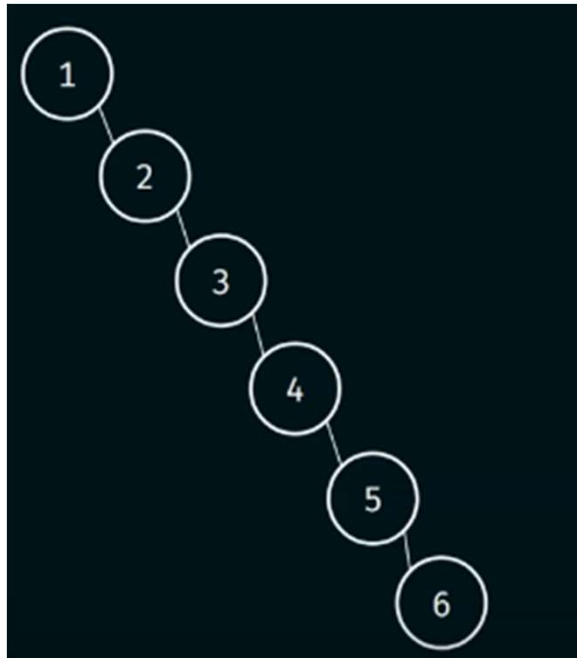
The balance factor is 1 for left high, -1 for right high and 0 for the balanced node.

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

WHY DO WE EVEN NEED AVL TREES?

The sequence of the data does matter in the creation of binary search trees.

Example: 1, 2, 3, 4, 5, 6

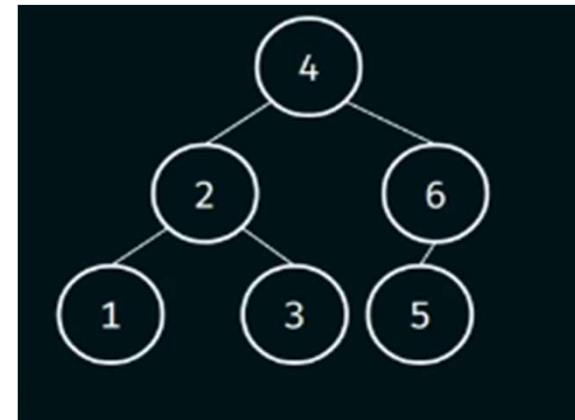


Height = n

Maximum height

Average number of comparisons
 $= (1 + 2 + 3 + 4 + 5 + 6)/6 = 3.5$

4, 2, 1, 3, 6, 5



Height = $\log n$

Minimum height

Average number of comparisons
 $= (1 + 2 + 2 + 3 + 3 + 3)/6 = 2.3$

We must always target for the least height possible

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

ROTATIONS IN AVL TREES

The structure of binary search tree depends on the sequence of data being inserted and this is the major disadvantage. Hence, AVL trees are needed.

AVL Trees are self-balancing BSTs

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

235

WHY AVL TREES ARE SELF-BALANCING BSTS?

Consider the following keys to be inserted in the BST:

Keys: 1, 2, 3

How many different arrangements of the above keys are possible?

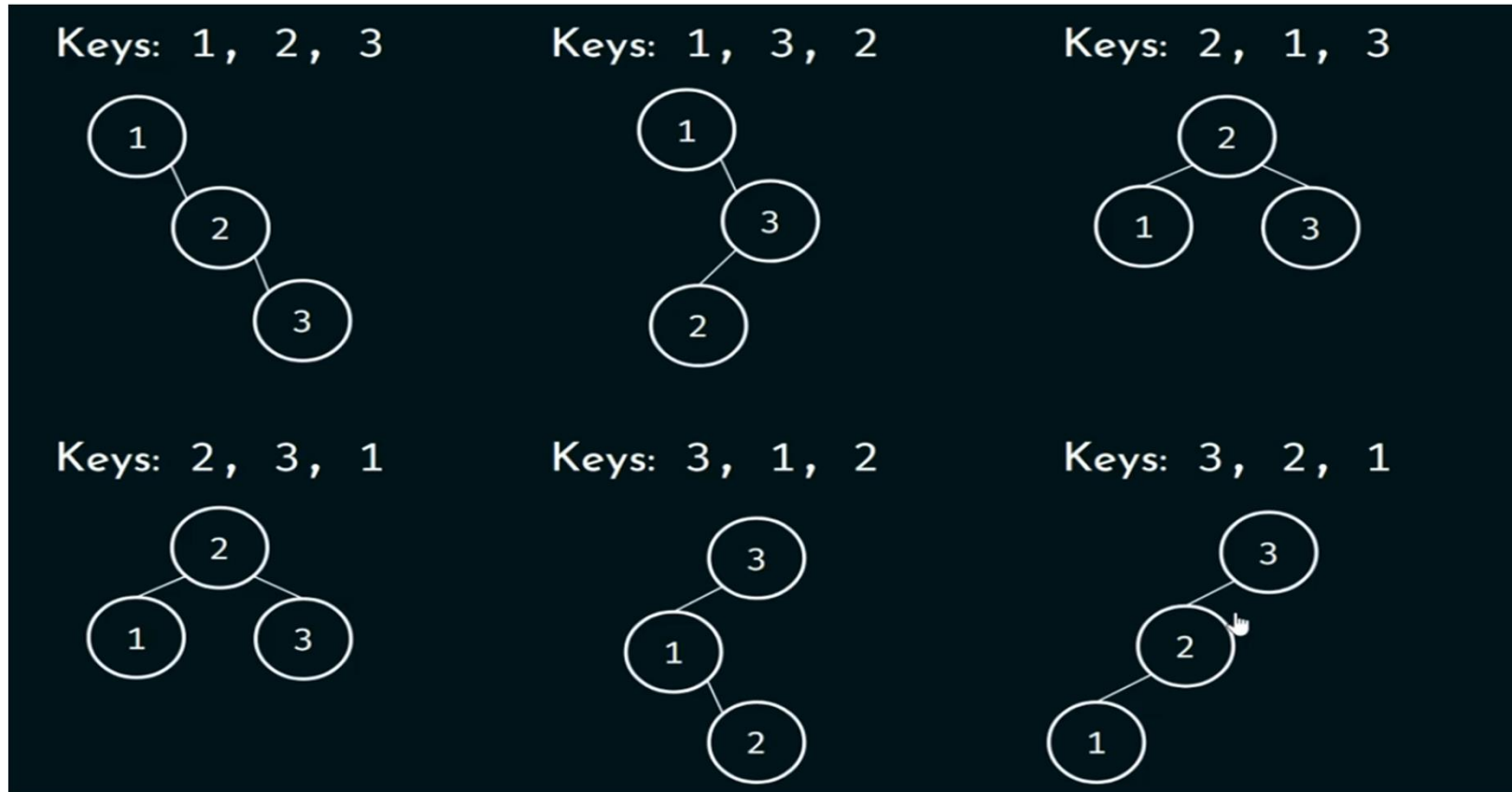
$$\begin{array}{ccc} 3 & 2 & 1 \\ \hline 3 \times 2 \times 1 & = 6 & = 3! \end{array}$$

Tuesday, August 18, 2026

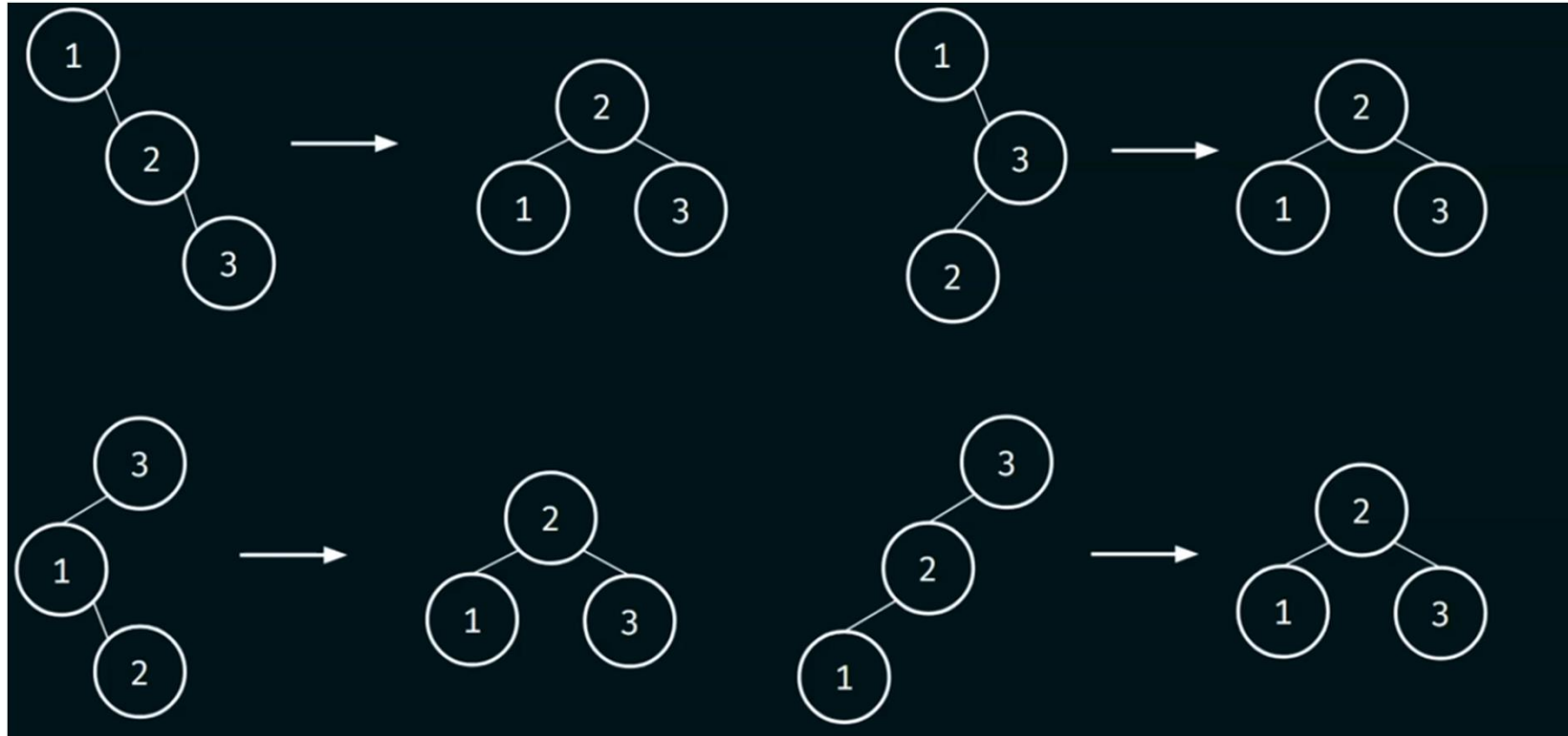
PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

236

ROTATIONS IN AVL TREES



ROTATIONS IN AVL TREES



Then, we will balance imbalance nodes using rotations.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

238

ROTATIONS IN AVL TREES

Balance factor helps in finding the imbalance nodes in the binary search tree.

Balance Factor = leftHeight – rightHeight

Three possible values: -1, 0 or 1.

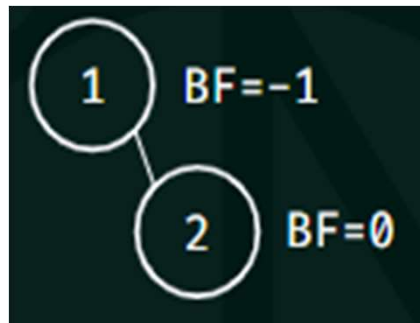
Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

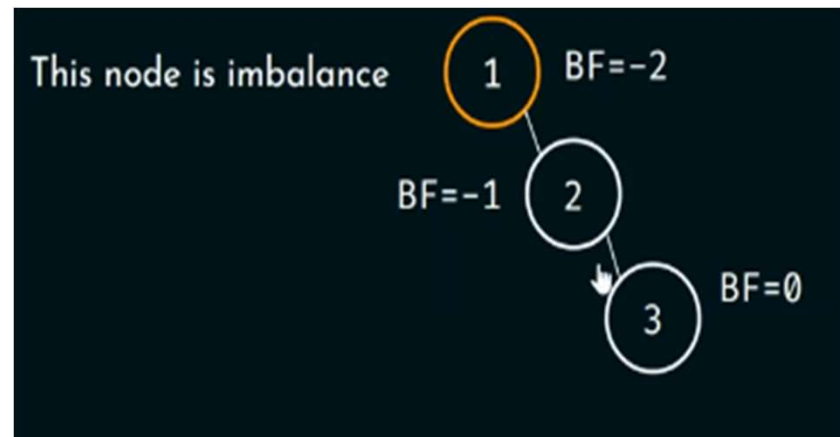
239

ROTATIONS IN AVL TREES

Keys: 1, 2



Keys 1, 2, 3



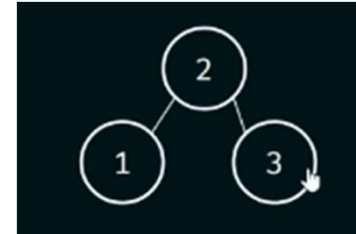
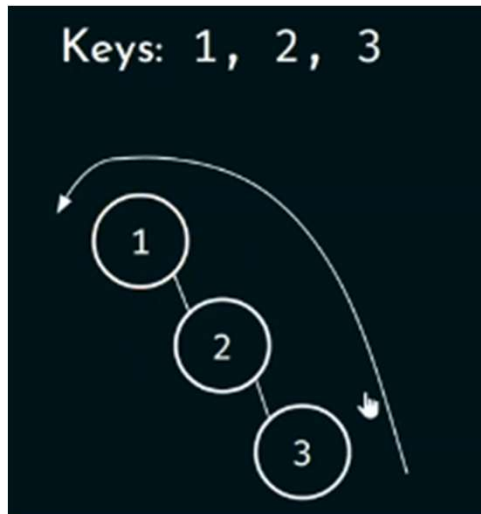
We can perform **left rotation** because the newly inserted node is the right child of the right child of the imbalance node.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

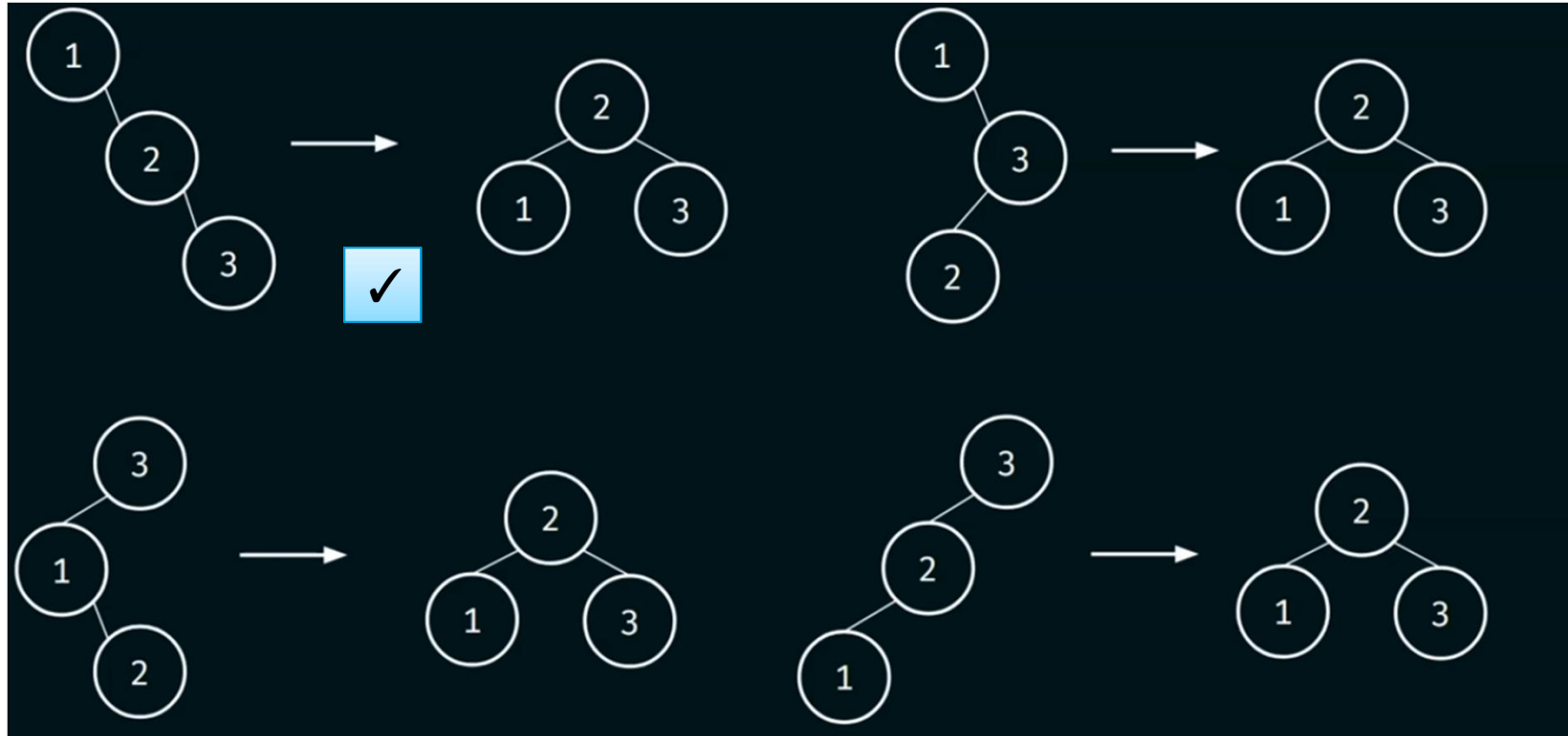
240

ROTATIONS IN AVL TREES



Tree obtained after left rotation

ROTATIONS IN AVL TREES



Each tree on the left can be converted to the tree on the right

Tuesday, August 18, 2026

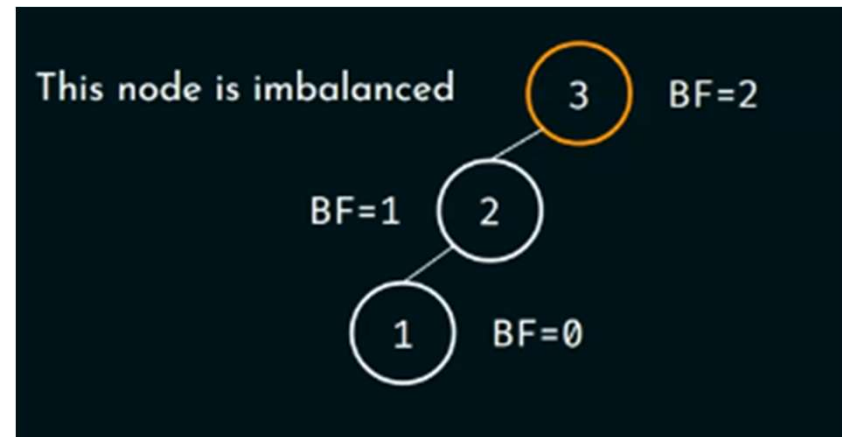
PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

ROTATIONS IN AVL TREES

Keys: 3, 2



Keys: 3, 2, 1



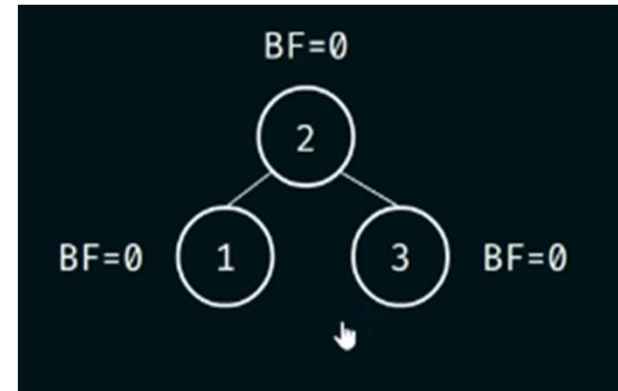
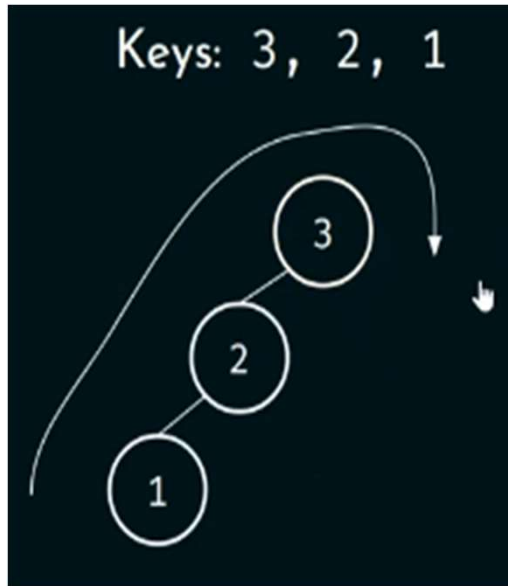
We can perform right rotation about node 3

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

243

ROTATIONS IN AVL TREES

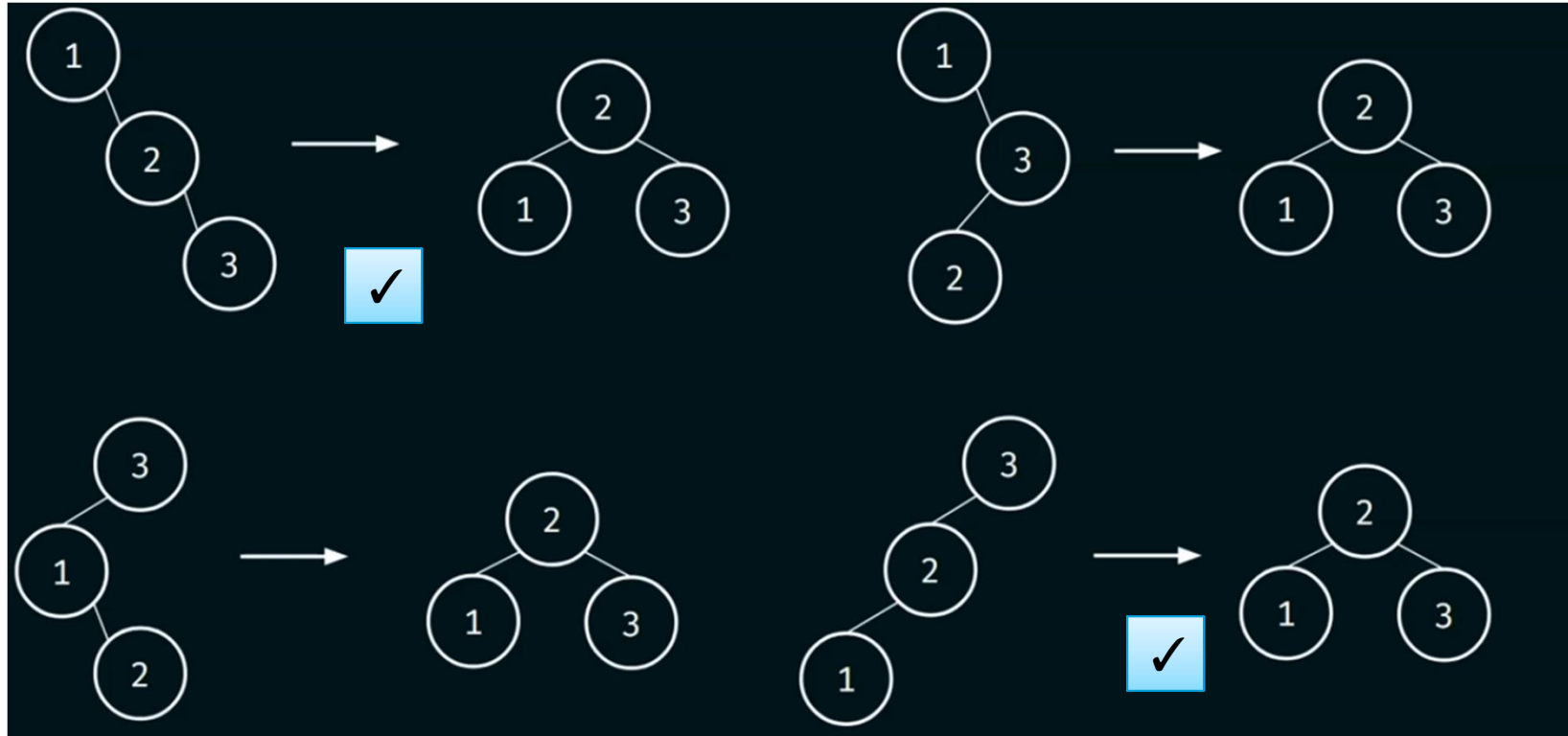


Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

244

ROTATIONS IN AVL TREES



Each tree on the left can be converted to the tree on the right

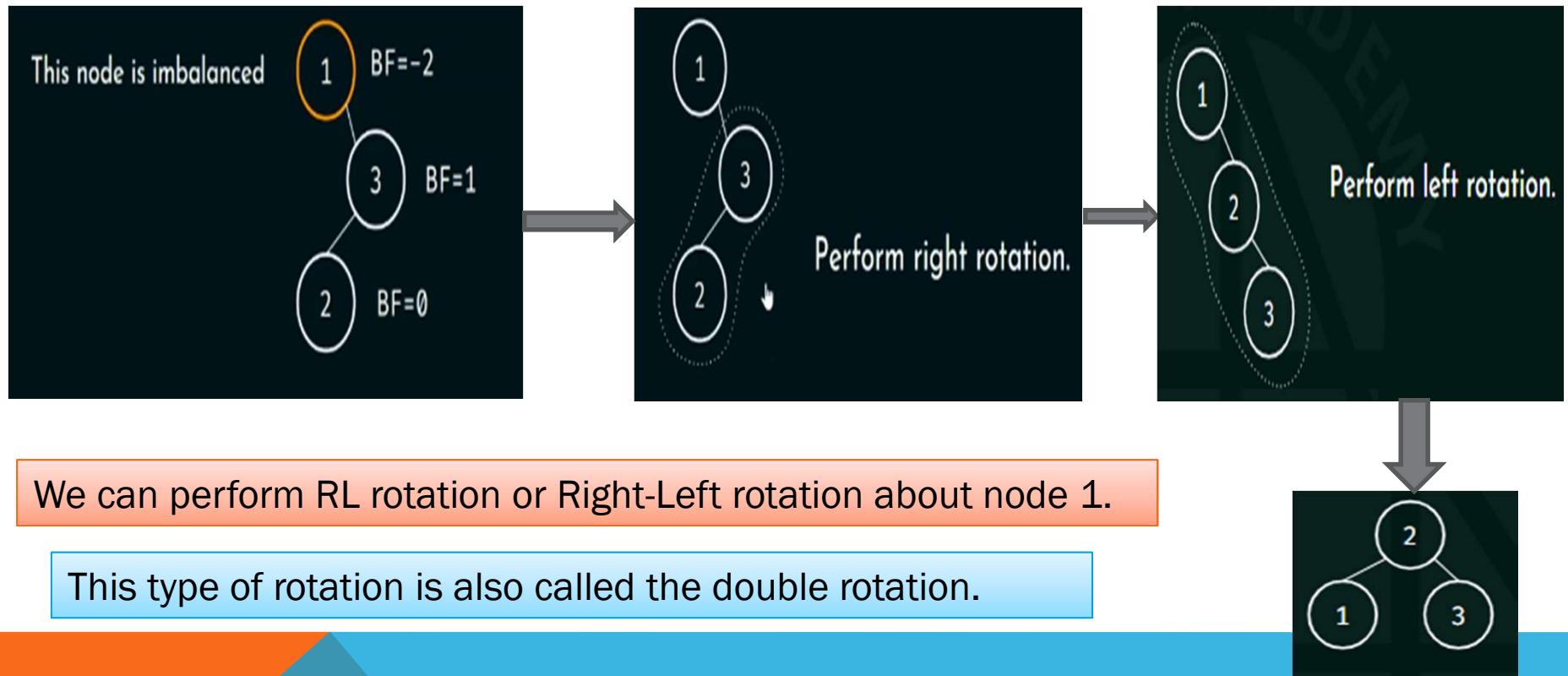
Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

245

ROTATIONS IN AVL TREES

Keys 1, 3, 2



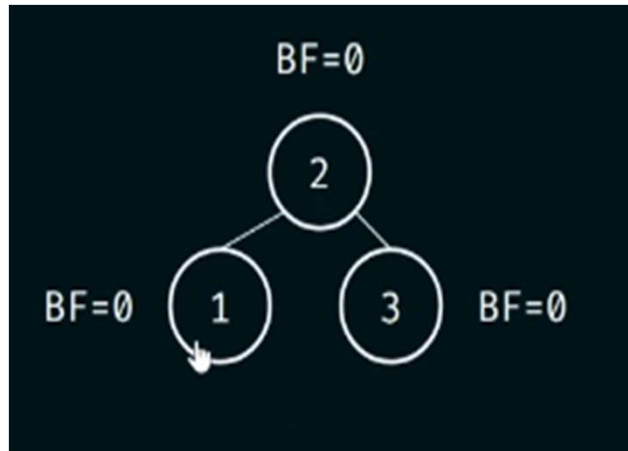
Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

246

ROTATIONS IN AVL TREES

Keys: 1, 3, 2

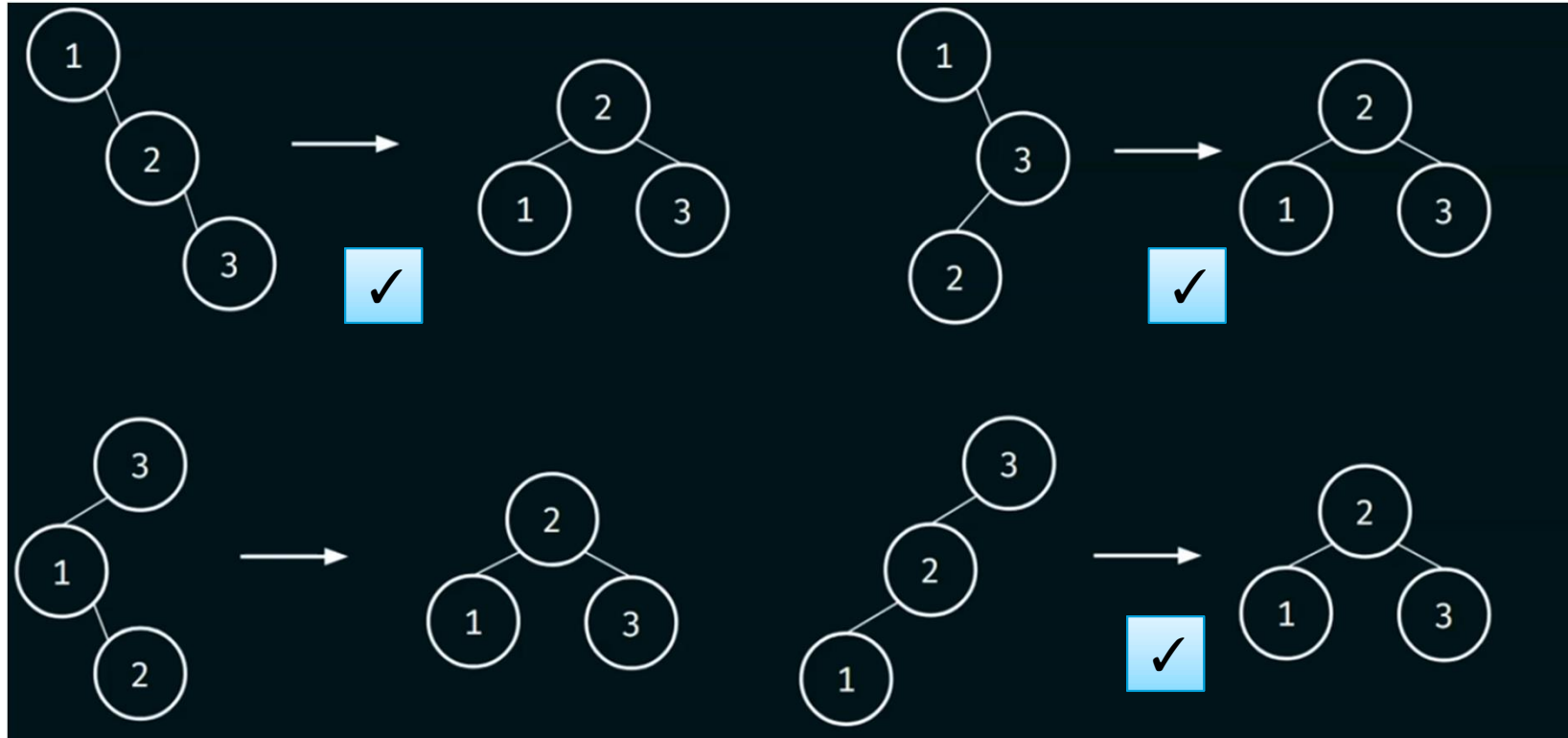


Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

247

ROTATIONS IN AVL TREES



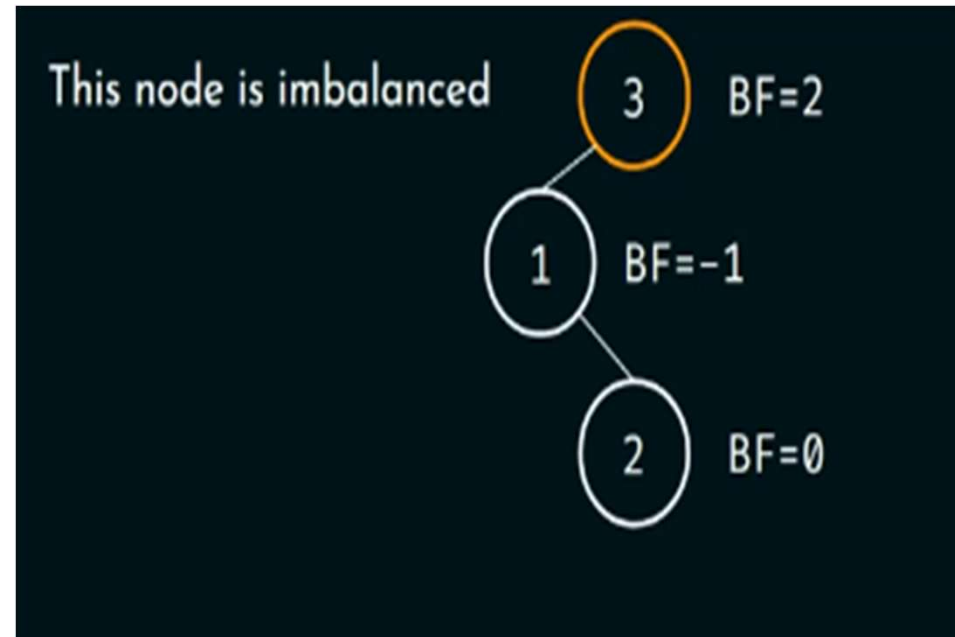
Each tree on the left can be converted to the tree on the right

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

ROTATIONS IN AVL TREES

Keys: 3, 1, 2



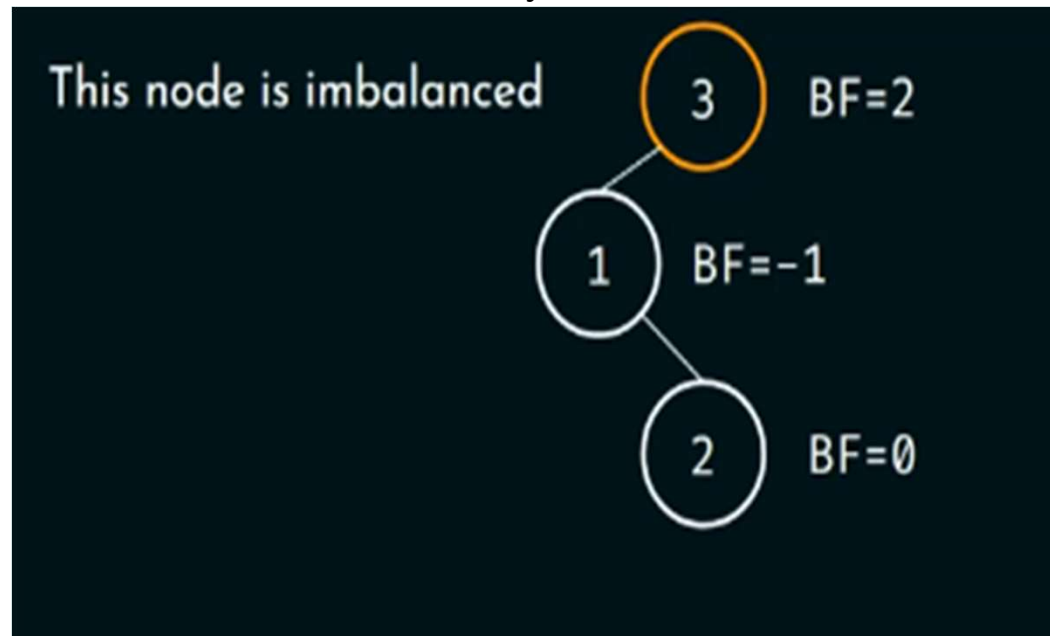
Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

249

ROTATIONS IN AVL TREES

Keys: 3, 1, 2



We can perform LR rotation or Left-Right rotation about node 3.

OR

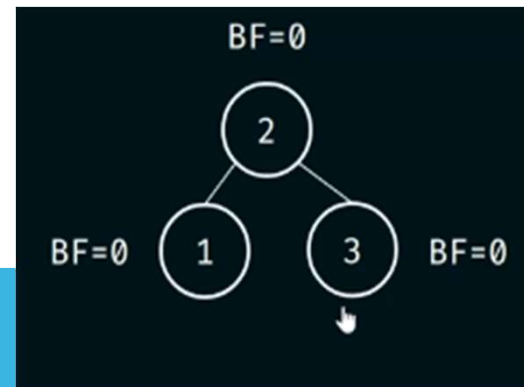
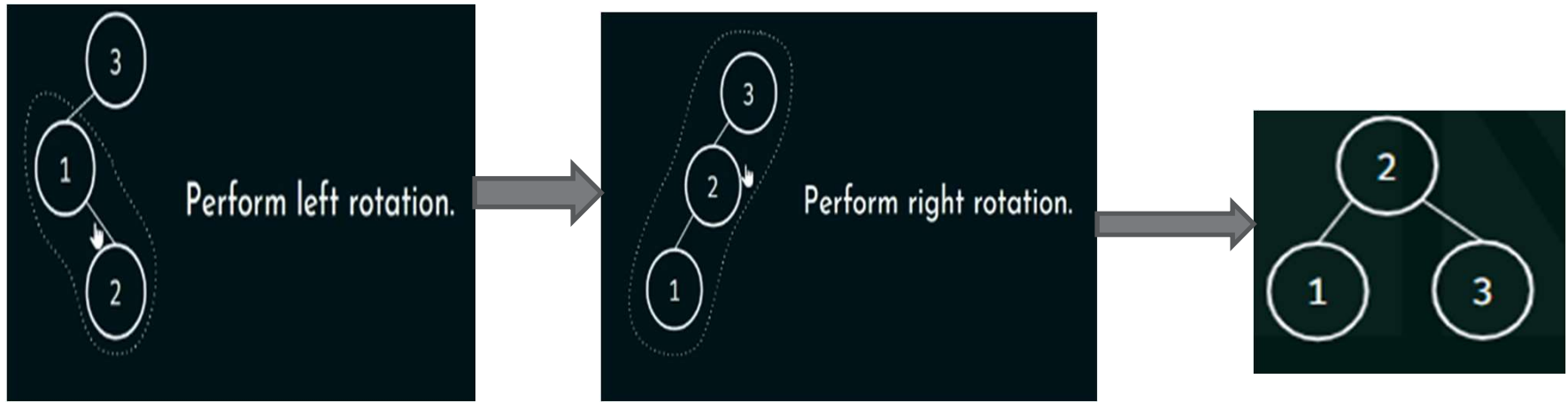
Left rotation about node 1 and Right rotation about node 3

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

ROTATIONS IN AVL TREES

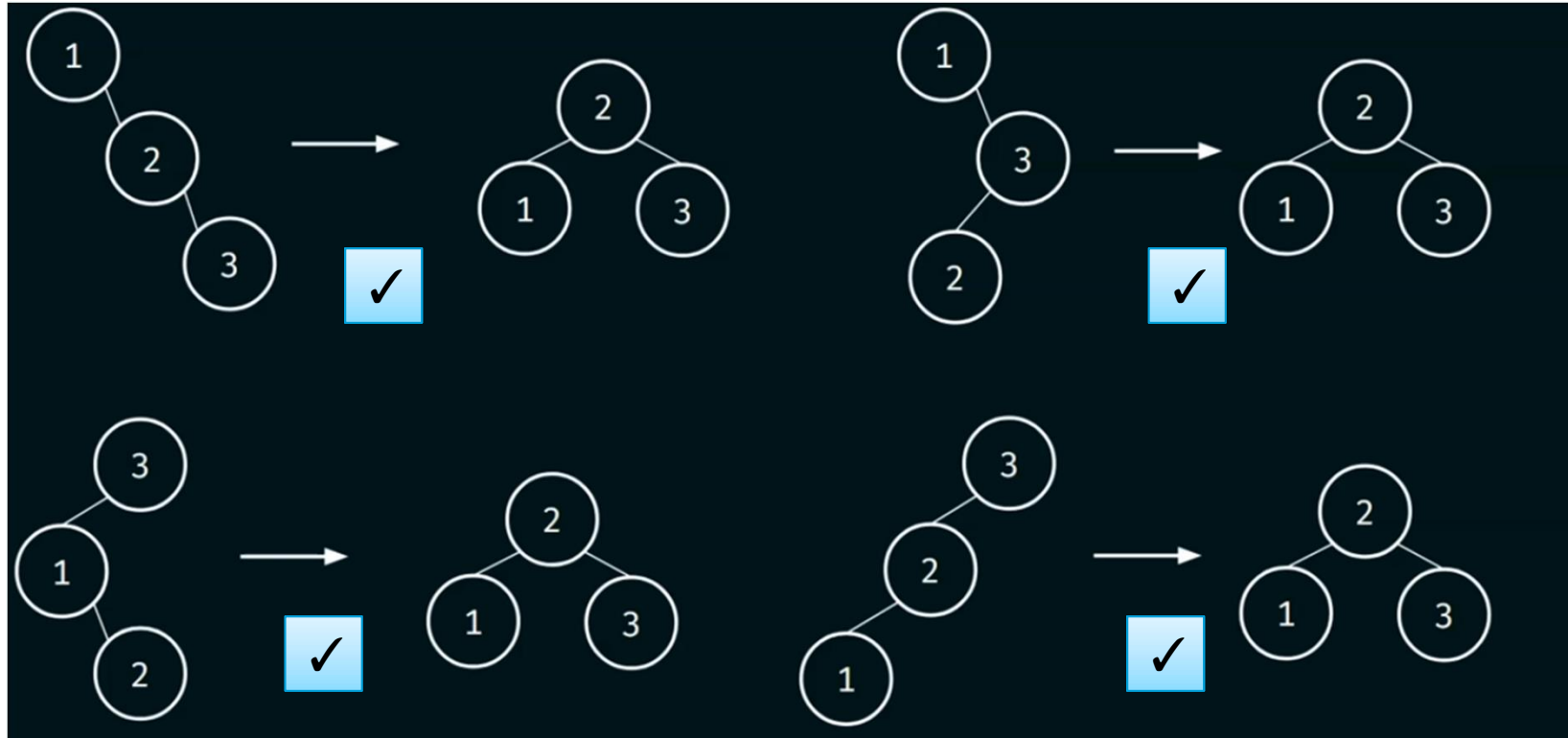
Keys: 3, 1, 2



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

ROTATIONS IN AVL TREES



Each tree on the left can be converted to the tree on the right

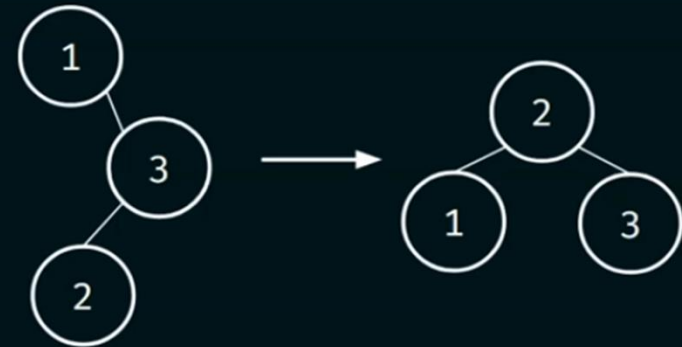
Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

ROTATIONS IN AVL TREES



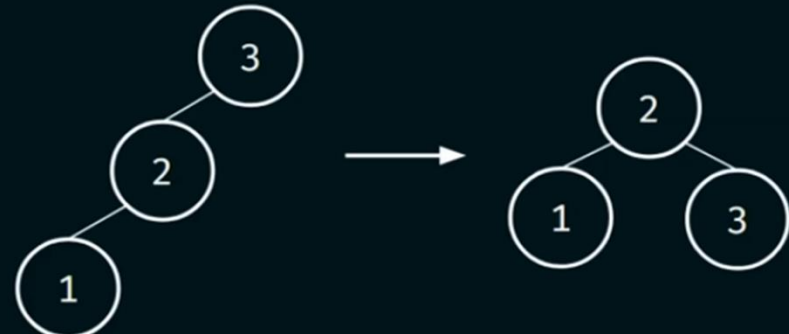
Left rotation



RL rotation



LR rotation



Right rotation

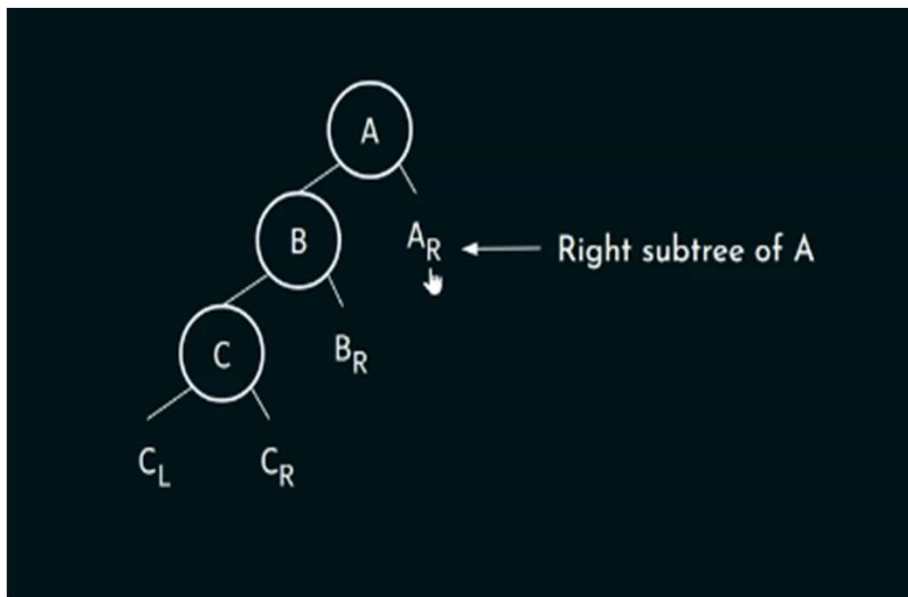
Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

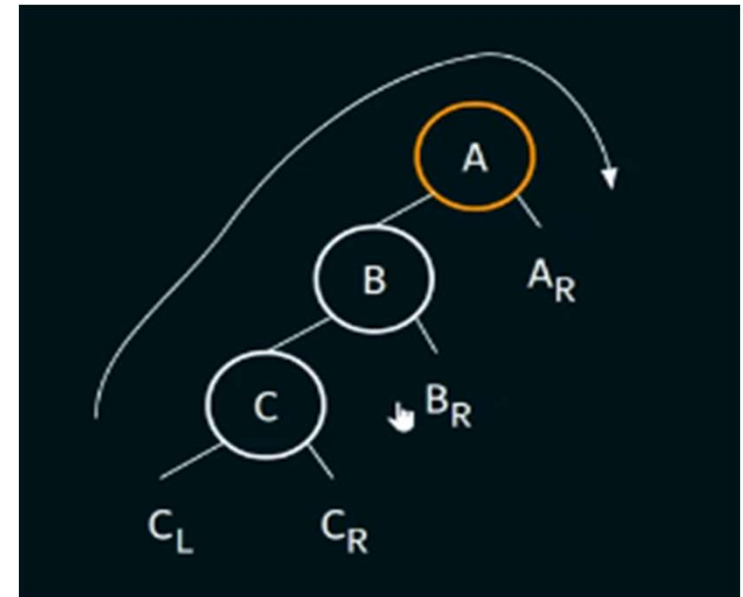
ROTATIONS IN AVL TREES

How to perform rotation(s) on a big-sized tree?

Consider the following tree



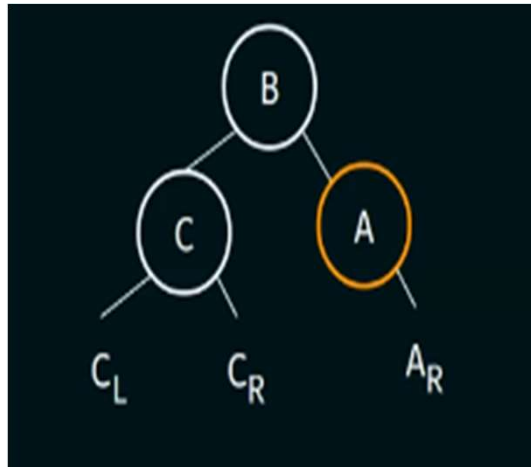
Let say node A is an imbalanced node.



We need to perform right rotation about the node A.

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

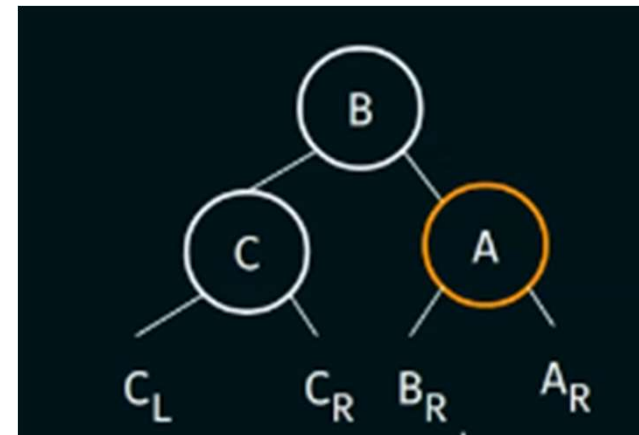
ROTATIONS IN AVL TREES



What about B_R ?

In the original tree, B_R is the right subtree of node B.

Note : Left rotation can also be performed in the same way.

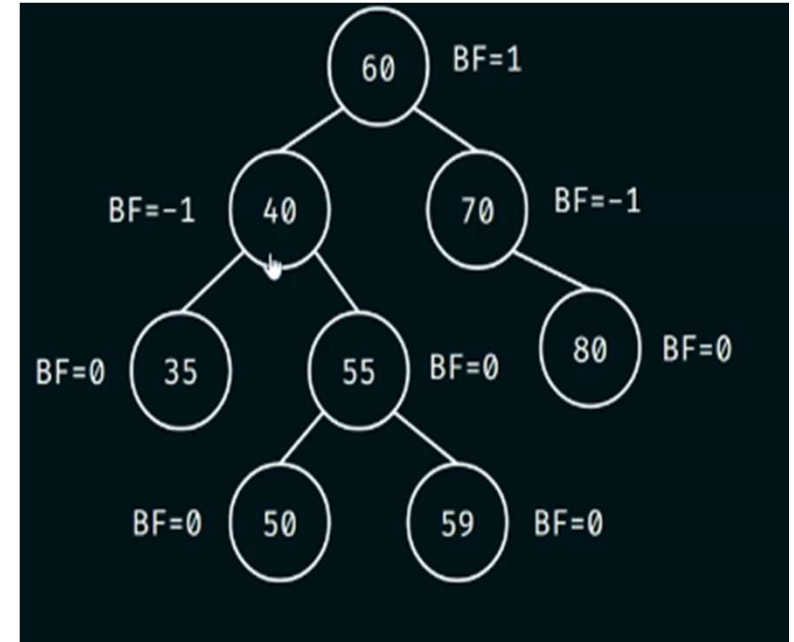
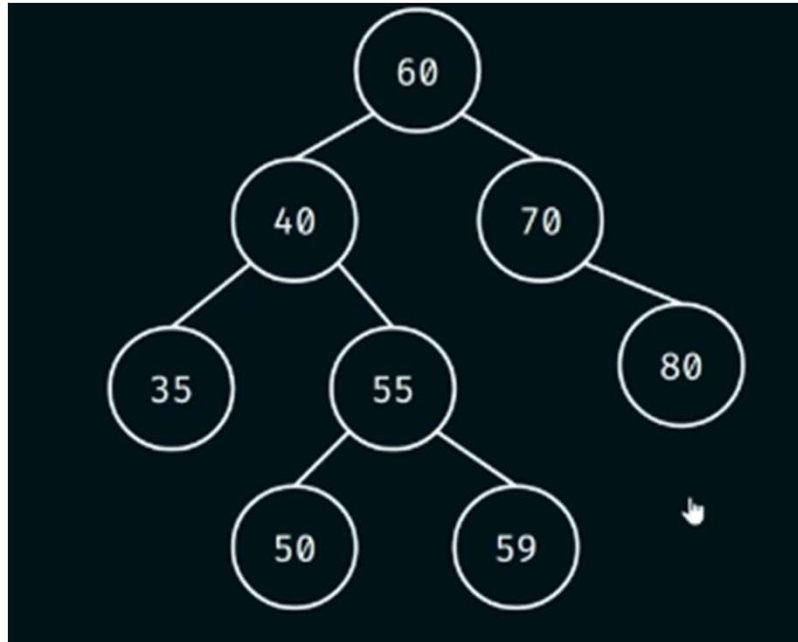


Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

ROTATIONS IN AVL TREES

Example



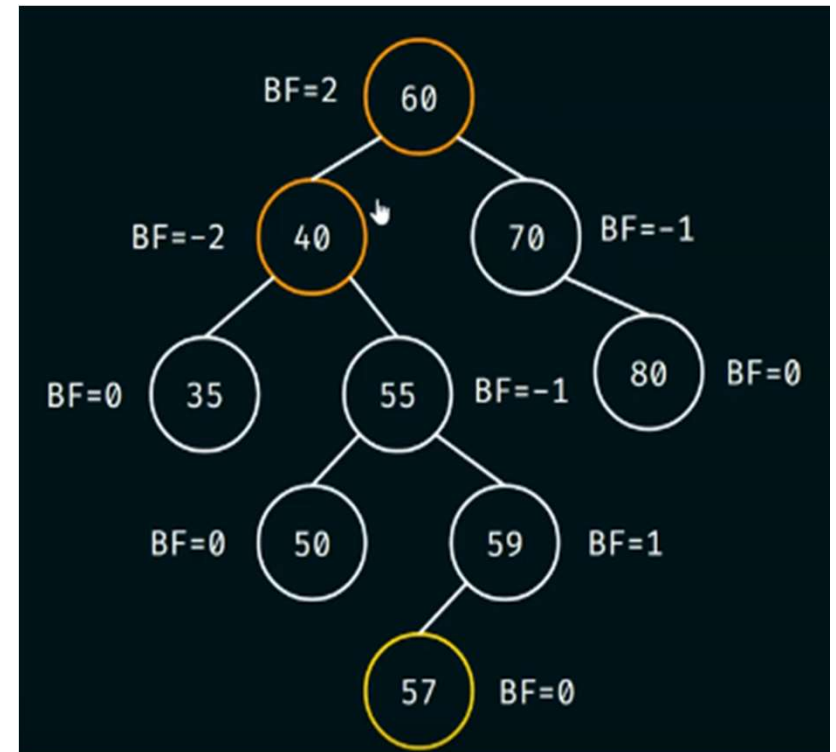
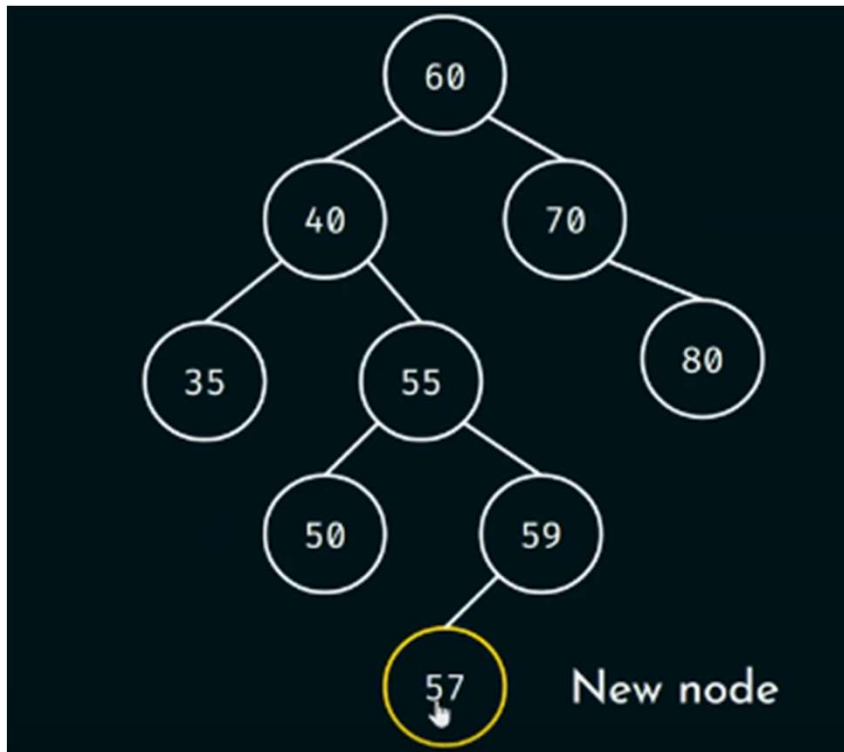
Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

256

ROTATIONS IN AVL TREES

Example



There are two imbalanced nodes.

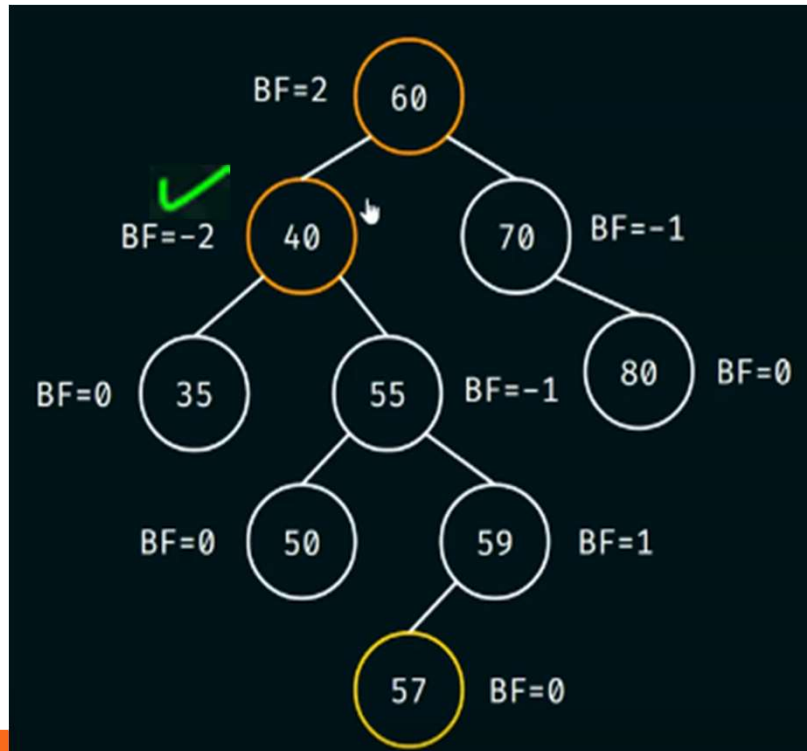
PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

257

Tuesday, August 18, 2026

ROTATIONS IN AVL TREES

Example



Which one to deal with first?

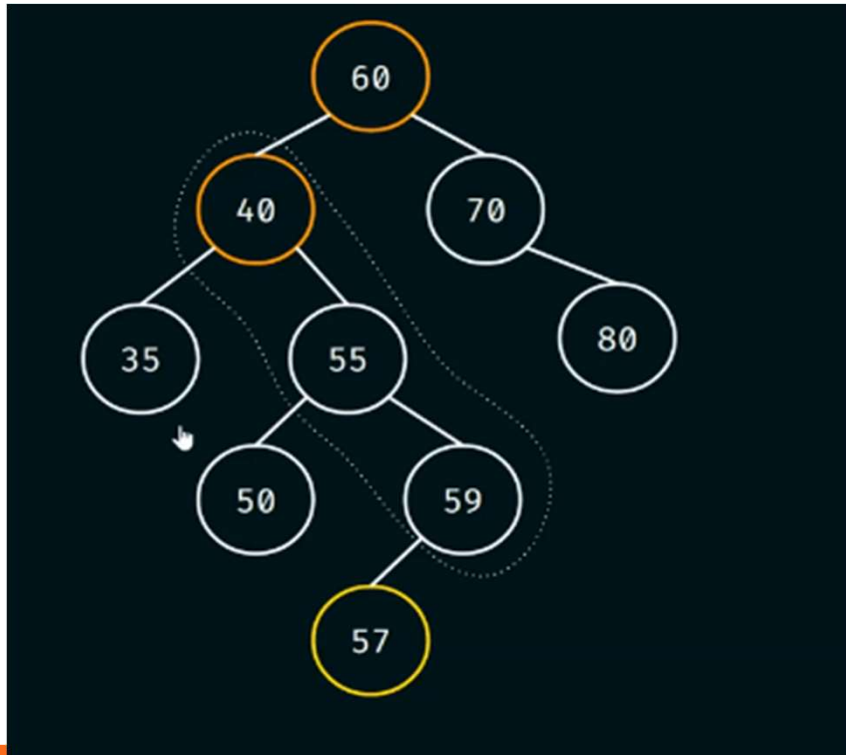
Check where the insertion is happened

Traverse upwards from that node until the first imbalance node appears

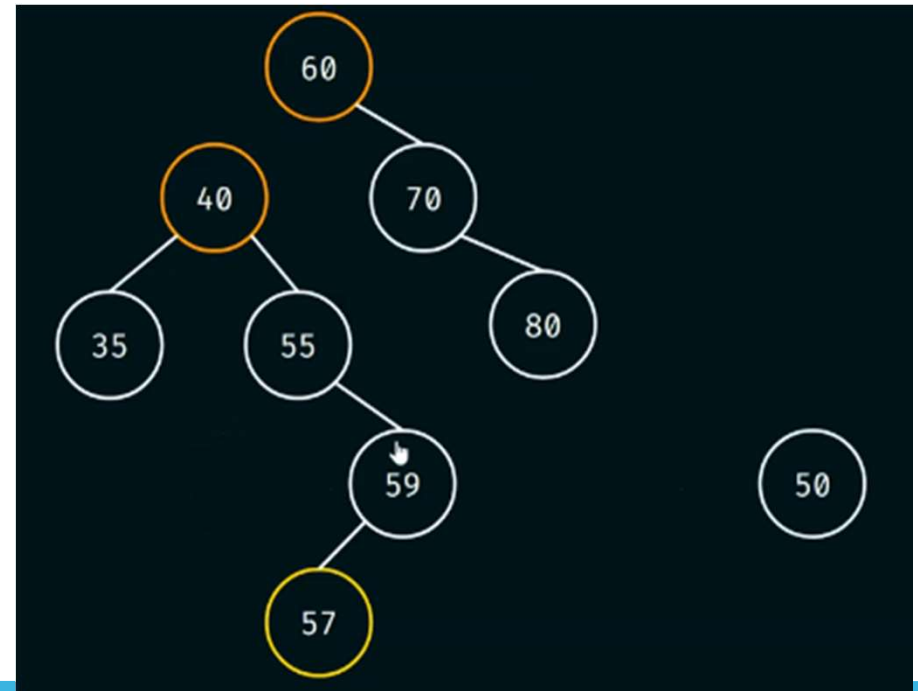
Now, check where the insertion has happened and consider only 3 nodes following the path.

ROTATIONS IN AVL TREES

Example



Perform the left rotation about node 40



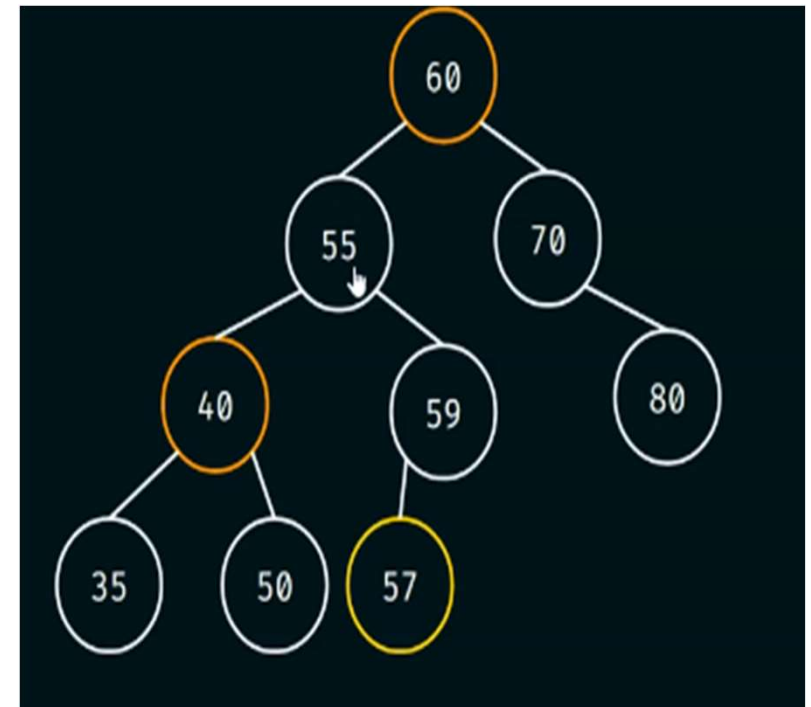
Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

259

ROTATIONS IN AVL TREES

Example

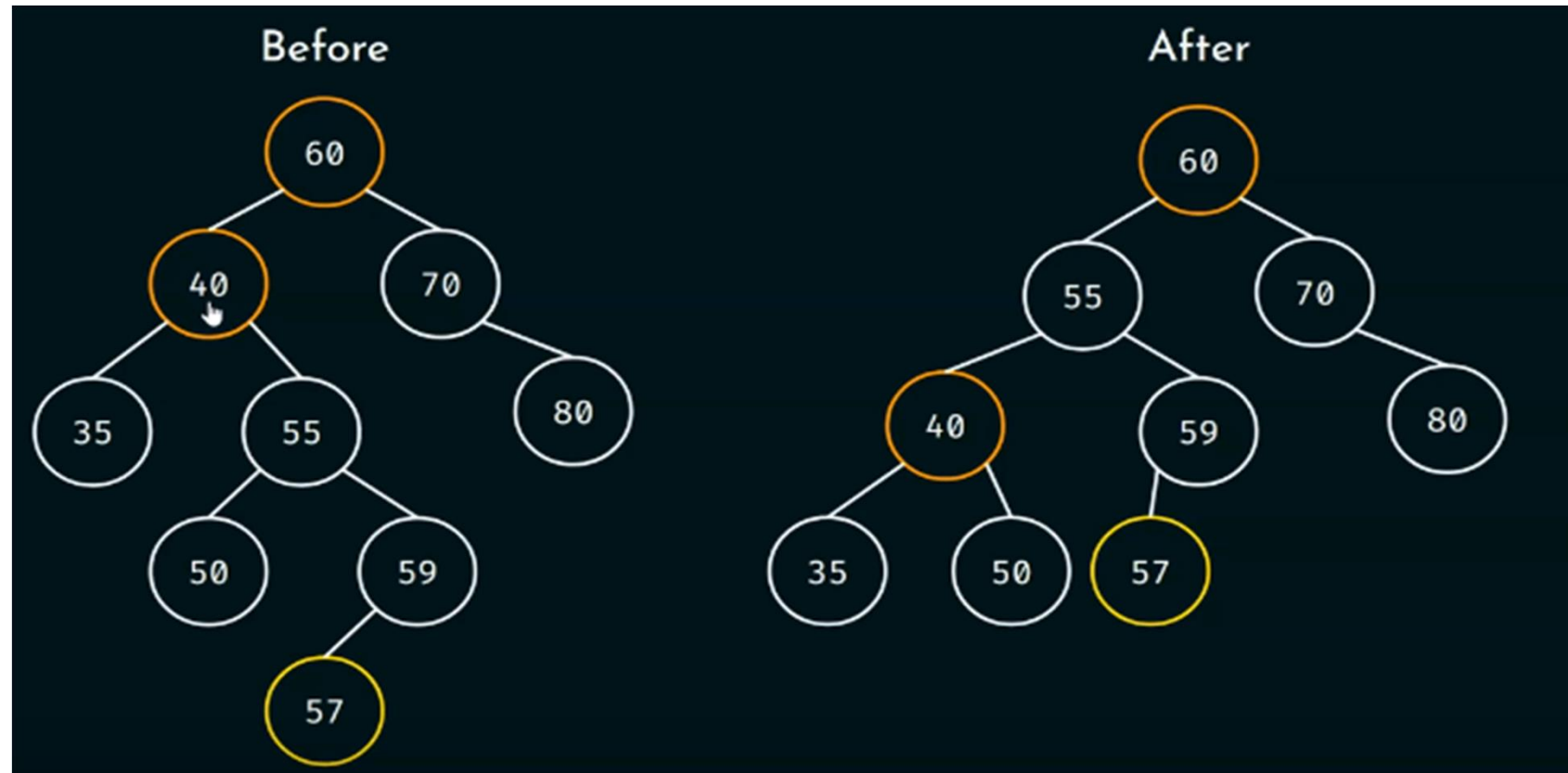


Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

260

ROTATIONS IN AVL TREES

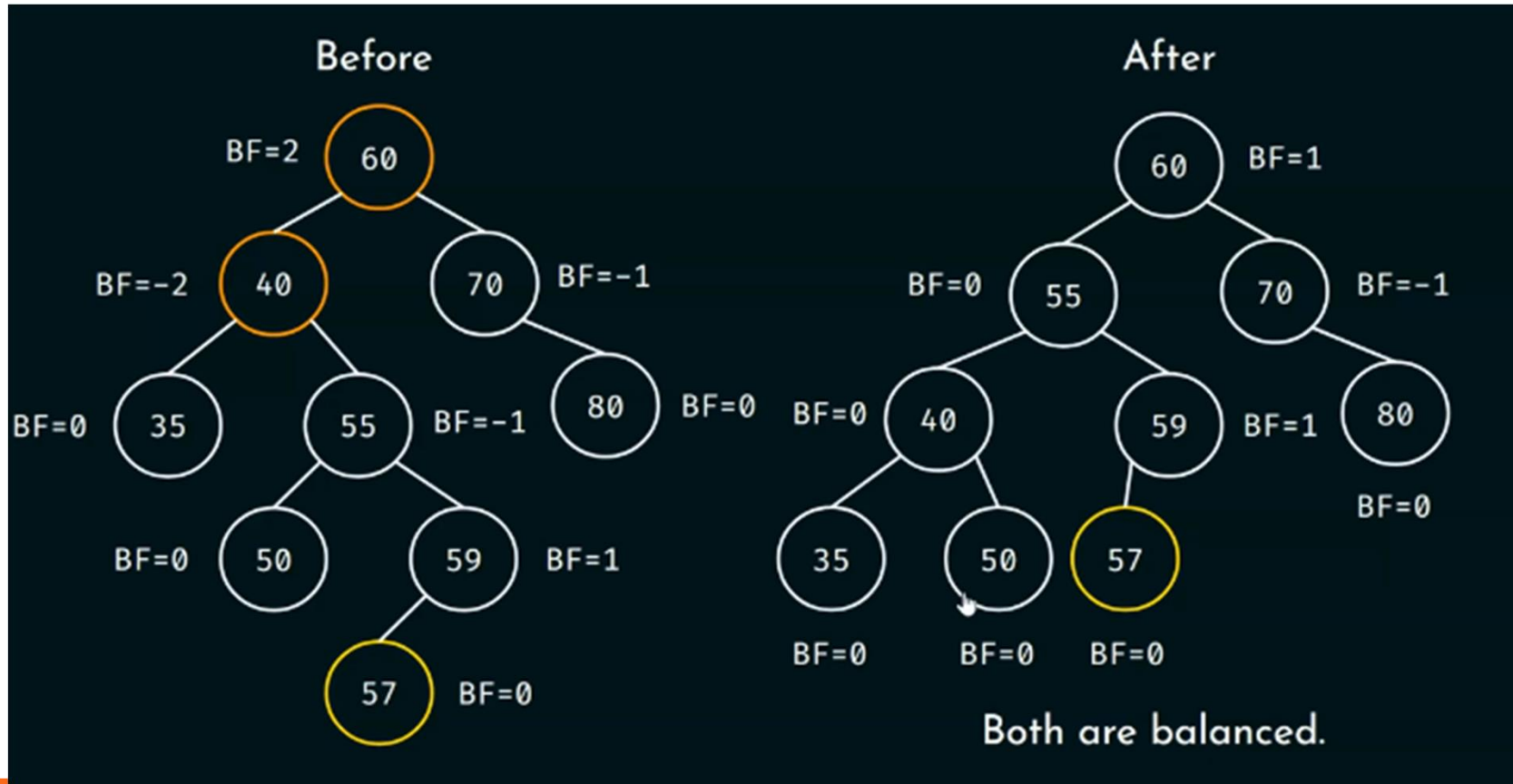


Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

261

ROTATIONS IN AVL TREES



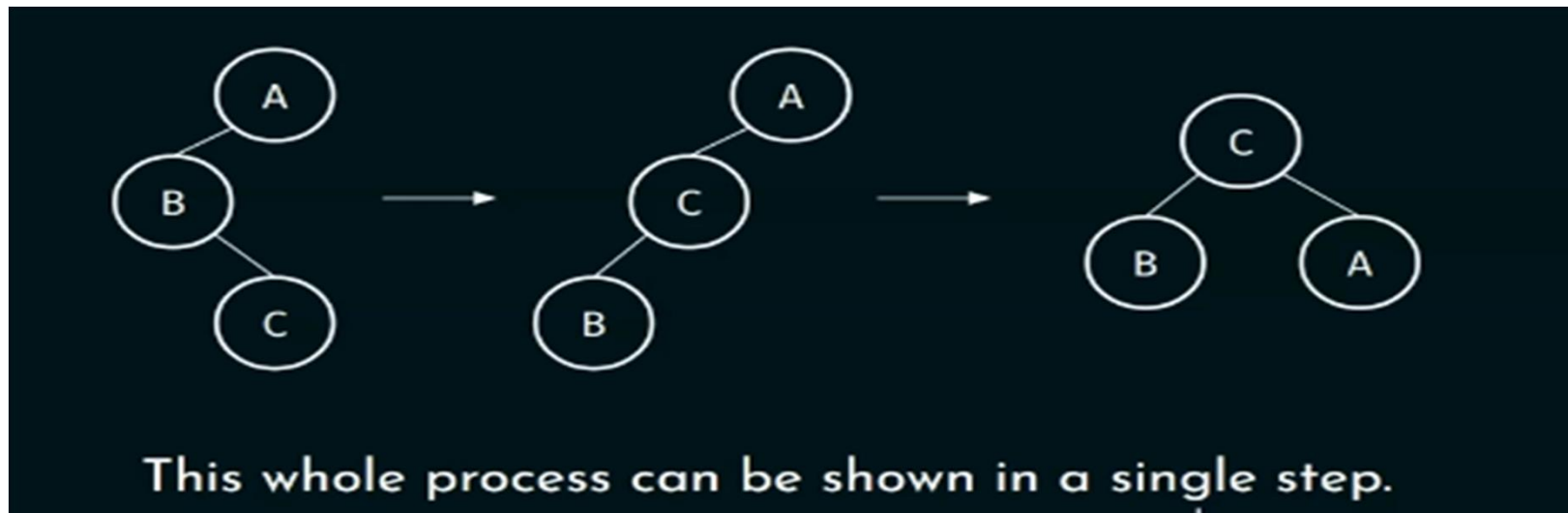
Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

262

ROTATIONS IN AVL TREES

LR Rotation: In order to understand LR rotation on a big-sized tree, We first need to recall how LR rotation was performed on a tree with just three nodes.



Tuesday, August 18, 2026

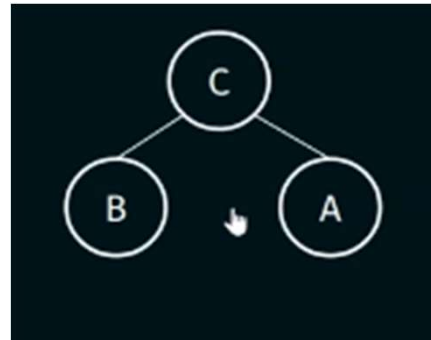
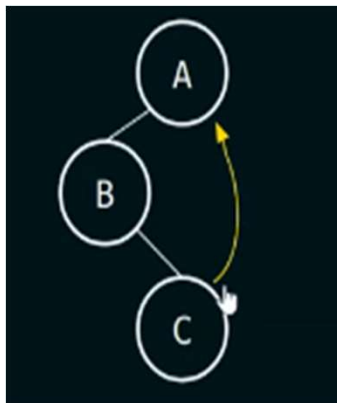
PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

263

ROTATIONS IN AVL TREES

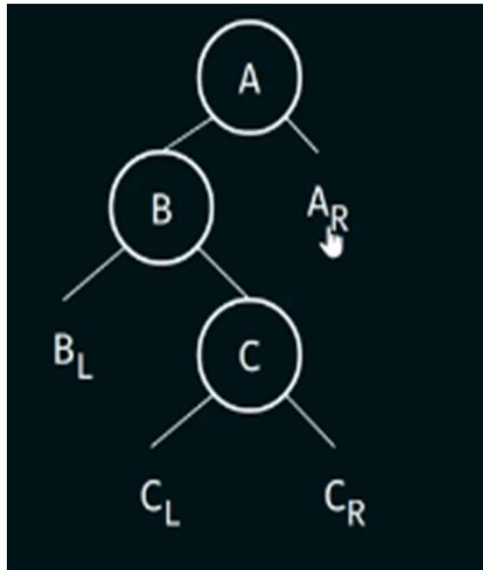
LR Rotation in a single step:

Make the last node, the root node and make the older root node, the right child of the current node.

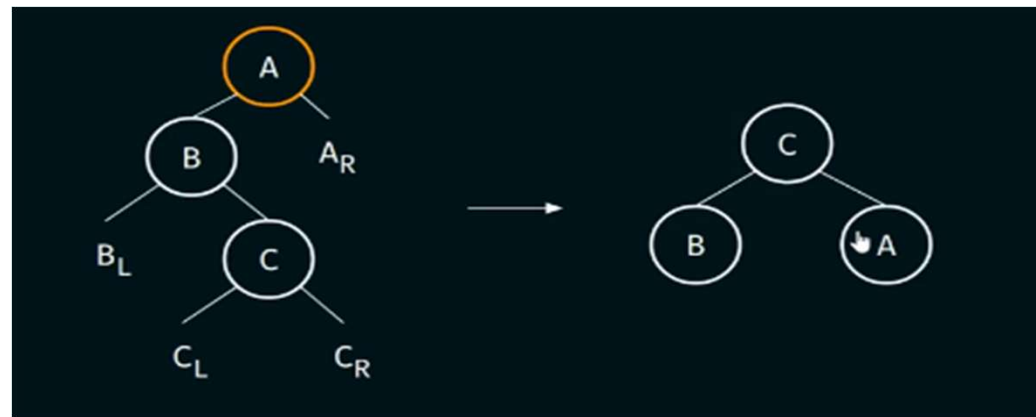


ROTATIONS IN AVL TREES

LR Rotation on a big-sized tree:



Make the last node, the root node and make the older root node, the right child of the current node



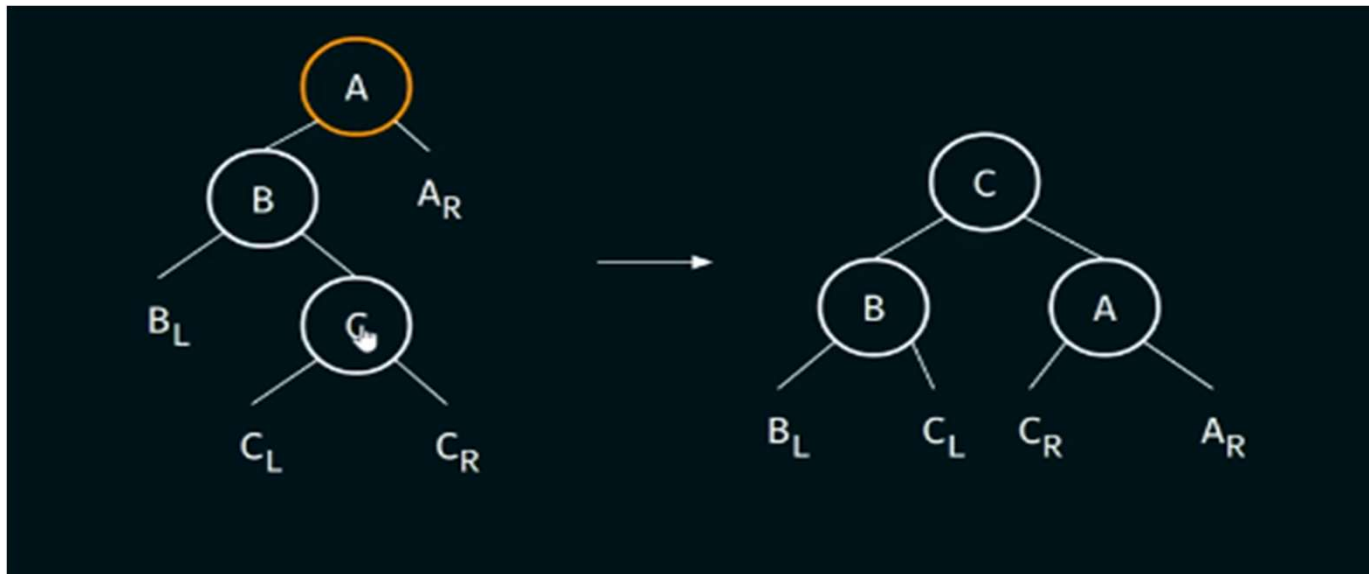
Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

265

ROTATIONS IN AVL TREES

LR Rotation on a big-sized tree:



Same thing can be done with RL rotation as well

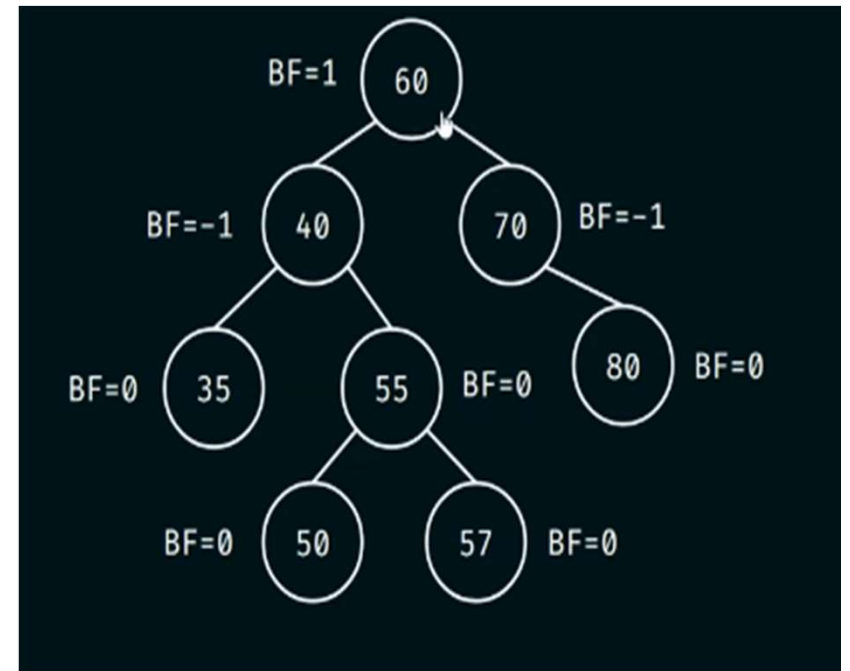
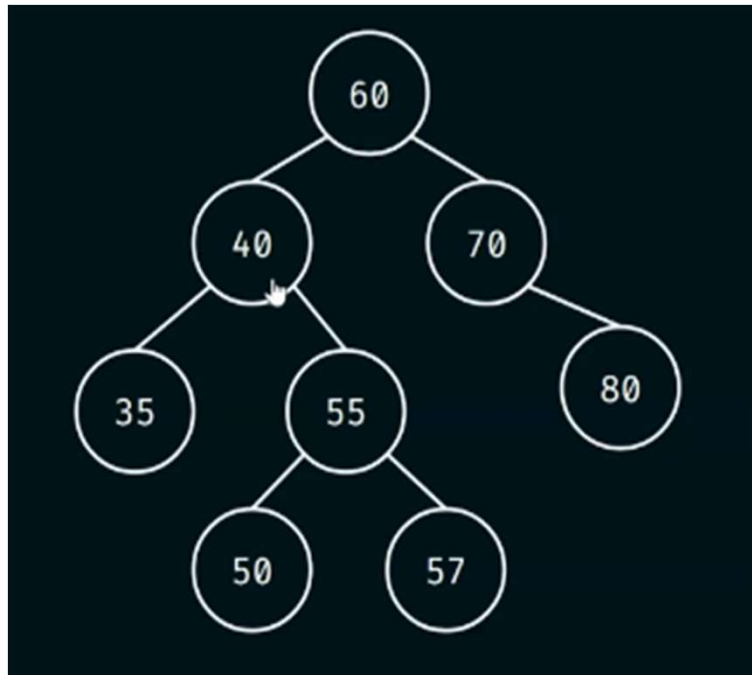
Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

266

ROTATIONS IN AVL TREES

Example:



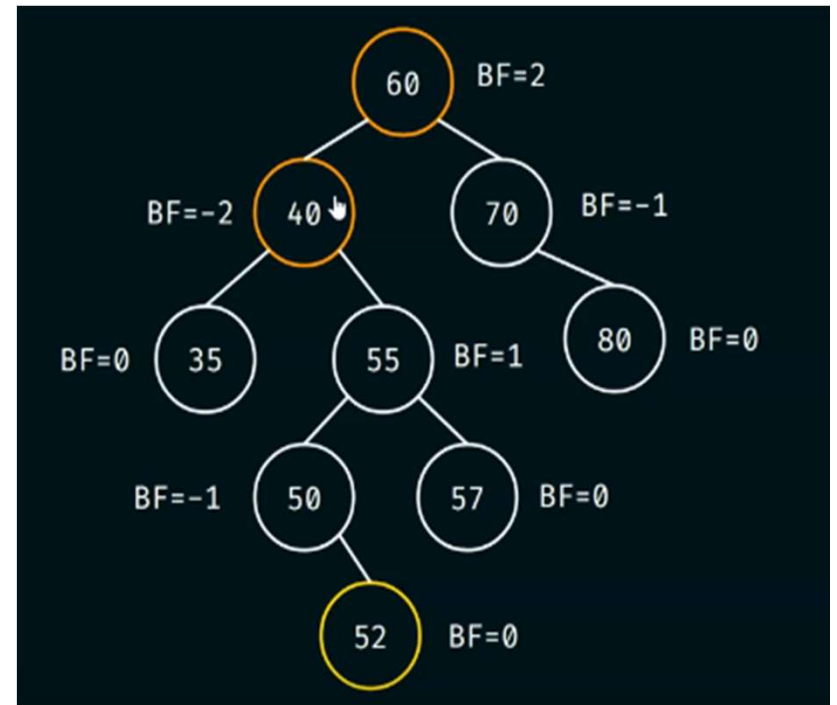
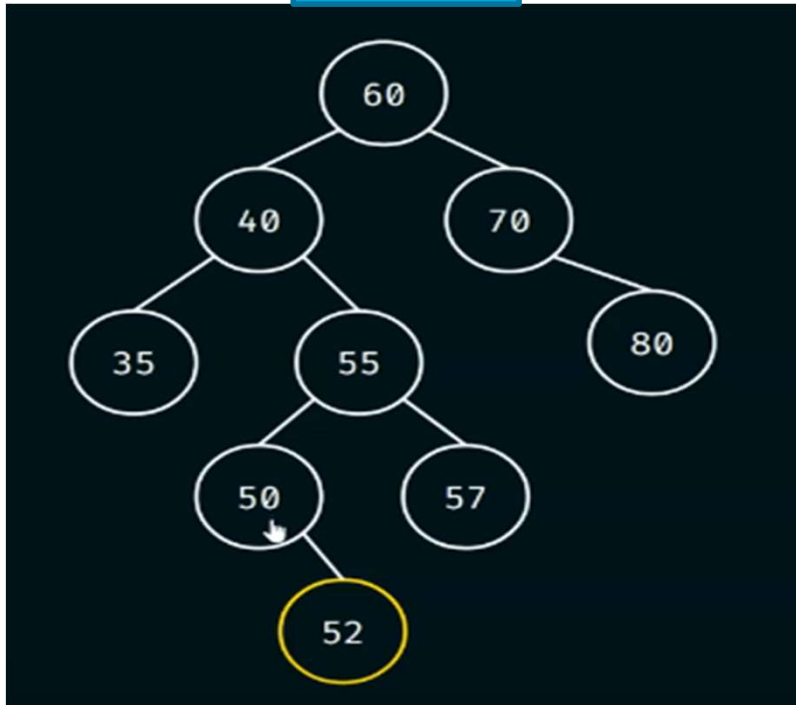
Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

267

ROTATIONS IN AVL TREES

Insert 52

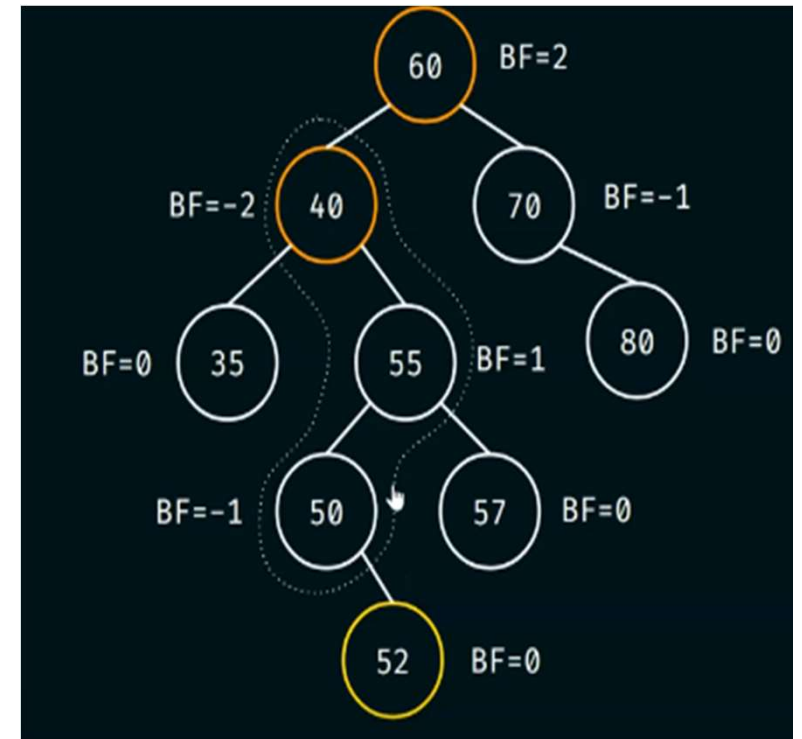
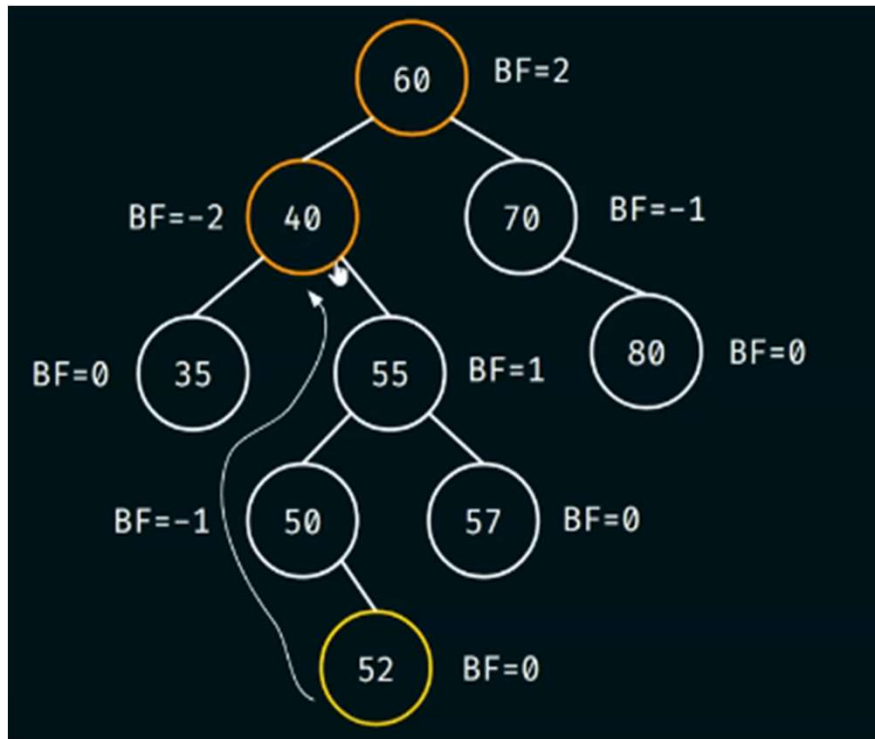


Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

268

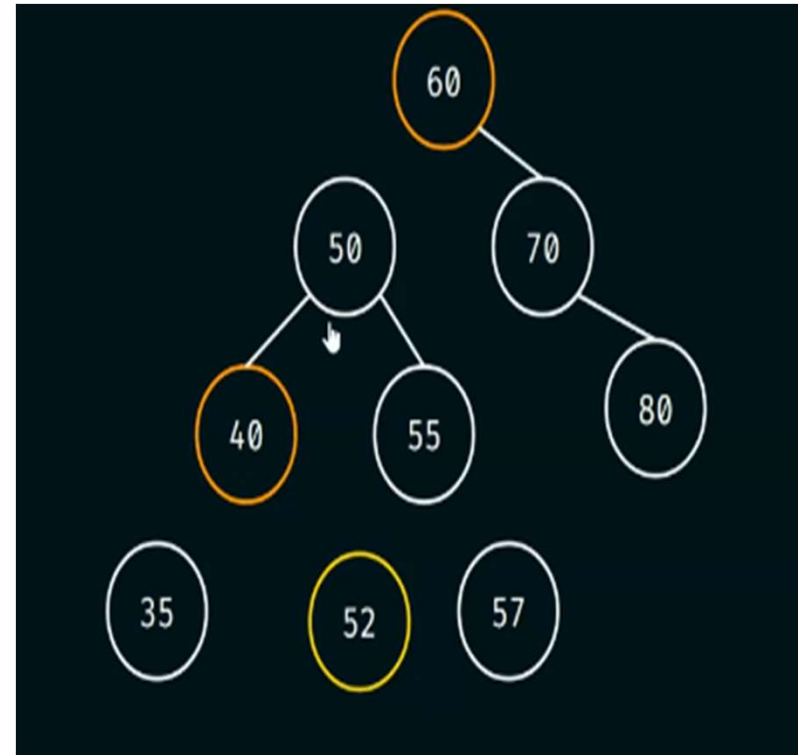
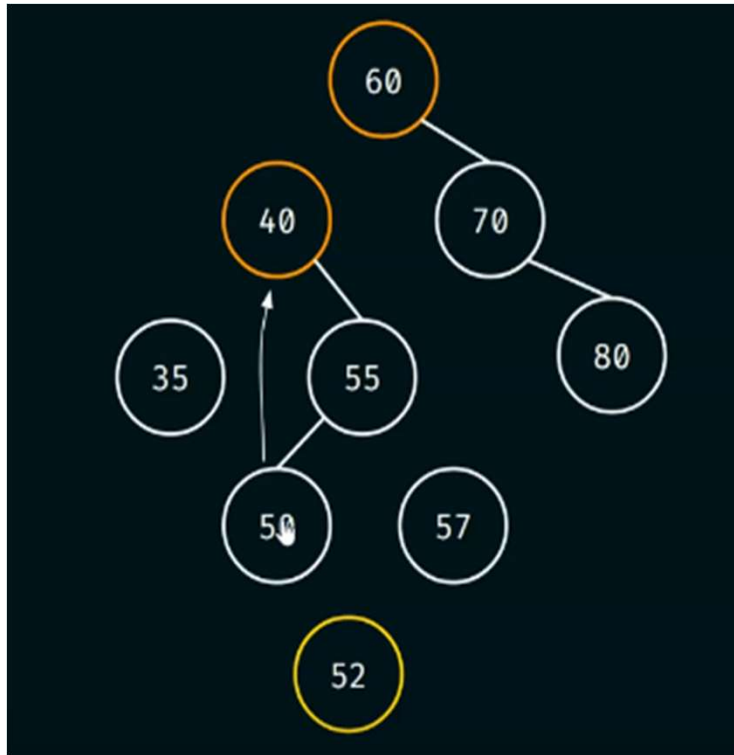
ROTATIONS IN AVL TREES



RL Rotation

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

ROTATIONS IN AVL TREES

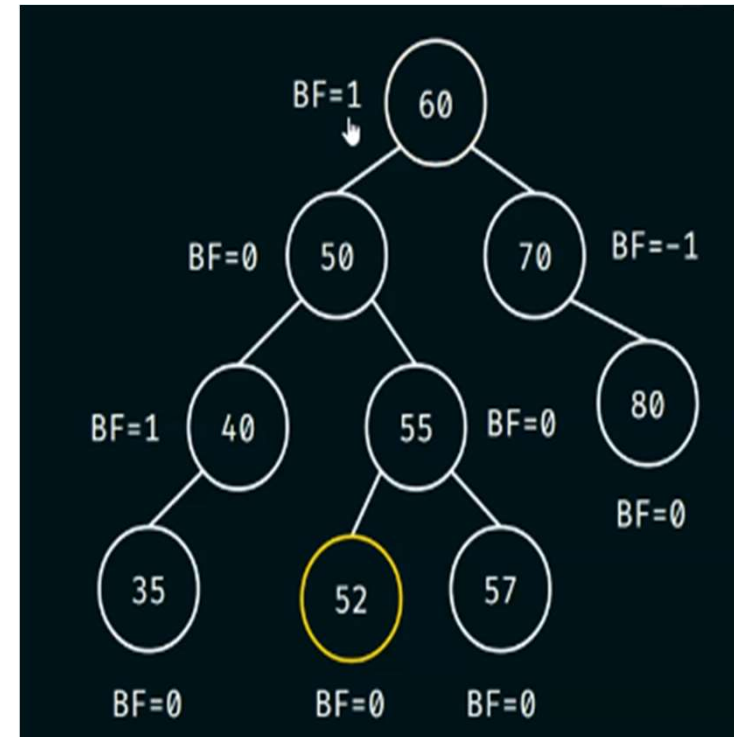
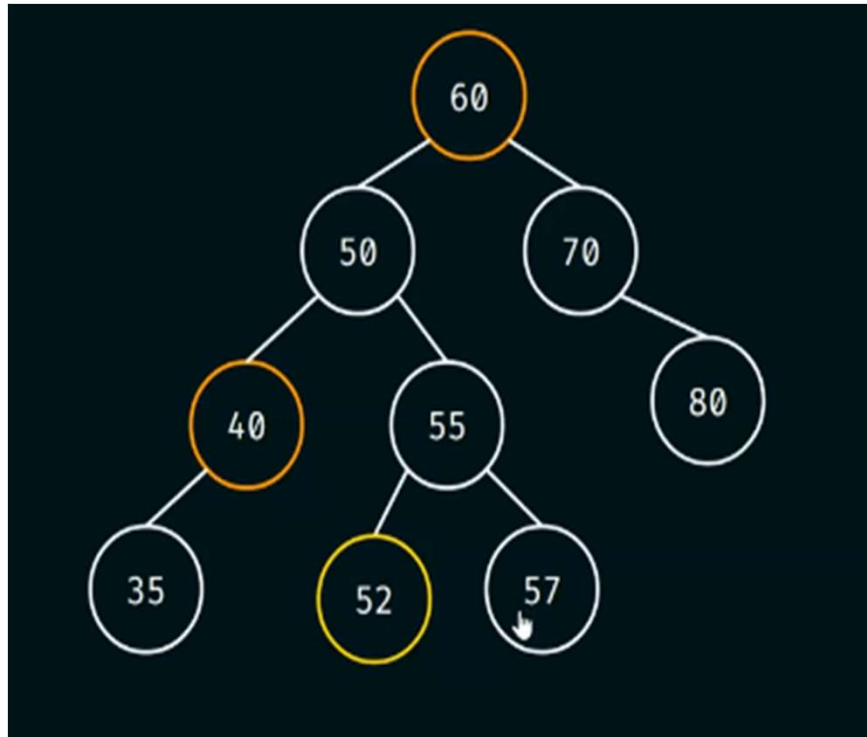


Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

270

ROTATIONS IN AVL TREES

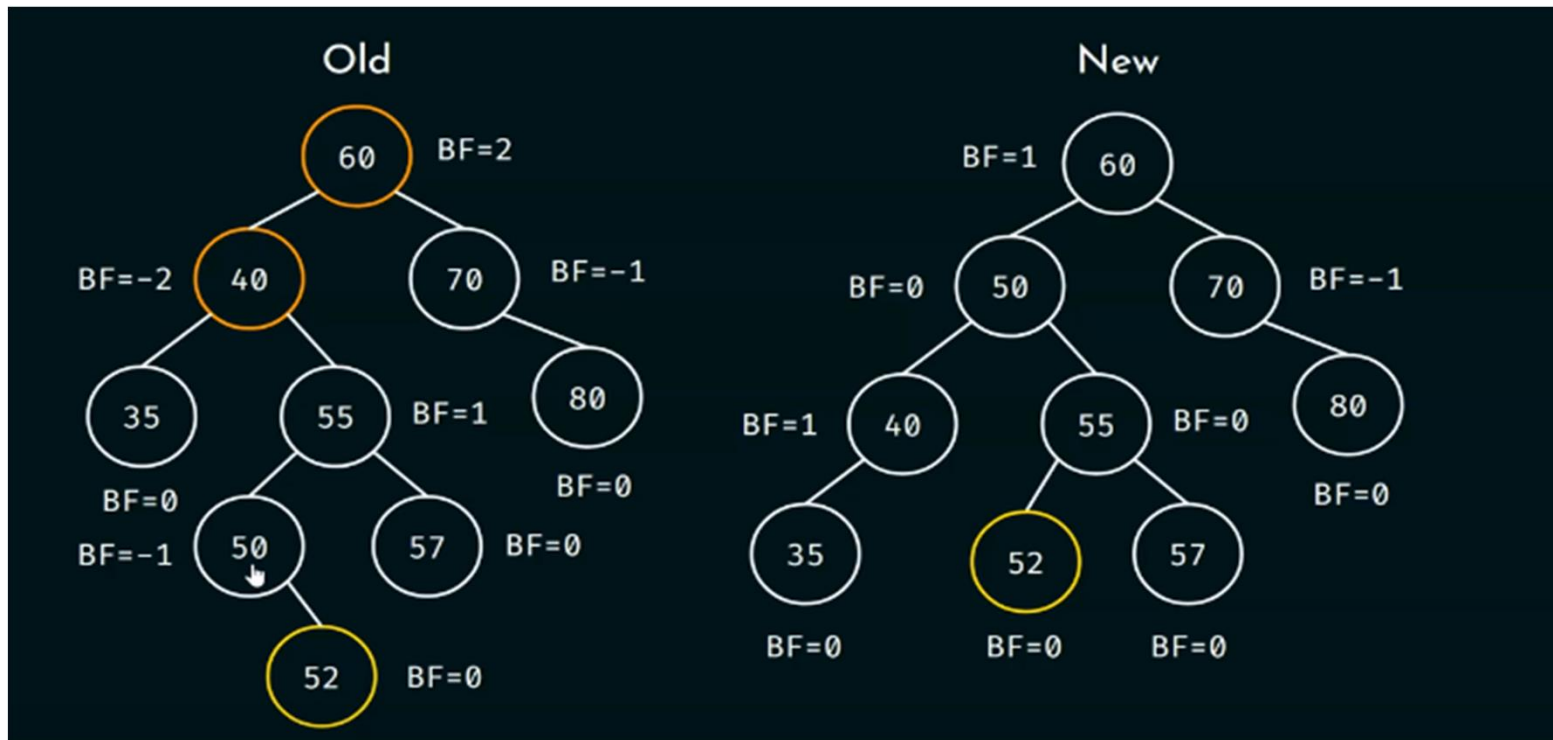


Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

271

ROTATIONS IN AVL TREES



Before Rotation

After Rotation

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

LEFT ROTATION IN AVL TREES (ALGORITHM)

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

273

ALGORITHM

Consider the following tree:

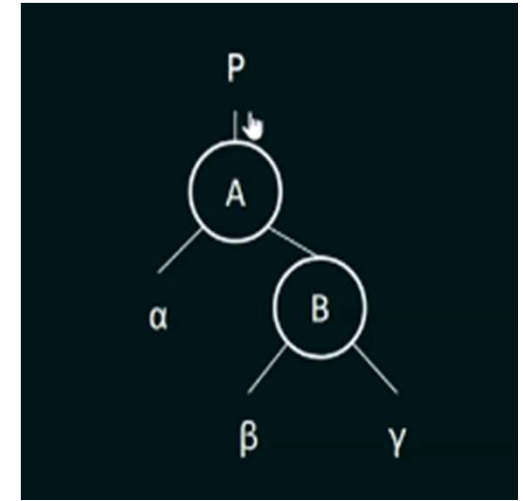
Step 1: If node B has a left subtree, then make β the right subtree of A.

Step 2: If P is NULL, then make B as the root of the tree.

Step 3: If P is not NULL, then check the following:

- a) If A is the left child of P, then make B as the left child of P.
- b) Else make B as the right child of P.

Step 4: Make B as the parent of A.



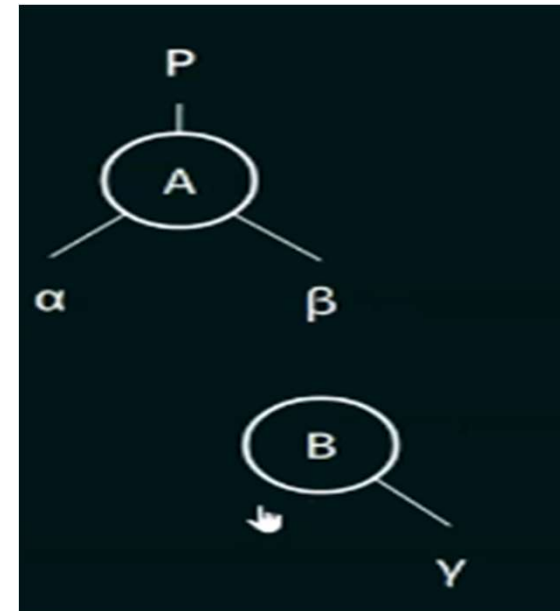
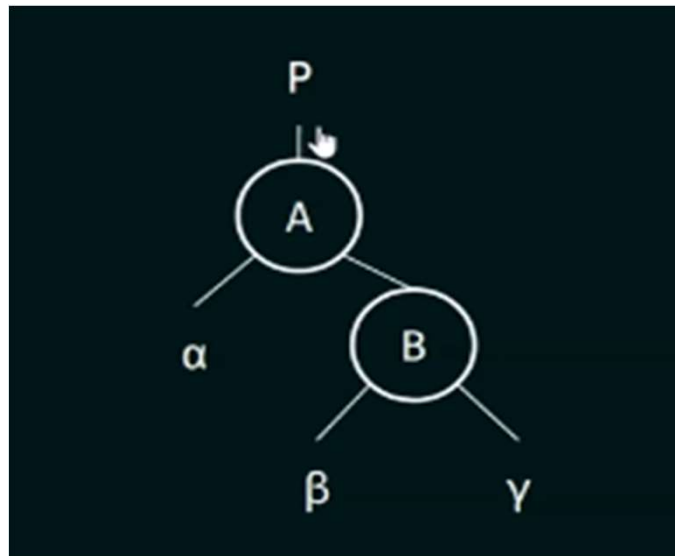
Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

274

ALGORITHM

Step 1: If node B has a left subtree, then make β the right subtree of A



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

275

ALGORITHM

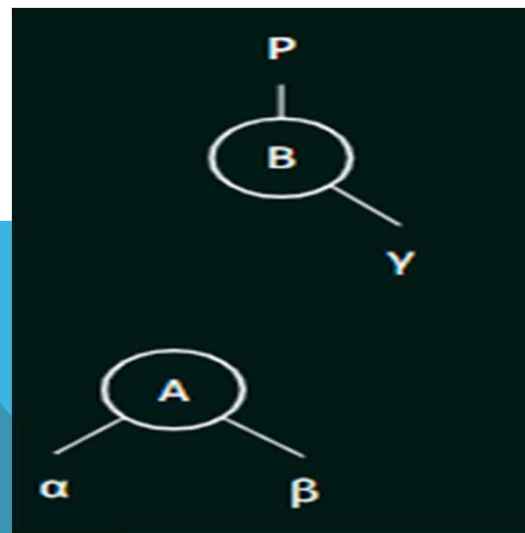
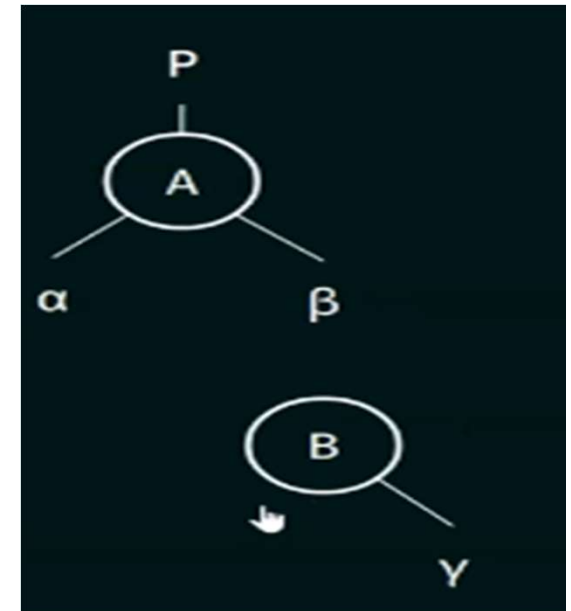
Step 2: If P is NULL, then make B as the root of the tree.

Let say P is not NULL

Step 3: If P is not NULL, then check the following:

Let say A is the left child of P

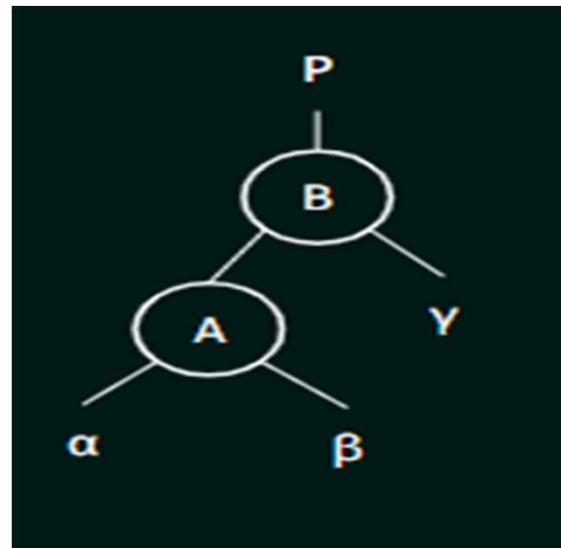
a) If A is the left child of P, then make B as the left child of P.



PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

ALGORITHM

Step 4: Make B as the parent of A.



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

277

LEFT ROTATION IN AVL TREES (PROGRAM)

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

278

ALGORITHM FOR LEFT ROTATION

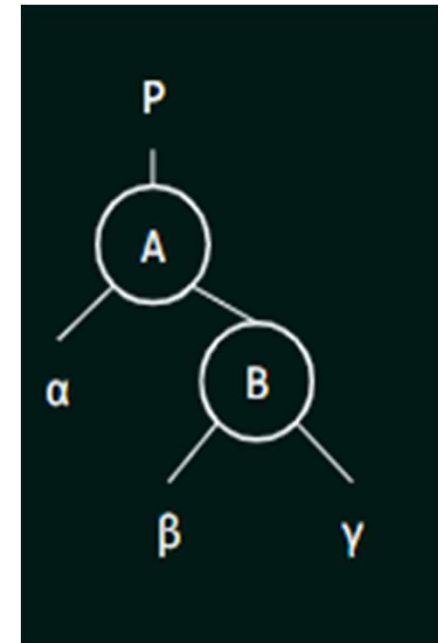
Step 1: If node B has a left subtree, then make β the right subtree of A.

Step 2: If P is NULL, then make B as the root of the tree.

Step 3: If P is not NULL, then check the following:

- a) If A is the left child of P, then make B as the left child of P.
- b) Else make B as the right child of P.

Step 4: Make B as the parent of A.

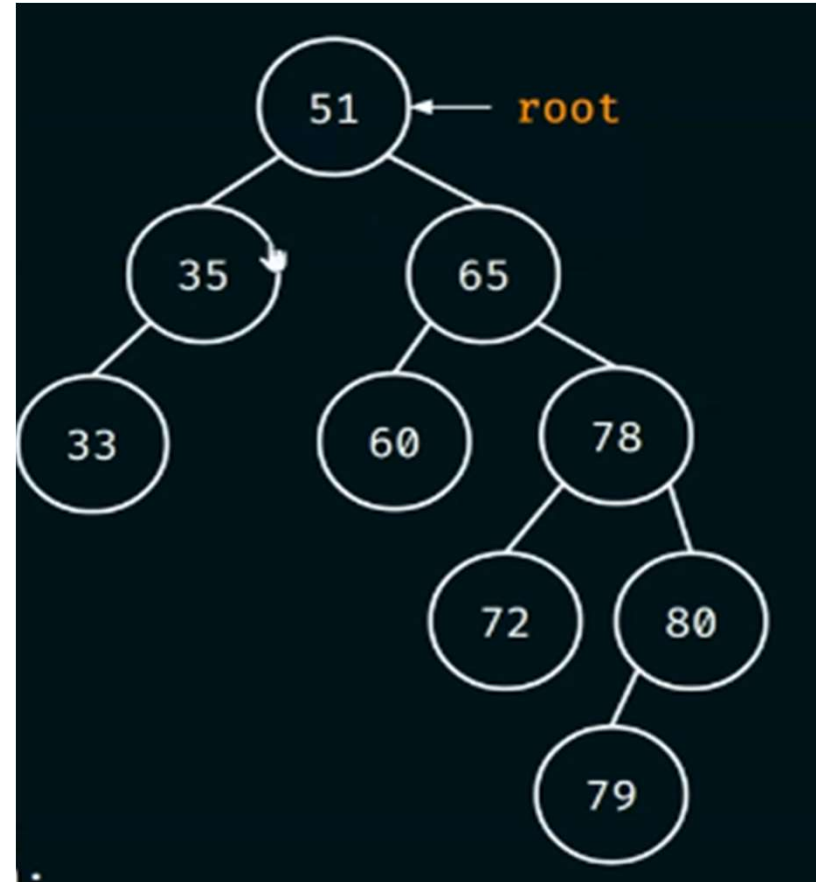


PROGRAM

```
struct node {
    struct node* left;
    int data;
    struct node* right;
};

int main()
{
    struct node* root = NULL;
    root = createNode(51);
    root->left = createNode(35);
    root->left->left = createNode(33);

    root->right = createNode(65);
    root->right->left = createNode(60);
    root->right->right = createNode(78);
    root->right->right->left = createNode(72);
    root->right->right->right = createNode(80);
    root->right->right->right->left = createNode(79);
}
```



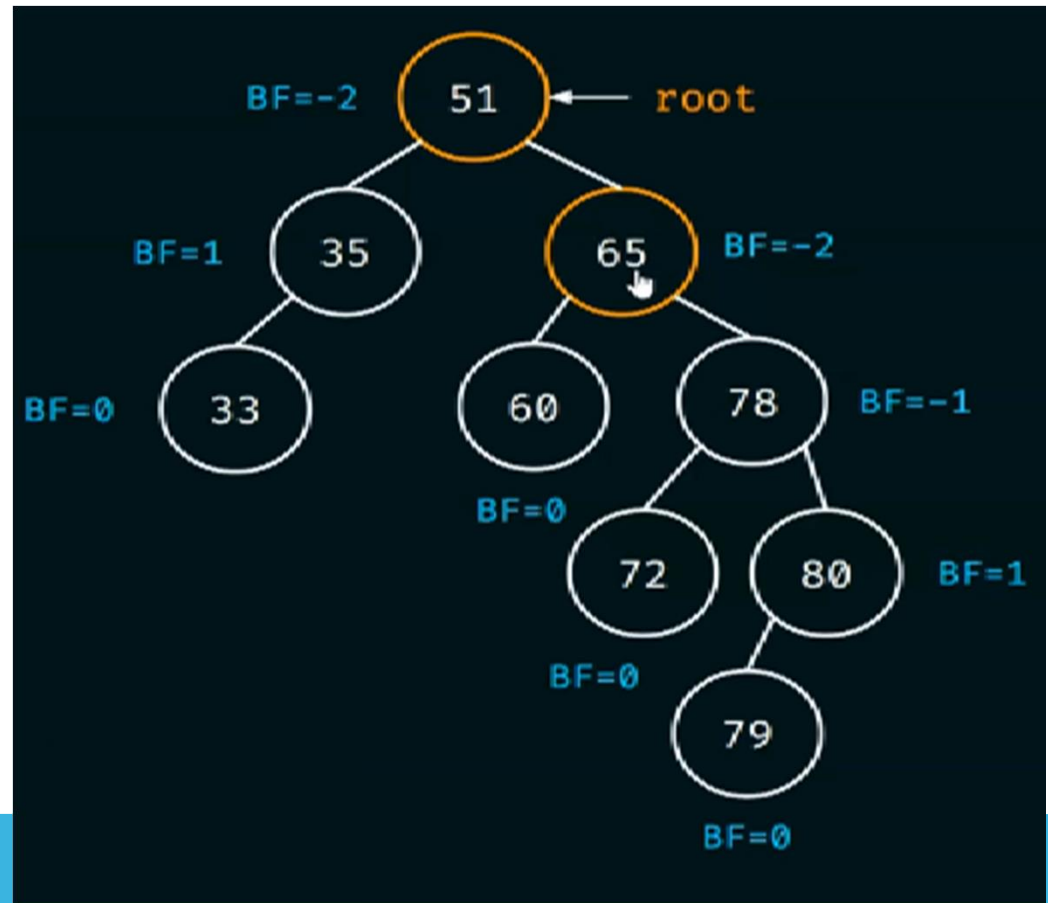
Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

280

PROGRAM

Let say left rotation is to be performed about node 65



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

281

PROGRAM

root->right = leftRotate(root->right);

Step 1: If node B has a left subtree, then make β the right subtree of A.

Step 2: If P is NULL, then make B as the root of the tree.

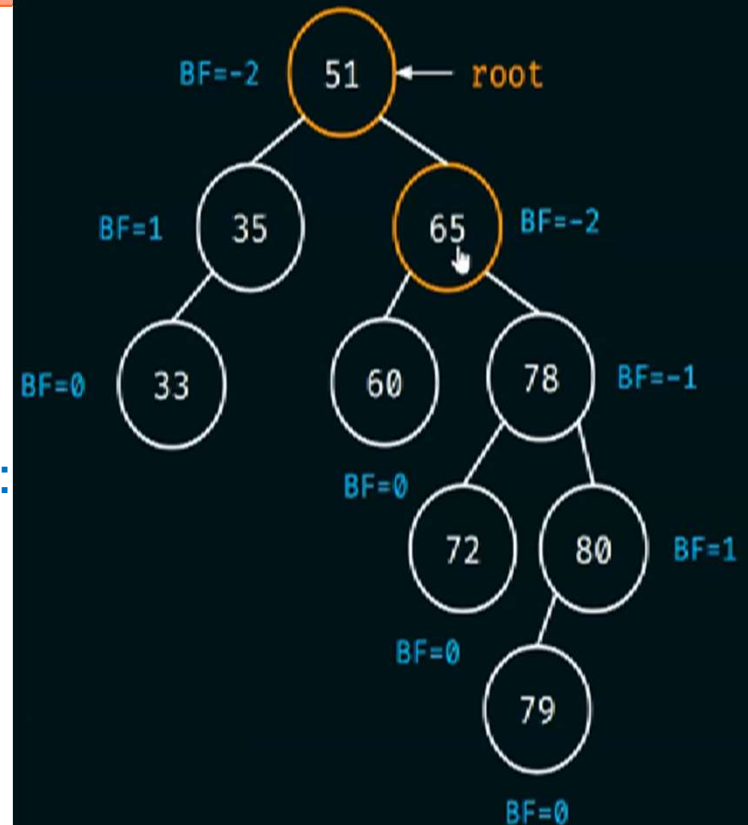
Step 3: If P is not NULL, then check the following:

a) If A is the left child of P, then make B as the left child of P.

b) Else make B as the right child of P.

Step 4: Make B as the parent of A.

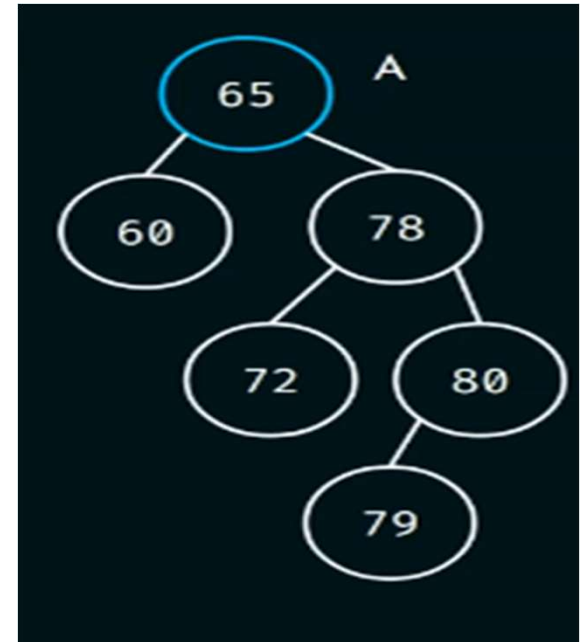
We have to consider two nodes 65 and 78.



These two steps will be handled in the main function.

PROGRAM

```
struct node* leftRotate(struct node* A)
{
}
```



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

283

PROGRAM

```
struct node* leftRotate(struct node* A)
{
    struct node* B = A->right;
}
```



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

284

ALGORITHM FOR LEFT ROTATION

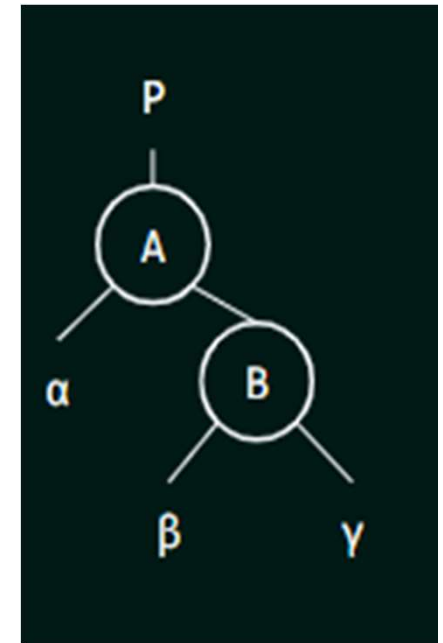
Step 1: If node B has a left subtree, then make β the right subtree of A.

Step 2: If P is NULL, then make B as the root of the tree.

Step 3: If P is not NULL, then check the following:

- a) If A is the left child of P, then make B as the left child of P.
- b) Else make B as the right child of P.

Step 4: Make B as the parent of A.



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

285

ALGORITHM FOR LEFT ROTATION

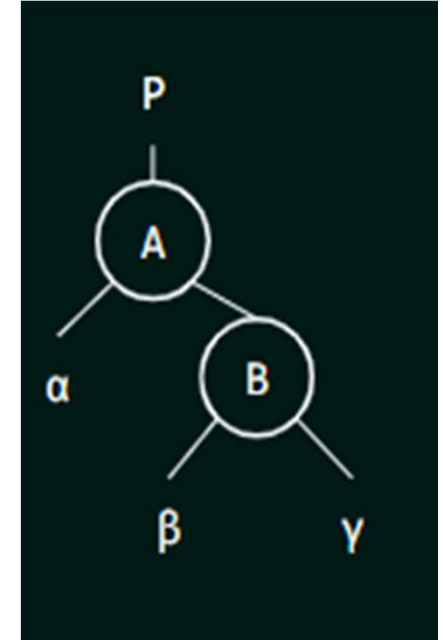
Step 1: If node B has a left subtree, then make β the right subtree of A.

Step 2: If P is NULL, then make B as the root of the tree.

Step 3: If P is not NULL, then check the following:

- a) If A is the left child of P, then make B as the left child of P.
- b) Else make B as the right child of P.

Step 4: Make B as the parent of A.



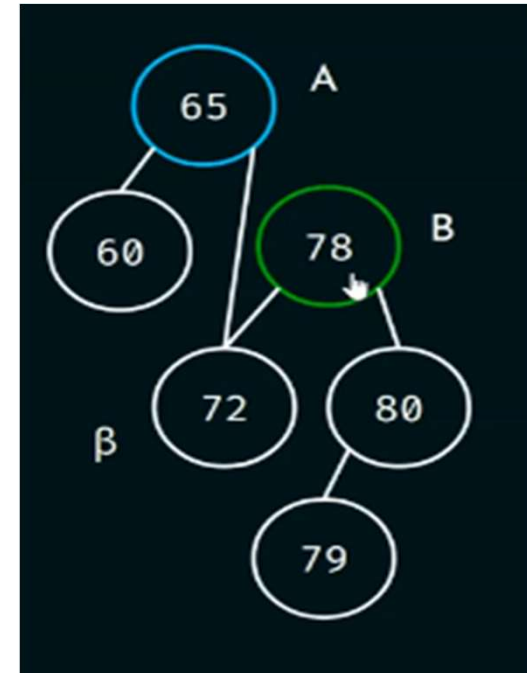
Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

286

PROGRAM

```
struct node* leftRotate(struct node* A)
{
    struct node* B = A->right;
    A->right = B->left;
}
```



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

287

ALGORITHM FOR LEFT ROTATION

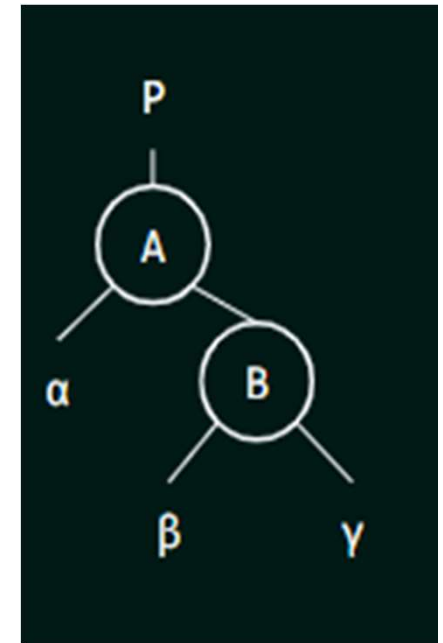
Step 1: If node B has a left subtree, then make β the right subtree of A.

Step 2: If P is NULL, then make B as the root of the tree.

Step 3: If P is not NULL, then check the following:

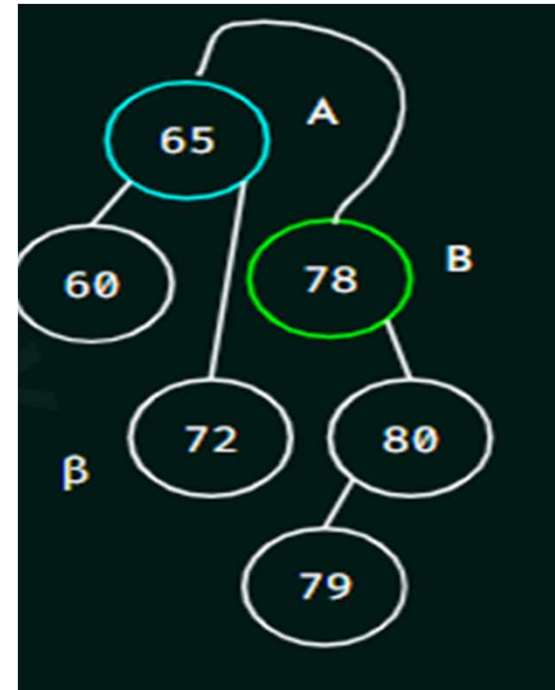
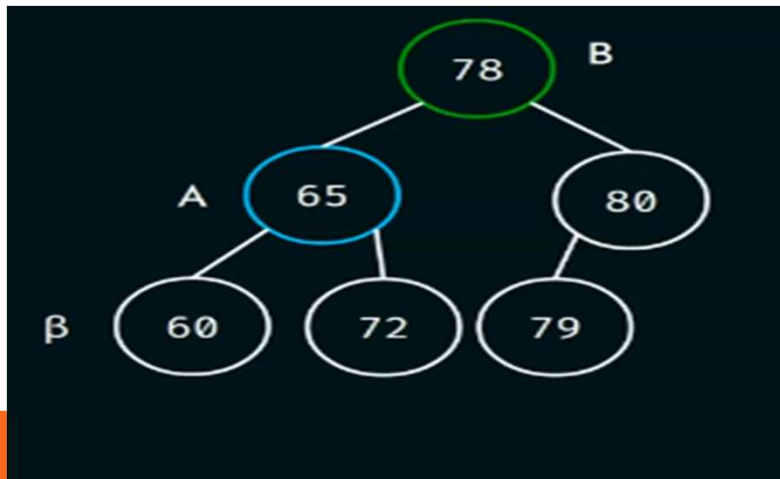
- a) If A is the left child of P, then make B as the left child of P.
- b) Else make B as the right child of P.

Step 4: Make B as the parent of A.



PROGRAM

```
struct node* leftRotate(struct node* A)
{
    struct node* B = A->right;
    A->right = B->left;
    B->left = A;
}
```

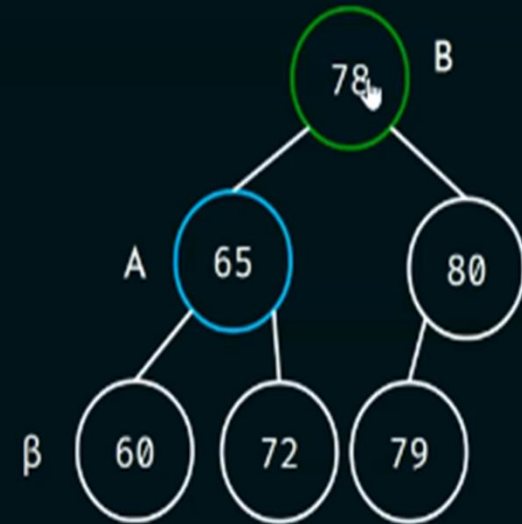


Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

PROGRAM

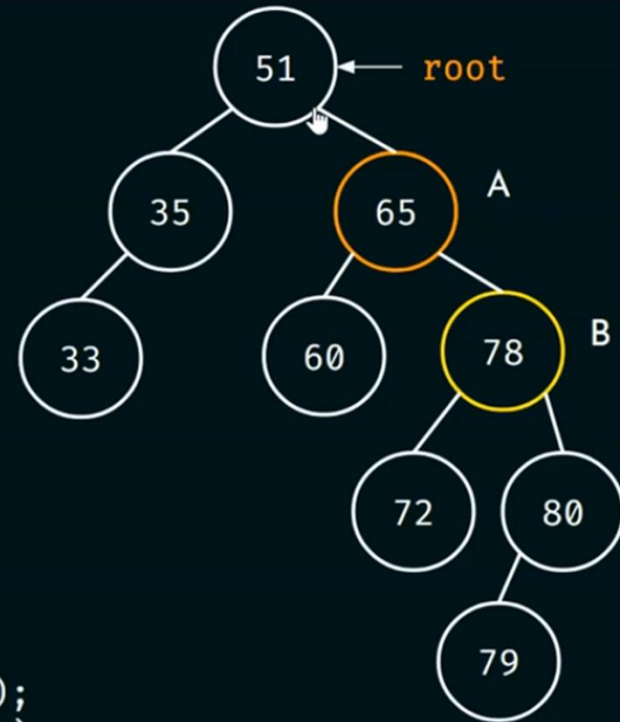
```
struct node* leftRotate(struct node* A)
{
    struct node* B = A→right;
    A→right = B→left;
    B→left = A;
    return B;
}
```



PROGRAM

```
struct node {
    struct node* left;
    int data;
    struct node* right;
};

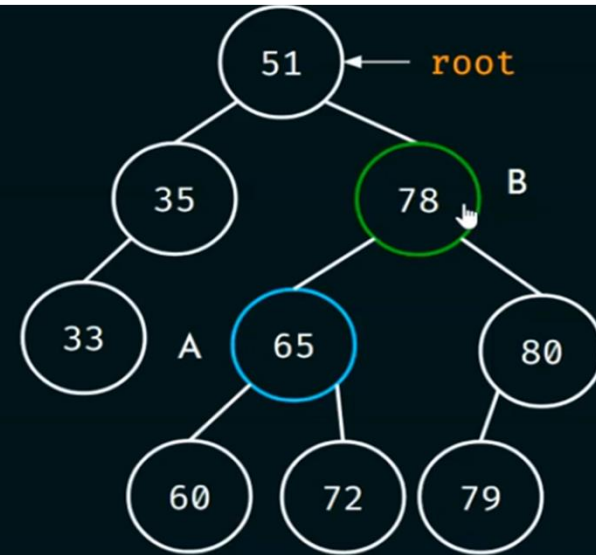
int main()
{
    struct node* root = NULL;
    root = createNode(51);
    root->left = createNode(35);
    root->left->left = createNode(33);
    root->right = createNode(65);
    root->right->left = createNode(60);
    root->right->right = createNode(78);
    root->right->right->left = createNode(72);
    root->right->right->right = createNode(80);
    root->right->right->right->left = createNode(79);
    root->right = leftRotate(root->right);
}
```



PROGRAM

```
struct node {
    struct node* left;
    int data;
    struct node* right;
};

int main()
{
    struct node* root = NULL;
    root = createNode(51);
    root->left = createNode(35);
    root->left->left = createNode(33);
    root->right = createNode(65);
    root->right->left = createNode(60);
    root->right->right = createNode(78);
    root->right->right->left = createNode(72);
    root->right->right->right = createNode(80);
    root->right->right->right->left = createNode(79);
    root->right = leftRotate(root->right);
}
```



BALANCE FACTOR OF TREE



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

PROGRAM

[View Source Code](#)

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

294

RIGHT ROTATION IN AVL TREES (ALGORITHM)

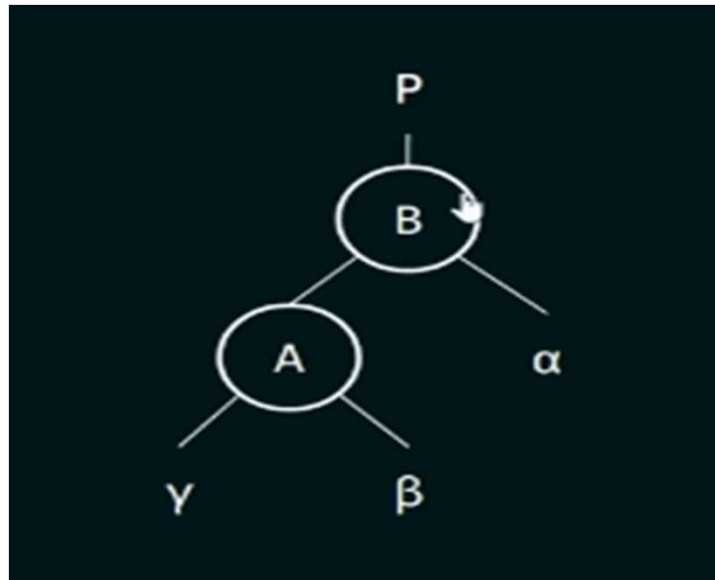
Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

295

ALGORITHM

Consider the following tree:



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

296

ALGORITHM FOR RIGHT ROTATION

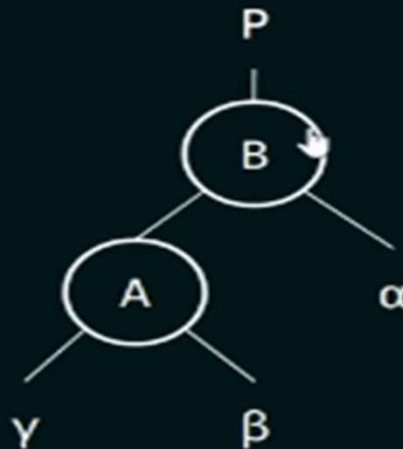
Step 1: If node A has a right subtree β , then make β the left subtree of B.

Step 2: If P is NULL, then make A as the root of the tree.

Step 3: If P is not NULL, then check the following:

- a) If B is the left child of P, then make A as the left child of P.
- b) Else make A as the right child of P.

Step 4: Make A as the parent of B



PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

ALGORITHM FOR RIGHT ROTATION

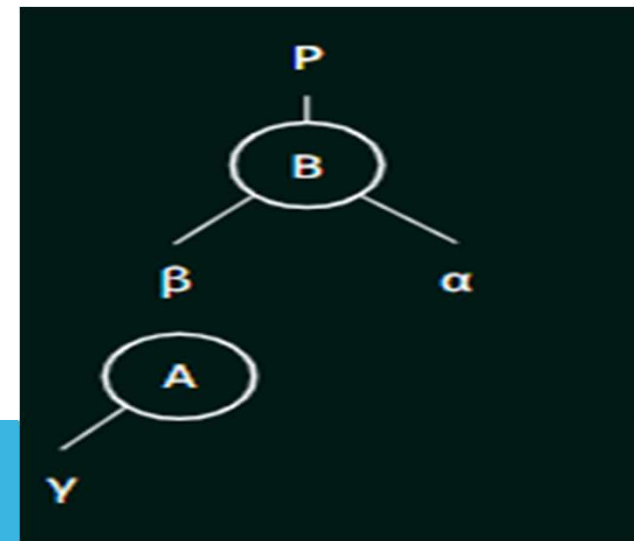
Step 1: If node A has a right subtree β , then make β the left subtree of B.

Step 2: If P is NULL, then make A as the root of the tree.

Step 3: If P is not NULL, then check the following:

- a) If B is the left child of P, then make A as the left child of P.
- b) Else make A as the right child of P.

Step 4: Make A as the parent of B



PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

ALGORITHM FOR RIGHT ROTATION

Step 1: If node A has a right subtree β , then make β the left subtree of B.

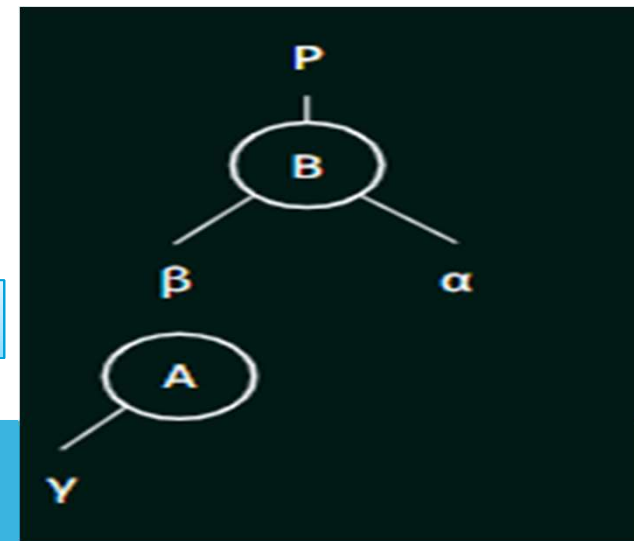
Step 2: If P is NULL, then make A as the root of the tree.

Step 3: If P is not NULL, then check the following:

- a) If B is the left child of P, then make A as the left child of P.
- b) Else make A as the right child of P.

Step 4: Make A as the parent of B

Let say P is not NULL and B is the left child of P



PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

ALGORITHM FOR RIGHT ROTATION

Step 1: If node A has a right subtree β , then make β the left subtree of B.

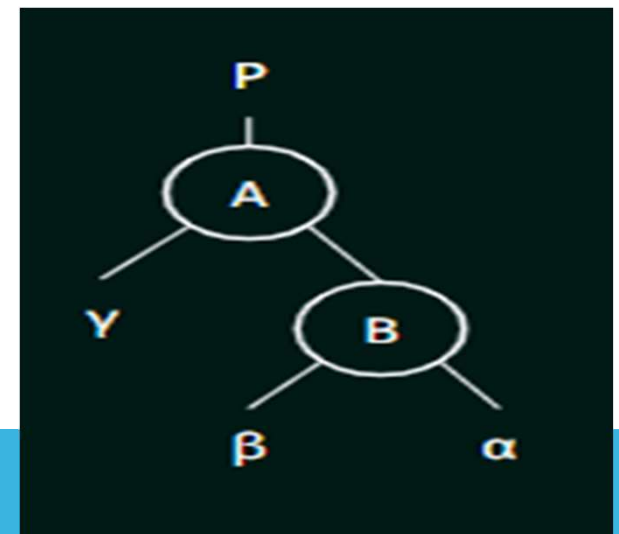
Step 2: If P is NULL, then make A as the root of the tree.

Step 3: If P is not NULL, then check the following:

a) If B is the left child of P, then make A as the left child of P.

b) Else make A as the right child of P.

Step 4: Make A as the parent of B

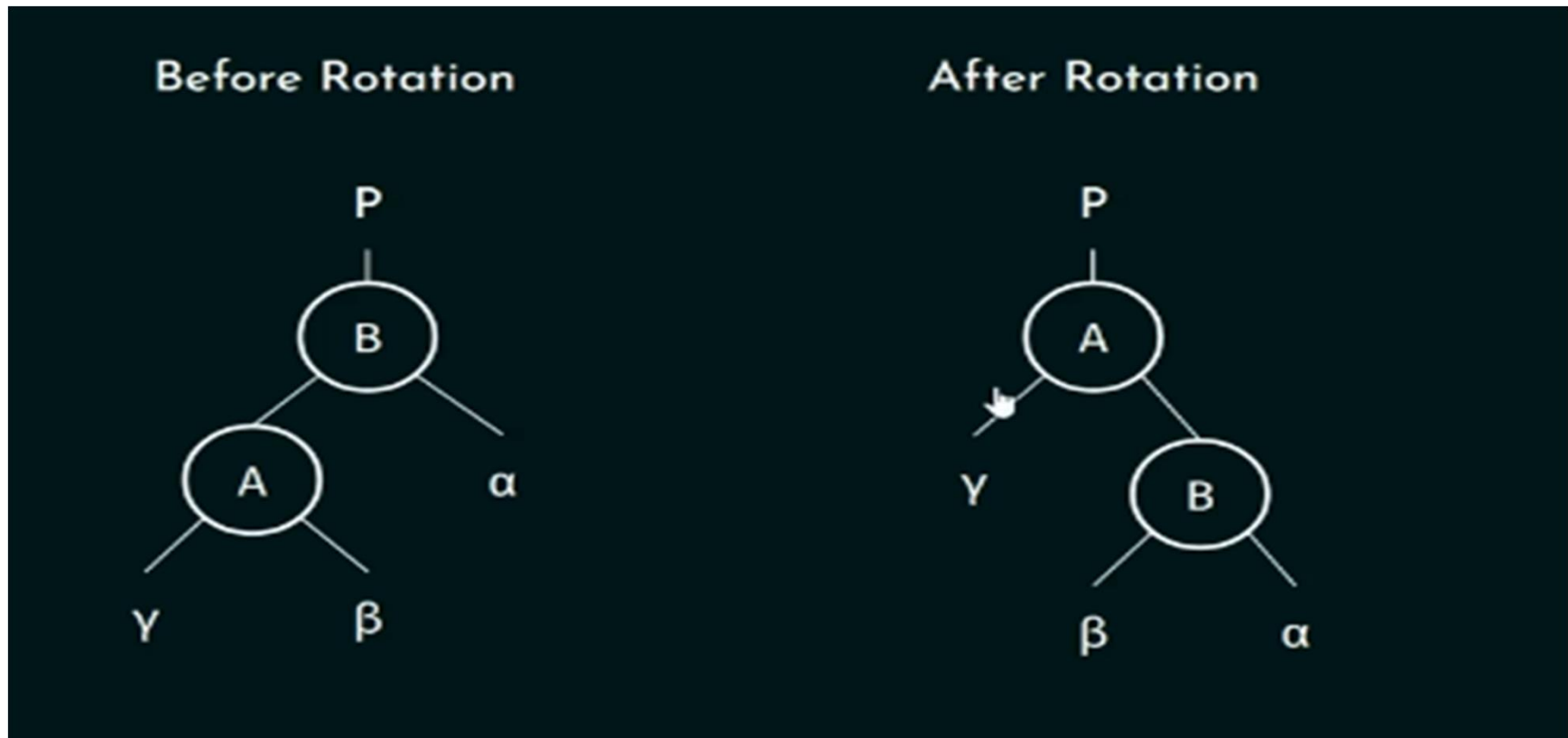


Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

300

ALGORITHM FOR RIGHT ROTATION



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

301

RIGHT ROTATION IN AVL TREES (PROGRAM)

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

302

ALGORITHM FOR RIGHT ROTATION

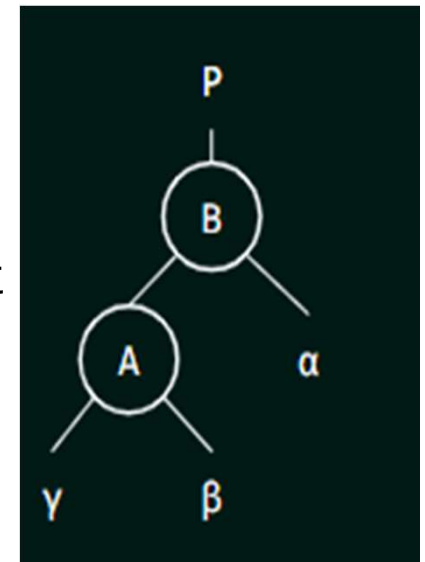
Step 1: If node A has a right subtree β , then make β the left subtree of B.

Step 2: If P is NULL, then make A as the root of the tree.

Step 3: If P is not NULL, then check the following:

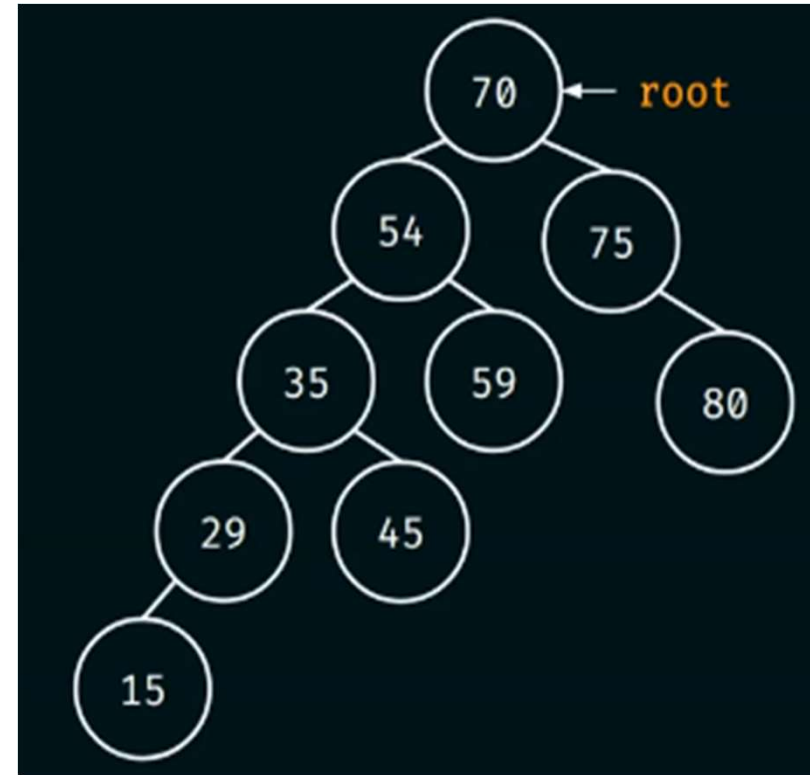
- a) If B is the left child of P, then make A as the left child of P.
- b) Else make A as the right child of P.

Step 4: Make A as the parent of B



PROGRAM

```
struct node {  
    struct node* left;  
    int data;  
    struct node* right;  
};  
int main()  
{  
    struct node* root = NULL;  
    root = createNode(70);  
    root->left = createNode(54);  
    root->left->left = createNode(35);  
    root->left->right = createNode(59);  
    root->left->left->left = createNode(29);  
    root->left->left->right = createNode(45);  
    root->right = createNode(75);  
    root->right->right = createNode(80);  
    root->left->left->left->left = createNode(15);  
}
```



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

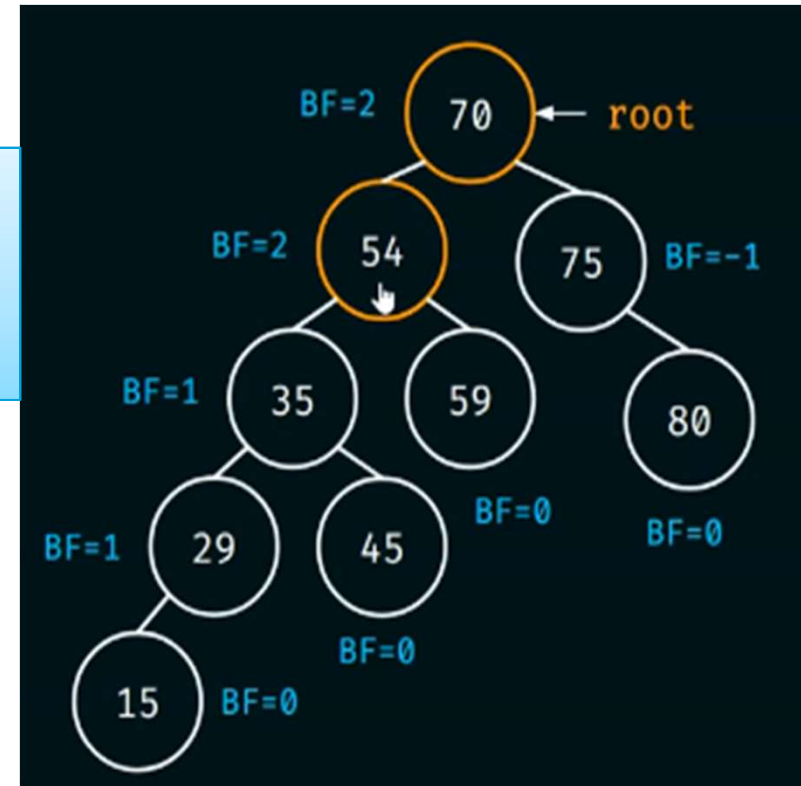
304

PROGRAM

```
struct node {  
    struct node* left;  
    int data;  
    struct node* right;  
};  
int main()  
{  
    struct node* root = NULL;  
    root = createNode(70);  
    root->left = createNode(54);  
    root->left->left = createNode(35);  
    root->left->right = createNode(59);  
    root->left->left->left = createNode(29);  
    root->left->left->right = createNode(45);  
    root->right = createNode(75);  
    root->right->right = createNode(80);  
    root->left->left->left->left = createNode(15);  
}
```

Let say the newly inserted node is 15

We have to consider two nodes 54 and 35.



Tuesday, August 18, 2026

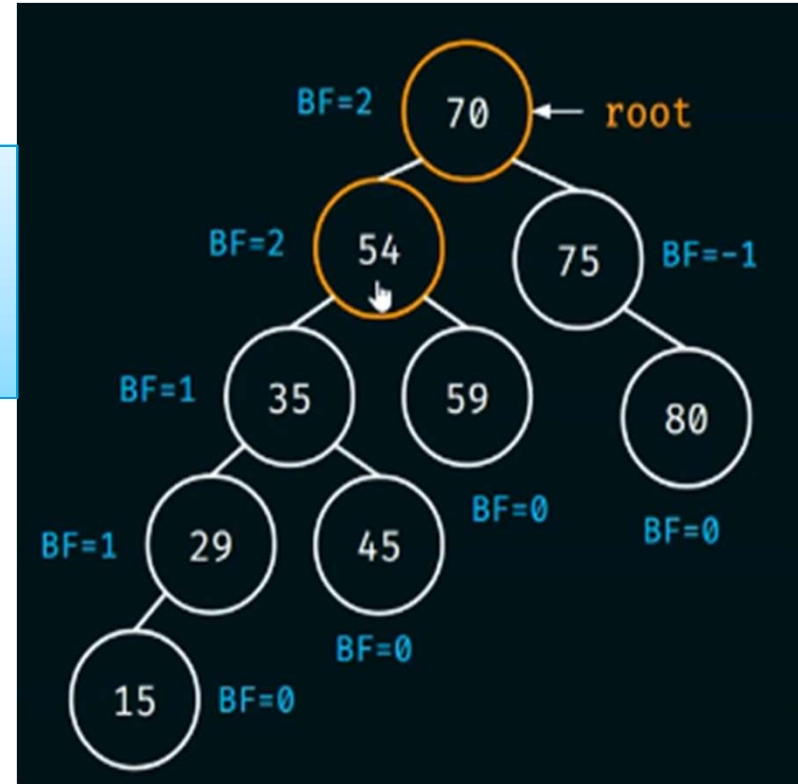
PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

305

PROGRAM

```
struct node {  
    struct node* left;  
    int data;  
    struct node* right;  
};  
int main()  
{  
    struct node* root = NULL;  
    root = createNode(70);  
    root->left = createNode(54);  
    root->left->left = createNode(35);  
    root->left->right = createNode(59);  
    root->left->left->left = createNode(29);  
    root->left->left->right = createNode(45);  
    root->right = createNode(75);  
    root->right->right = createNode(80);  
    root->left->left->left->left = createNode(15);  
  
    root->left = rightRotate(root->left);
```

We have to consider two nodes 54 and 35.



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

306

PROGRAM

Algorithm for right rotation:

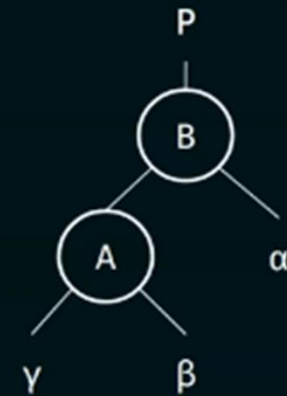
Step 1: If node A has a right subtree β , then make β the left subtree of B. ➡

Step 2: If P is NULL, then make A as the root of the tree.

Step 3: If P is not NULL, then check the following:

- a) If B is the left child of P, then make A as the left child of P.
- b) Else make A as the right child of P.

Step 4: Make A as the parent of B.



These two steps will be handled in the main function.

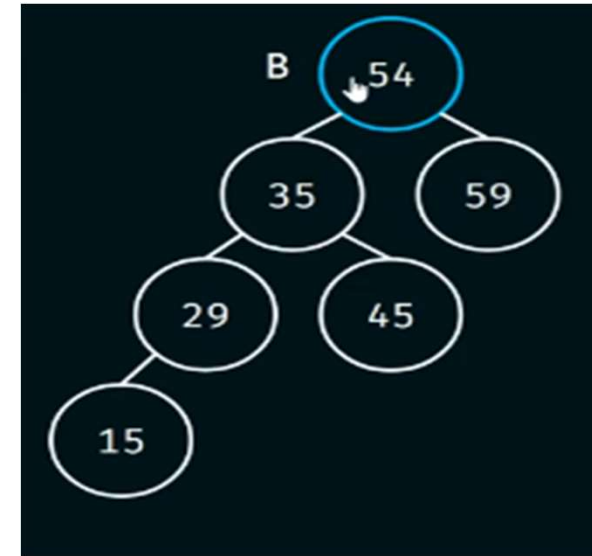
Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

307

PROGRAM

```
struct node* rightRotate(struct node* B)
{
}
```



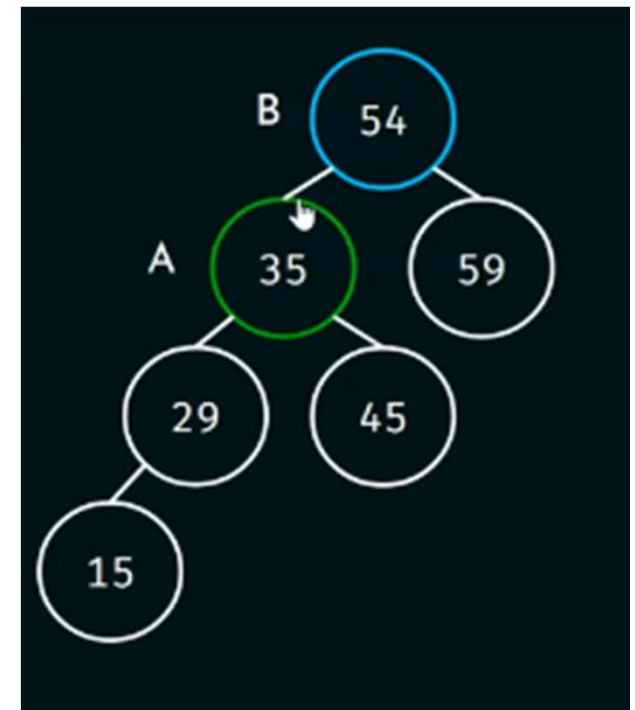
Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

308

PROGRAM

```
struct node* rightRotate(struct node* B)
{
    struct node* A = B->left;
}
```



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

309

PROGRAM

Algorithm for right rotation:

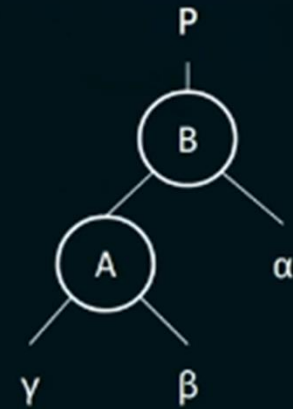
Step 1: If node A has a right subtree β , then make β the left subtree of B.

Step 2: If P is NULL, then make A as the root of the tree.

Step 3: If P is not NULL, then check the following:

- a) If B is the left child of P, then make A as the left child of P.
- b) Else make A as the right child of P.

Step 4: Make A as the parent of B.



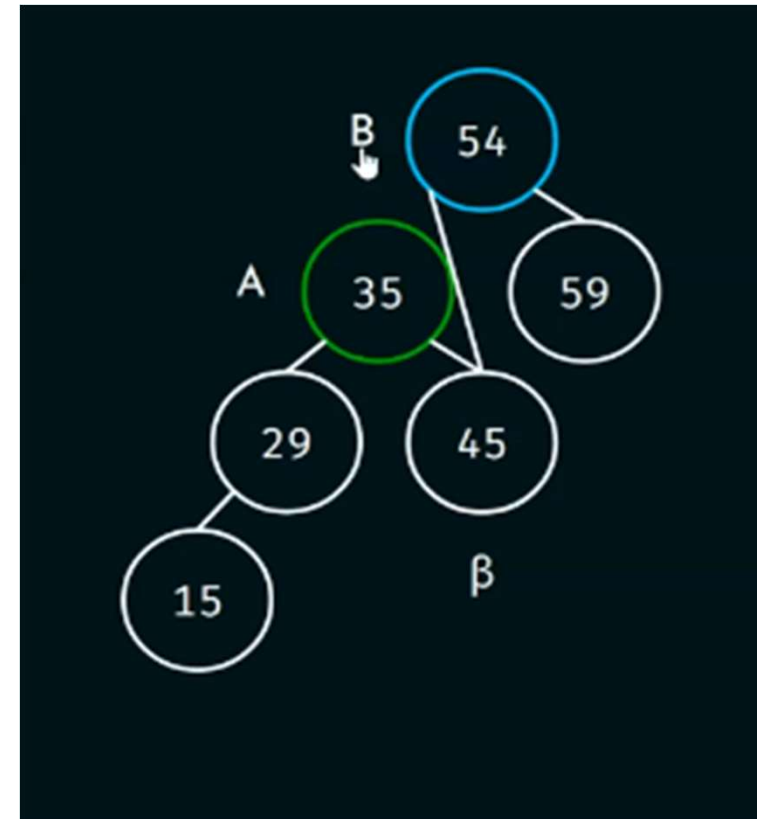
These two steps will be handled in the main function.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

PROGRAM

```
struct node* rightRotate(struct node* B)
{
    struct node* A = B->left;
    B->left = A->right;
}
```



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

311

PROGRAM

Algorithm for right rotation:

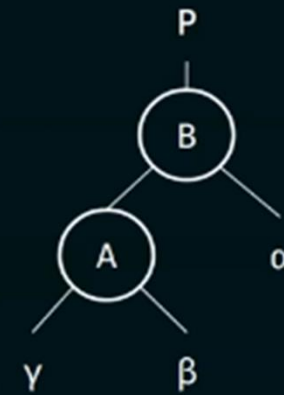
Step 1: If node A has a right subtree β , then make β the left subtree of B.

Step 2: If P is NULL, then make A as the root of the tree.

Step 3: If P is not NULL, then check the following:

- a) If B is the left child of P, then make A as the left child of P.
- b) Else make A as the right child of P.

Step 4: Make A as the parent of B.



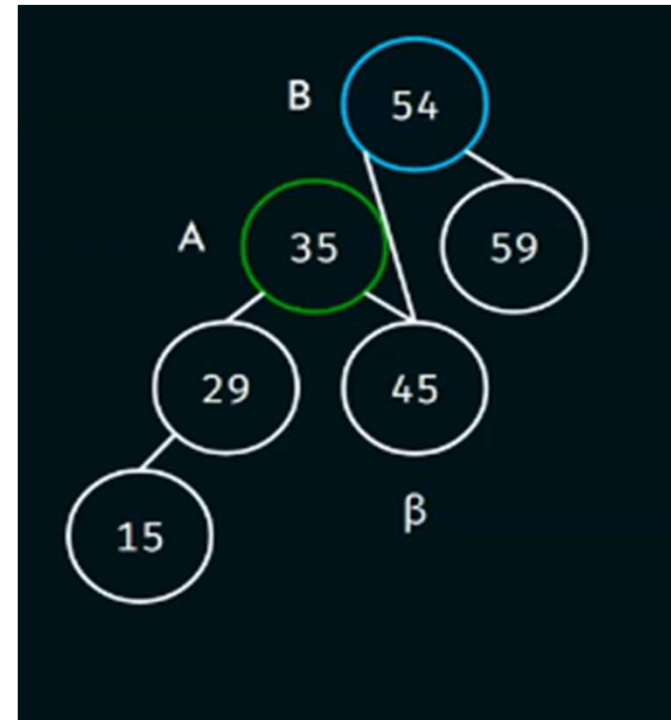
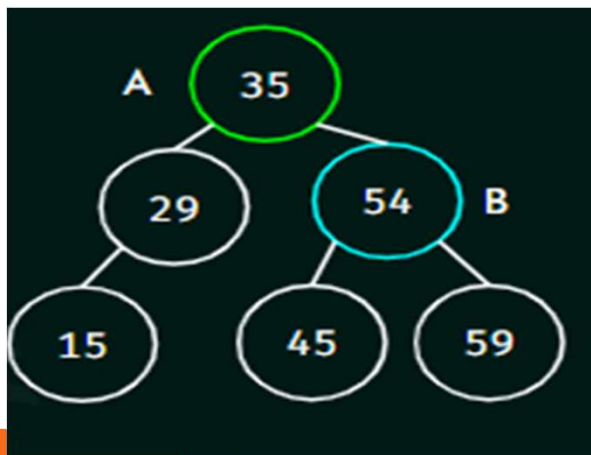
These two steps will be handled in the main function.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

PROGRAM

```
struct node* rightRotate(struct node* B)
{
    struct node* A = B->left;
    B->left = A->right;
    A->right = B;
}
```



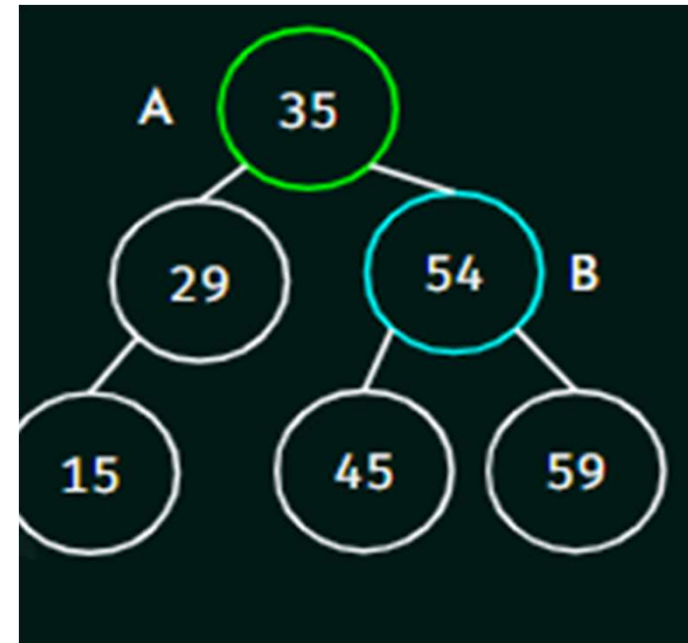
Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

313

PROGRAM

```
struct node* rightRotate(struct node* B)
{
    struct node* A = B->left;
    B->left = A->right;
    A->right = B;
    return A;
}
```

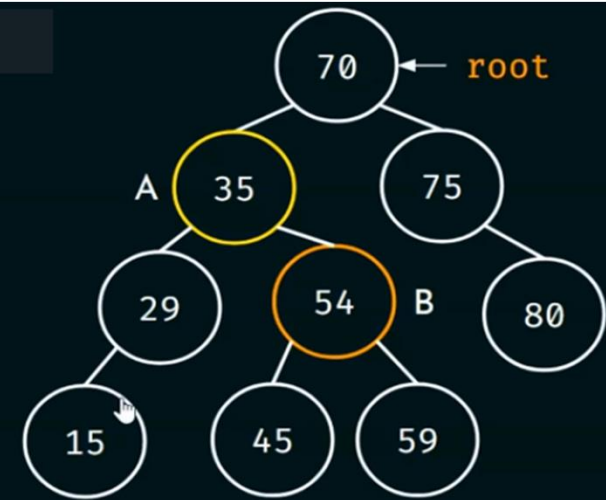


PROGRAM

```
struct node {
    struct node* left;
    int data;
    struct node* right;
};

int main()
{
    struct node* root = NULL;
    root = createNode(70);
    root->left = createNode(54);
    root->left->left = createNode(35);
    root->left->right = createNode(59);
    root->left->left->left = createNode(29);
    root->left->left->right = createNode(45);
    root->right = createNode(75);
    root->right->right = createNode(80);
    root->left->left->left->left = createNode(15);
    root->left = rightRotate(root->left);
}
```

player.vdocipher.com – To exit full screen, press Esc



Activate Windows
Go to Settings to activate Windows.

Tuesday, August 18, 2026

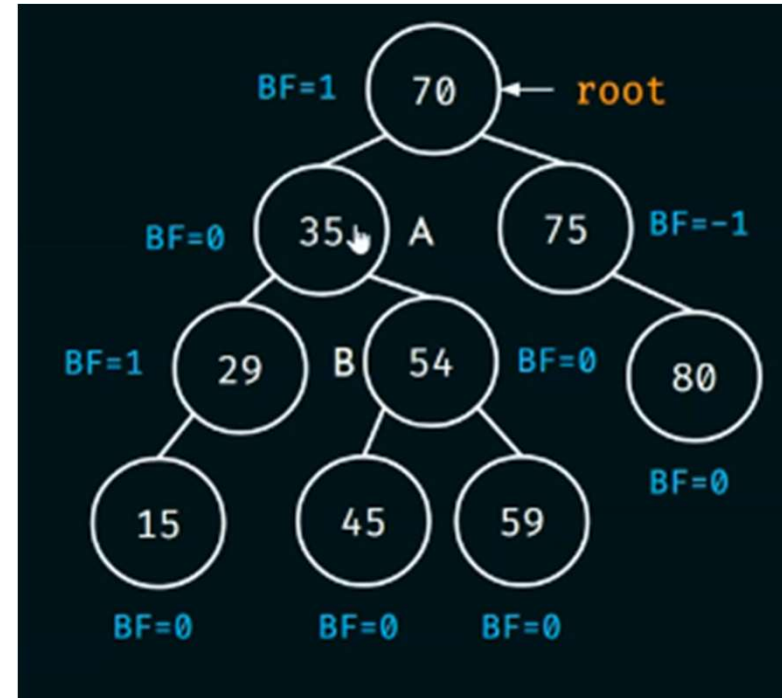
PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

315

PROGRAM

```
struct node {
    struct node* left;
    int data;
    struct node* right;
};

int main()
{
    struct node* root = NULL;
    root = createNode(70);
    root->left = createNode(54);
    root->left->left = createNode(35);
    root->left->right = createNode(59);
    root->left->left->left = createNode(29);
    root->left->left->right = createNode(45);
    root->right = createNode(75);
    root->right->right = createNode(80);
    root->left->left->left->left = createNode(15);
    root->left = rightRotate(root->left);
}
```



Tree is perfectly balanced.

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

PROGRAM

[View Source Code](#)

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

317

LEFT-RIGHT AND RIGHT-LEFT ROTATIONS (PROGRAM)

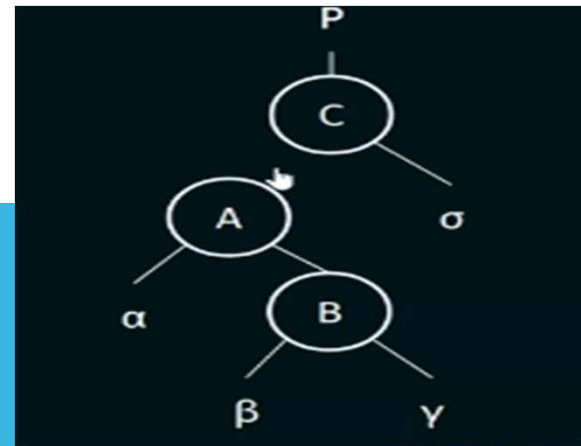
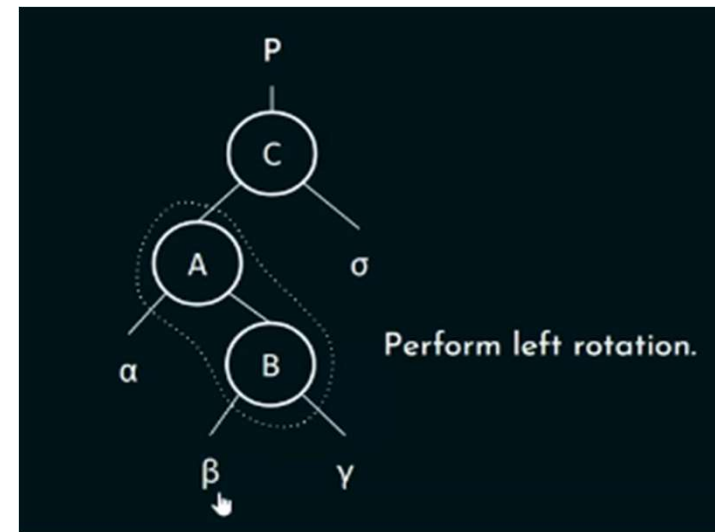
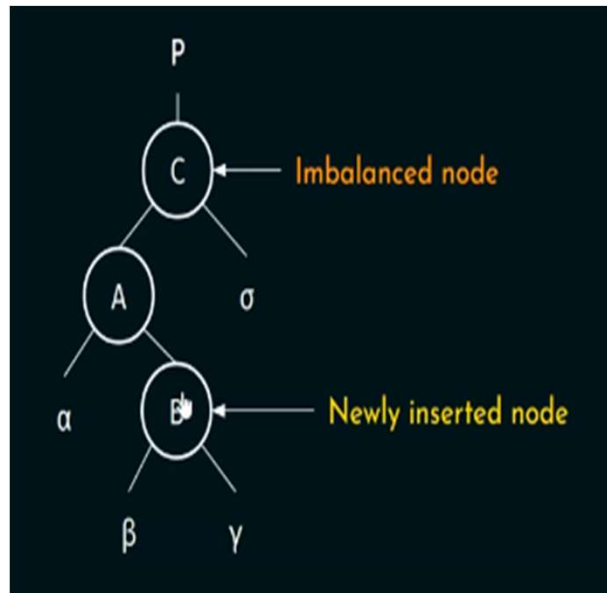
Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

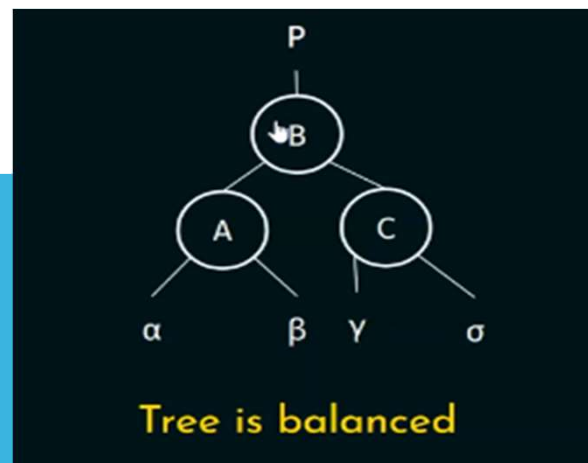
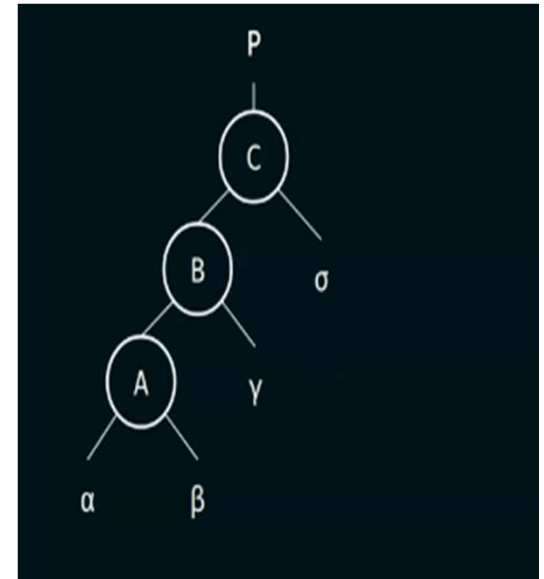
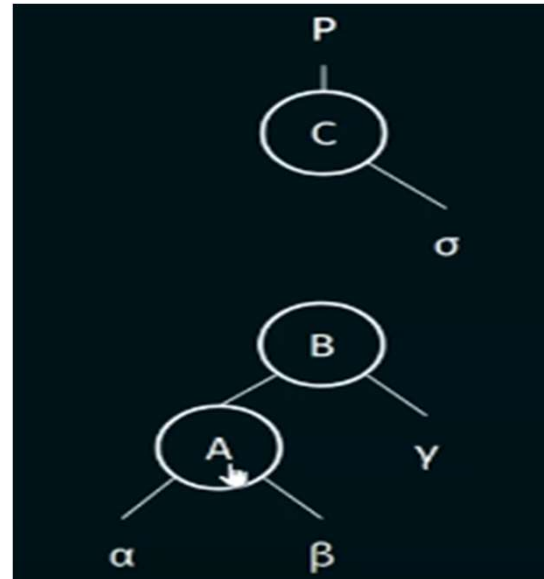
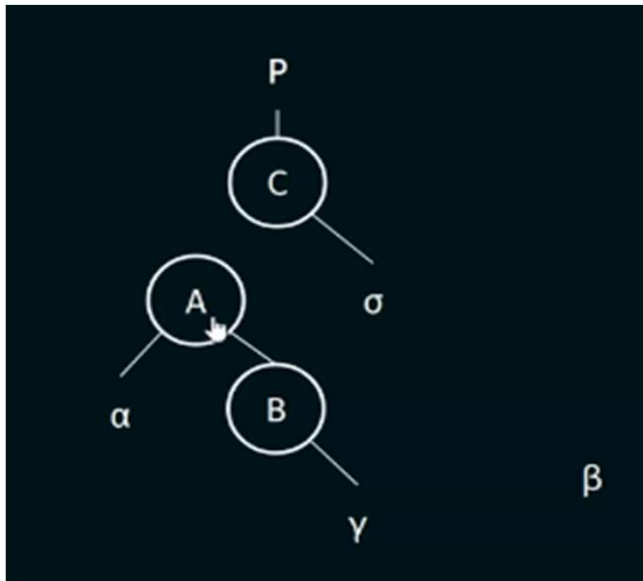
318

LEFT-RIGHT ROTATION

Consider the following tree:



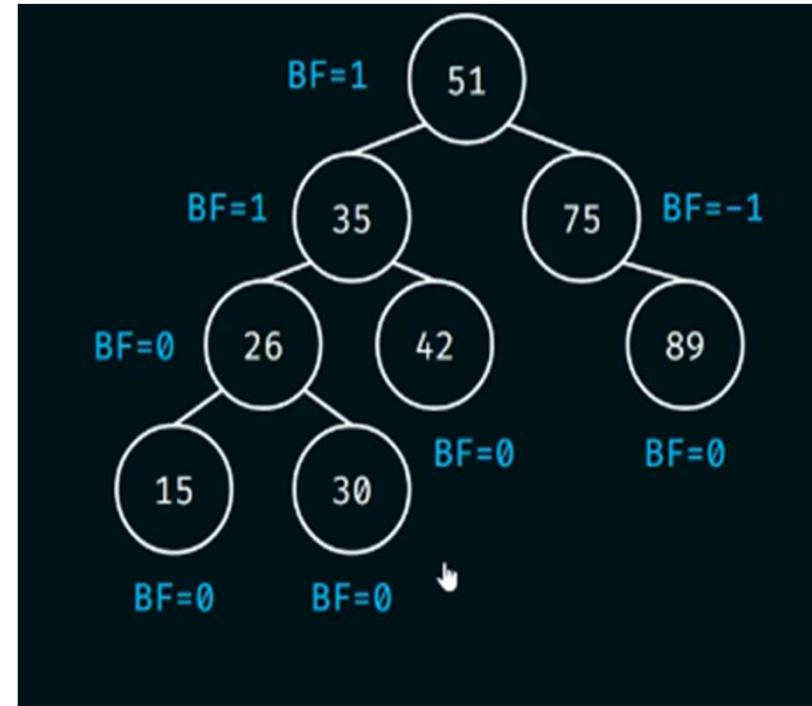
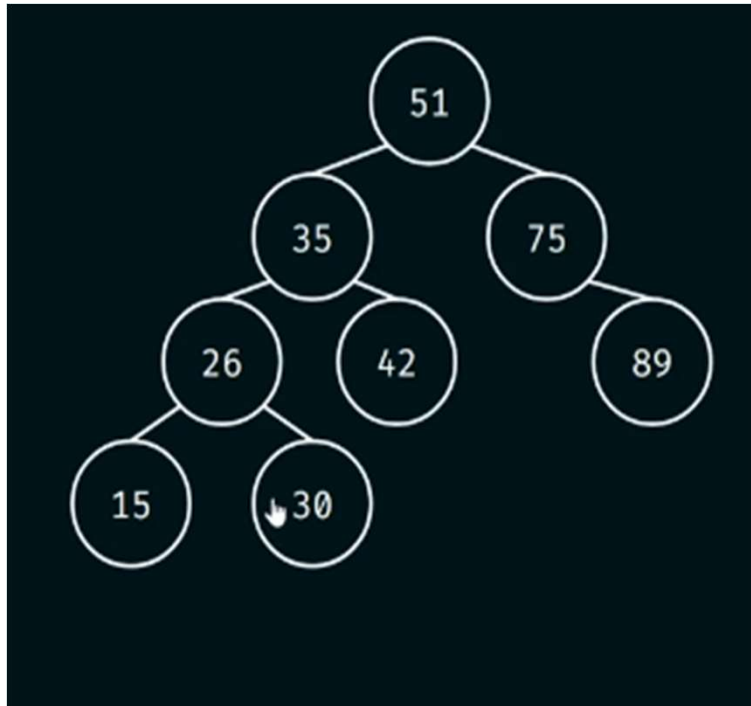
LEFT-RIGHT ROTATION



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

EXAMPLE

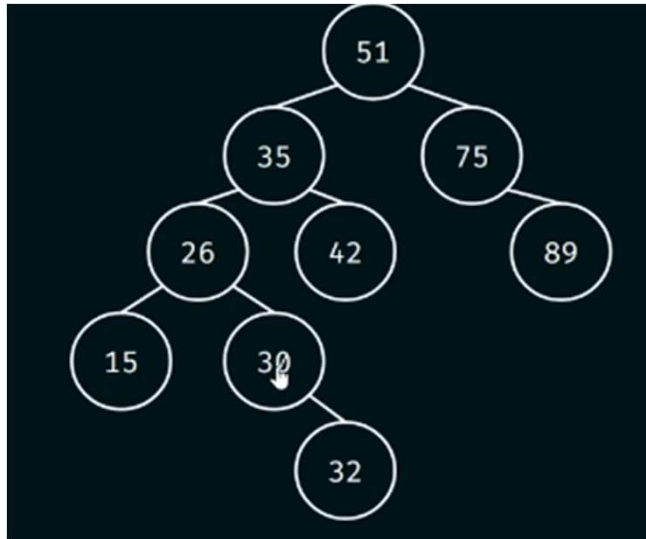


Tuesday, August 18, 2026

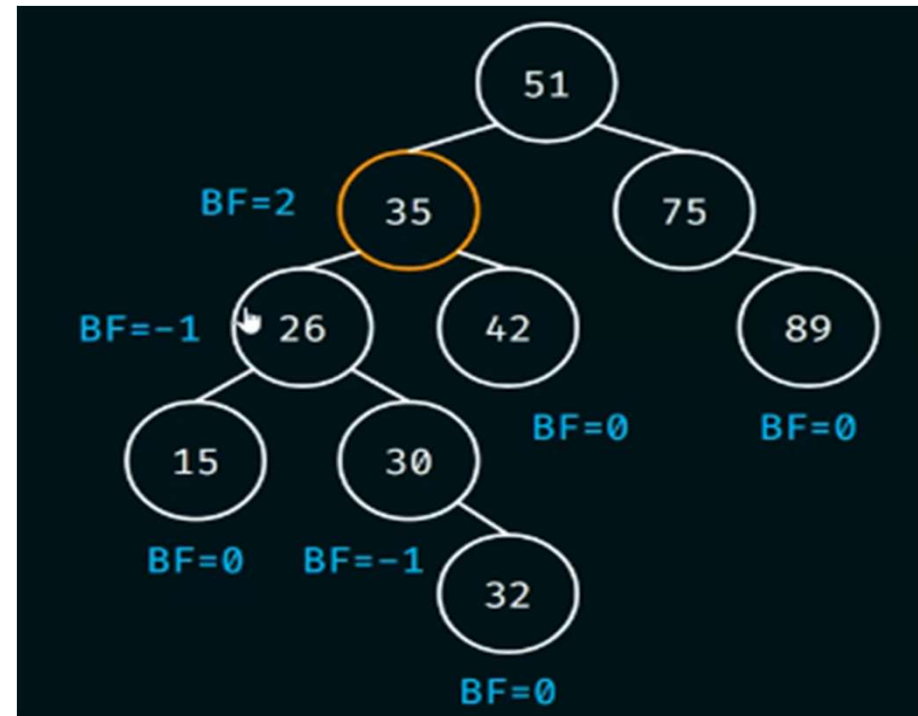
PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

321

EXAMPLE



Insert 32

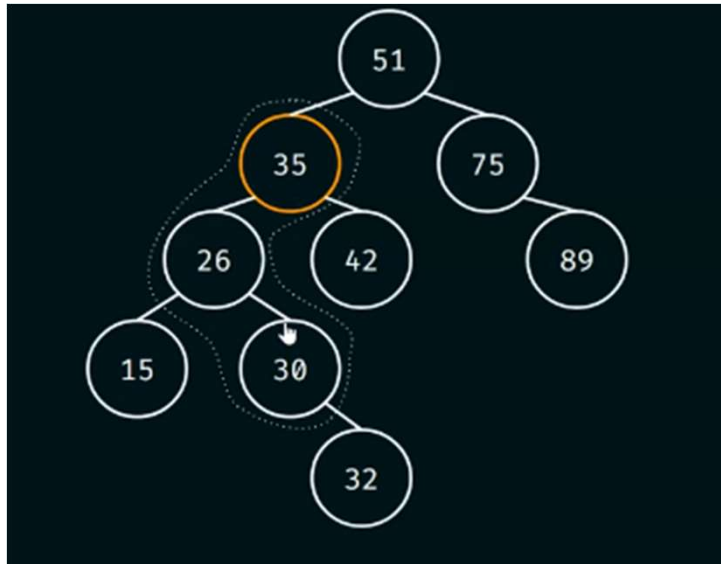


Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

322

EXAMPLE

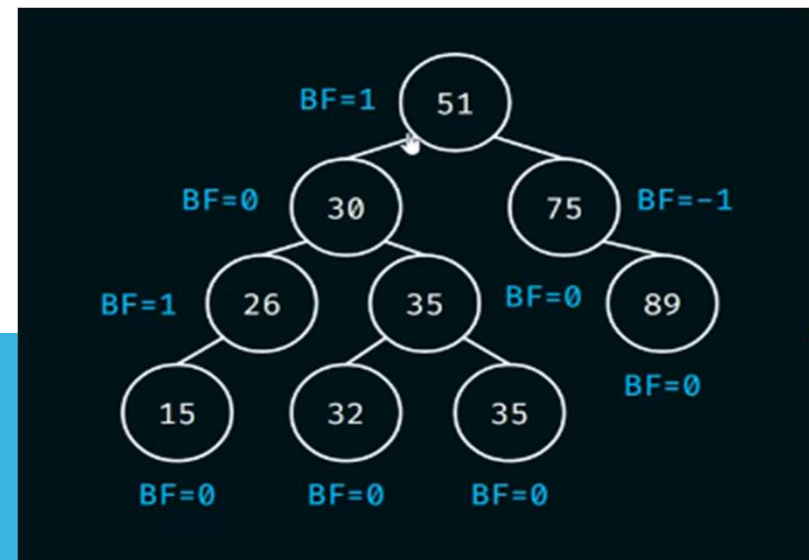
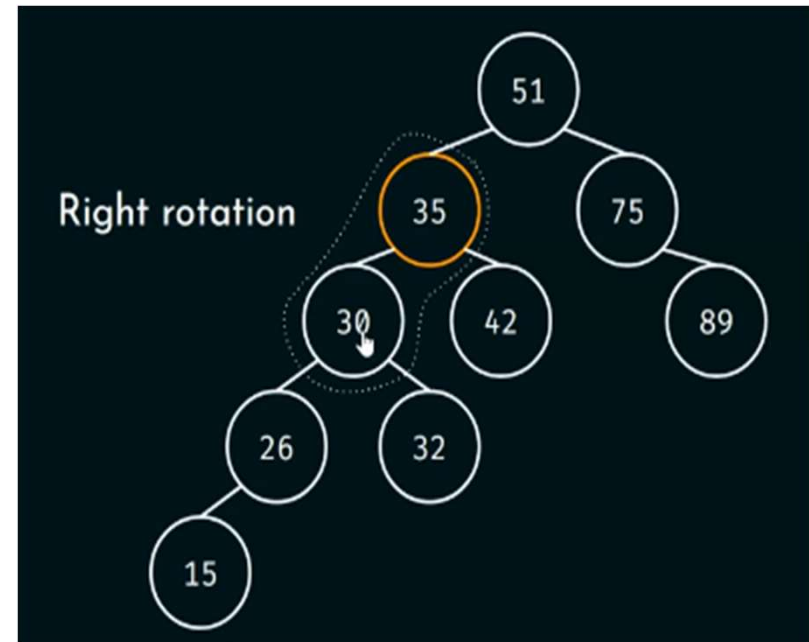
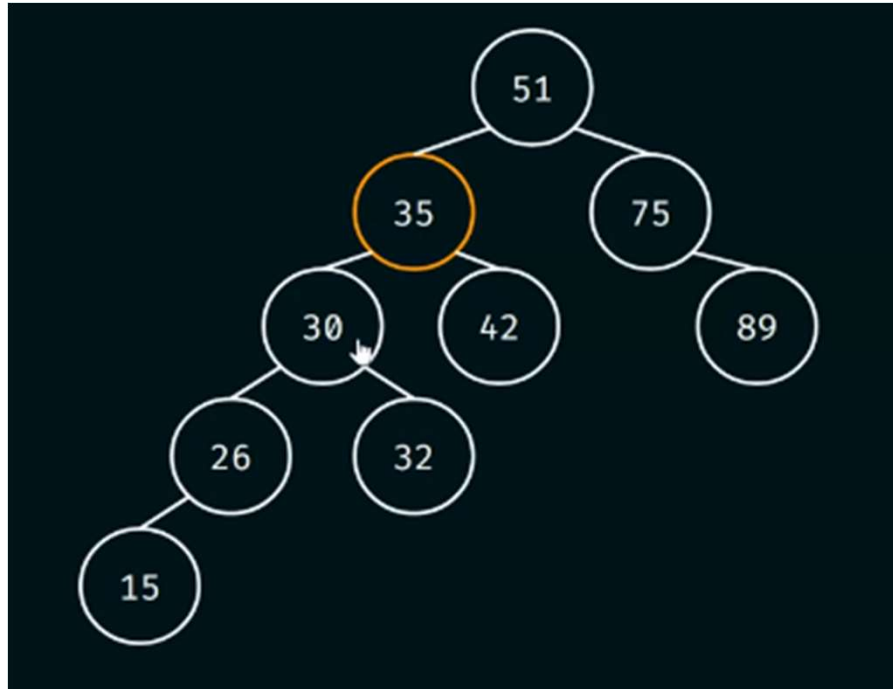


Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

323

EXAMPLE



Tuesday, August 18, 2026

ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

324

PROGRAM

[View Source Code](#)

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

325

INSERTION IN AVL TREE

Steps involved in insertion:

Step 1: Insert a new node in its correct position.

Step 2: Calculate the balance factor of each node and find the imbalanced node.

Step 3: Check which rotation needs to be performed about the imbalanced node.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

326

INSERTION IN AVL TREE

The process to check which rotation needs to be performed is as follows:

1. If $\text{balanceFactor} > 1$, then do the following
 - a. If $\text{newNodeKey} > \text{leftChildKey}$, then perform left-right rotation.
 - b. Else perform the right rotation.
2. If $\text{balanceFactor} < -1$, then do the following
 - a. If $\text{newNodeKey} < \text{rightChildKey}$, then perform the right-left rotation.
 - b. Else perform the left rotation.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

327

PRACTICE PROBLEMS

1. Construct an AVL tree by inserting the following elements in order of their occurrence 64, 1, 44, 26, 13, 110, 98, 85
2. Construct an AVL tree by inserting the following elements in order of their occurrence 1 – 8
3. Construct an AVL Tree by inserting 50,40,60,30,45,55,70

Tasks Draw tree after every insertion

Calculate Balance Factor

Mention whether rotation is required

4. Insert 15,10,20,8,12,17,25,6 Find Heights, Balance Factors and Final AVL Tree

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

328

PRACTICE PROBLEMS

5. Insert 25,20,30,10,22,28,35,5 and Perform AVL insertion.
6. Insert the following keys into an AVL Tree: 40,20,10,25,30,22,50,60
 - a) Draw the tree after each insertion. (4 Marks)
 - b) Calculate the balance factor of every node. (2 Marks)
 - c) Identify each rotation performed (LL, RR, LR, RL). (2 Marks)
 - d) Draw the final AVL Tree. (2 Marks)
7. Insert the following keys: 50,30,70,20,40,60,80,10,25,35,45,55,65,75,85 For each insertion:
 - a. Draw the AVL Tree.
 - b. Calculate the balance factor of every affected node.
 - c. State whether a rotation is needed.
 - d. If required, identify the rotation (LL, RR, LR, or RL).
 - e. Draw the final balanced AVL Tree.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

PRACTICE PROBLEMS

8. Insert the keys 14, 8, 12, 36, 23, 5, 67, 78, 20 into an empty AVL tree. Then delete 36 and 67 sequentially. Draw the tree after each deletion and indicate the rotations applied.
9. Construct an AVL tree using the elements: 50, 30, 70, 20, 40, 60, 80, 10. Delete the node 70 and show all steps to rebalance the tree if needed.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

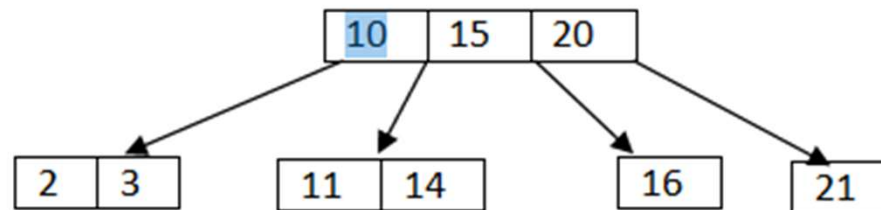
330

B-TREE

Definition

A B-Tree (Balanced Tree) is a self-balancing multi-way search tree used to store large amounts of sorted data efficiently. It is widely used in databases, file systems, and indexing because it minimizes disk accesses.

Unlike a Binary Search Tree (BST), where each node has at most 2 children, a B-Tree node can have many children.



PROPERTIES OF A B-TREE

Consider a B-Tree of order m .

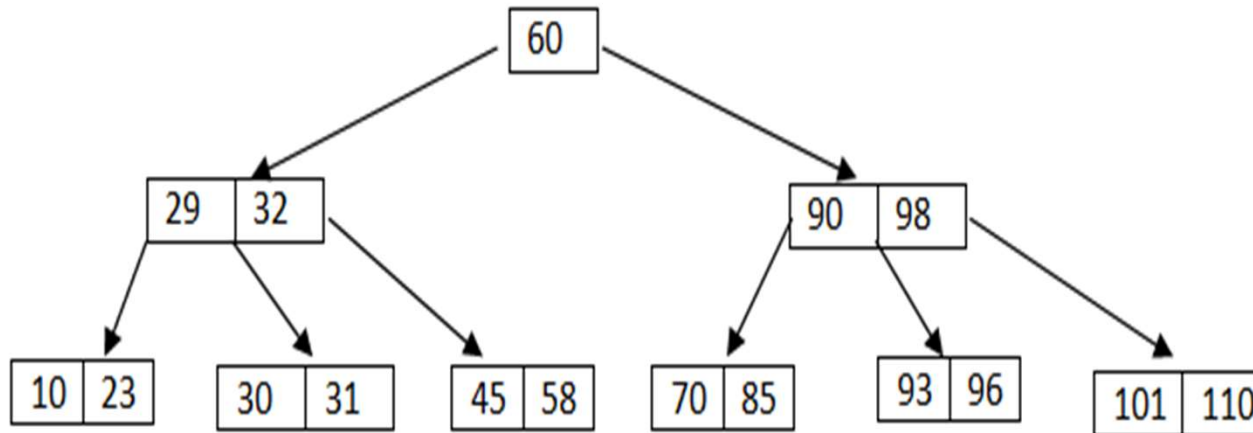
1. Every node can have at most m children.
2. Every node can contain at most $m - 1$ keys.
3. Every internal node (except root) has at least $\lceil m/2 \rceil$ children.
4. Every internal node (except root) contains at least $\lceil m/2 \rceil - 1$ keys.
5. The root has at least 2 children (unless it is the only node).
6. All leaf nodes are at the same level.
7. Keys inside a node are stored in sorted order.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

332

EXAMPLE –A B-TREE OF ORDER 4



Terminology

Order (m): Maximum number of children.

Degree: Number of children of a node.

Leaf Node: Node with no children.

Internal Node: Node having children.

Maximum children = 4

Maximum keys = 3

Minimum children = 2

Minimum keys = 1

Tuesday, August 18, 2026

PREPARED BY:

K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

B-TREE INSERTION ALGORITHM

Step 1

Insert the key into the appropriate leaf node.

Step 2

If the node has fewer than maximum keys, insertion is complete.

Step 3

If the node overflows

- Split the node
- Move the middle key to the parent
- Create two new child nodes

Step 4

Repeat until the root.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

334

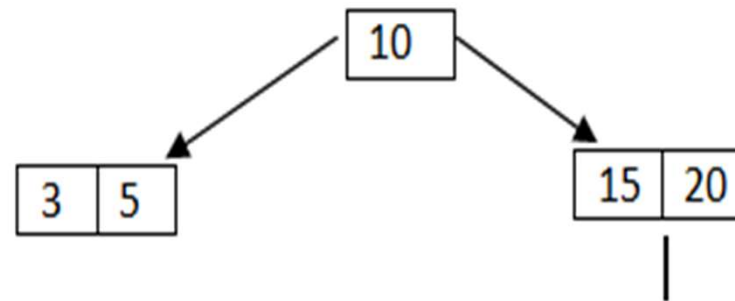
EXAMPLE – 5-WAY TREE

5	10	15	20
---	----	----	----

Insert 3

+	3	5	10	15	20	
---	---	---	----	----	----	--

10 should move one
step up word



PROBLEM

Construct 5 order tree from the following data

1,7,6,2,11,5,10,13,12,20,16,24,3,4,18,19,14,25

Insert 1



Insert 7



Insert 6



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

336

PROBLEM

Construct 5 order tree from the following data

1,7,6,2,11,5,10,13,12,20,16,24,3,4,18,19,14,25

Insert 2

1	2	6	7
---	---	---	---

Insert 11

1	2	6	7	11
---	---	---	---	----

Over flow occurs so take
median and shift one step
upwards

Tuesday, August 18, 2026

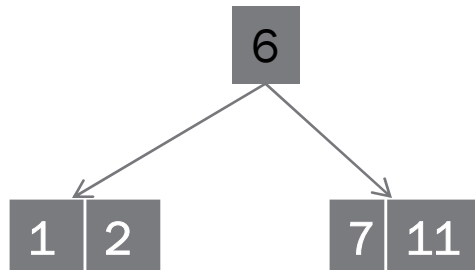
PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

337

PROBLEM

Construct 5 order tree from the following data

1,7,6,2,11,5,10,13,12,20,16,24,3,4,18,19,14,25



Split the node at median and push the median to the parent node

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

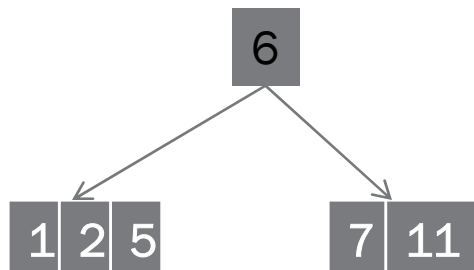
338

PROBLEM

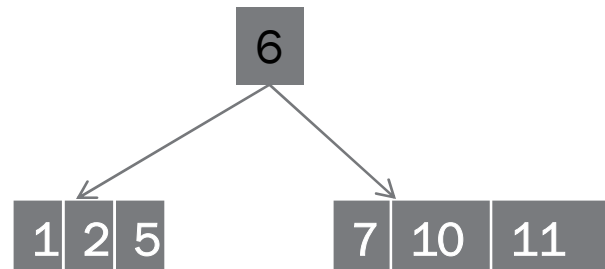
Construct 5 order tree from the following data

1,7,6,2,11,5,10,13,12,20,16,24,3,4,18,19,14,25

Insert 5



Insert 10

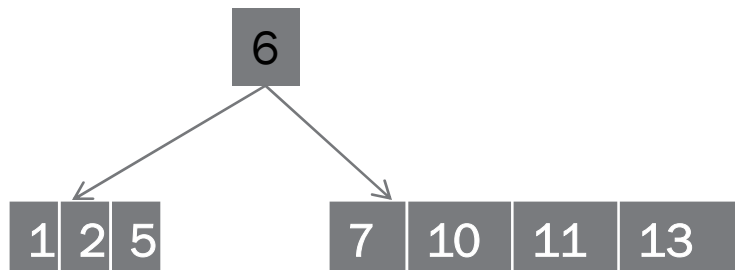


PROBLEM

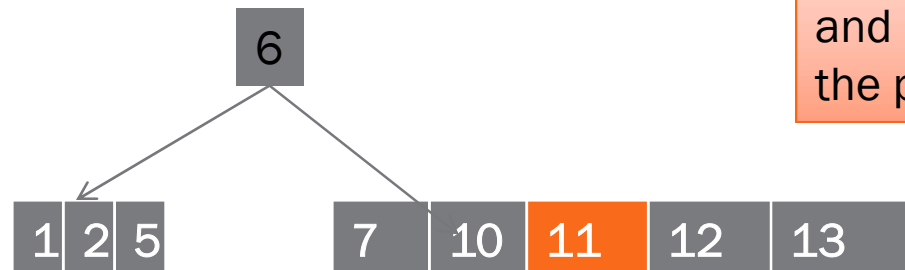
Construct 5 order tree from the following data

1,7,6,2,11,5,10,13,12,20,16,24,3,4,18,19,14,25

Insert 13



Insert 12



Split the node at median and push the median to the parent node

Over flow occurs so take median and shift one step upwards

Tuesday, August 18, 2026

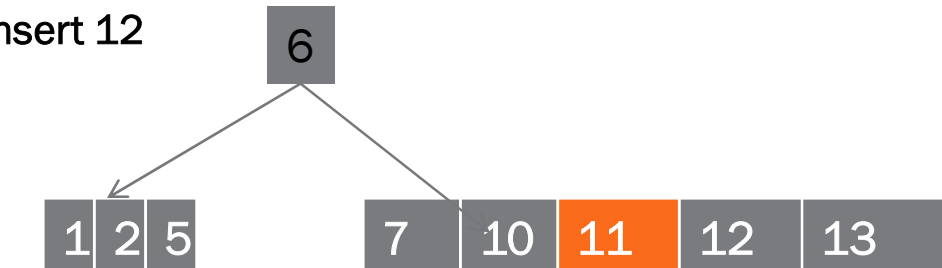
PREPARED BY:
R. S. KANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

PROBLEM

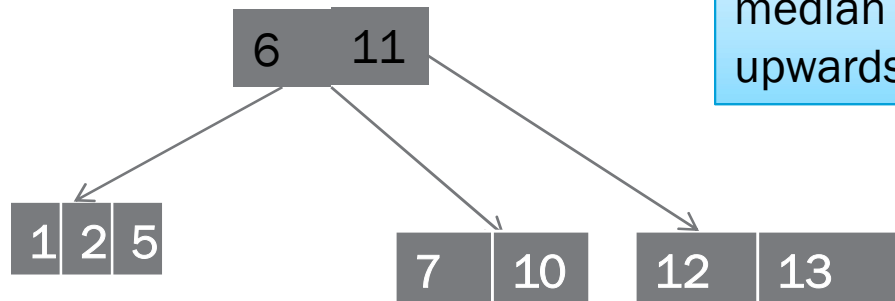
Construct 5 order tree from the following data

1,7,6,2,11,5,10,13,12,20,16,24,3,4,18,19,14,25

Insert 12



Split the node at median and push the median to the parent node



Over flow occurs so take median and shift one step upwards

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

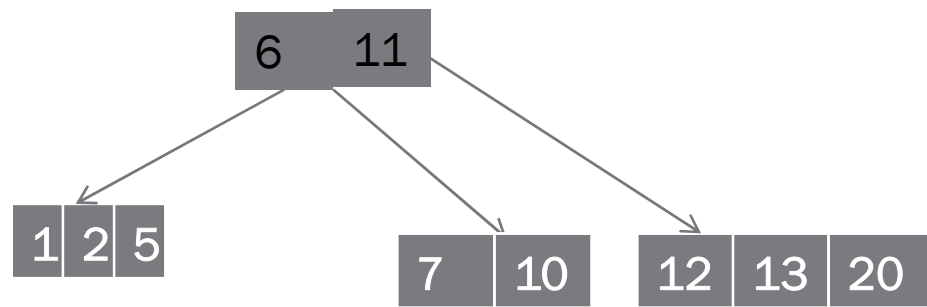
341

PROBLEM

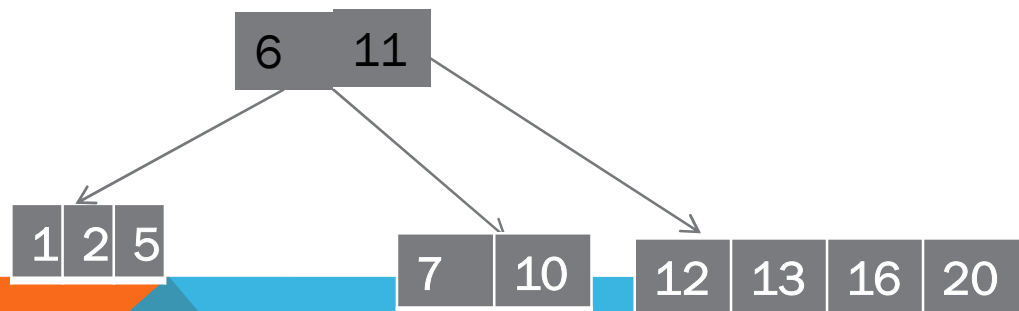
Construct 5 order tree from the following data

1,7,6,2,11,5,10,13,12,20,16,24,3,4,18,19,14,25

Insert 20



Insert 16



Tuesday, August 18, 2026

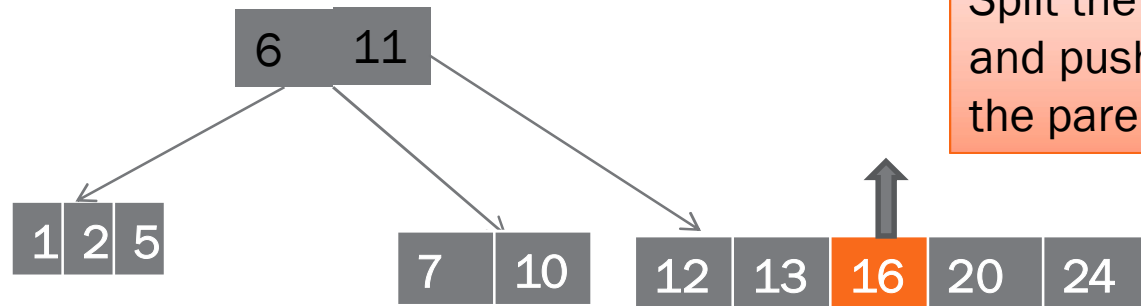
PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

PROBLEM

Construct 5 order tree from the following data

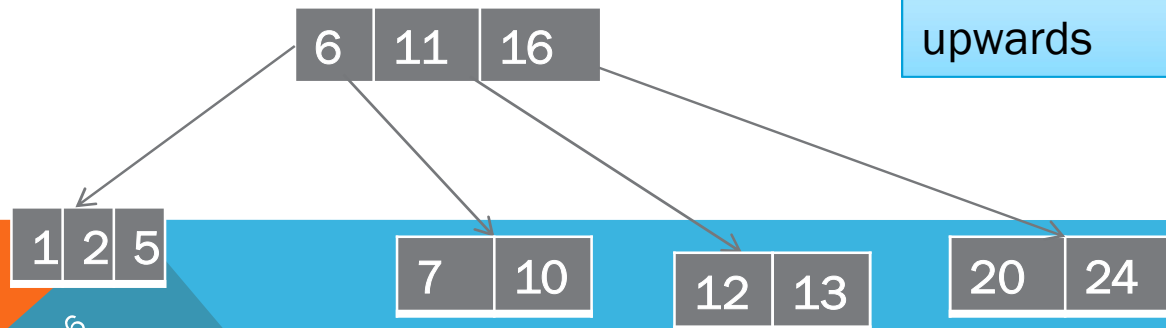
1,7,6,2,11,5,10,13,12,20,16,24,3,4,18,19,14,25

Insert 24



Split the node at median and push the median to the parent node

Over flow occurs so take median and shift one step upwards



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

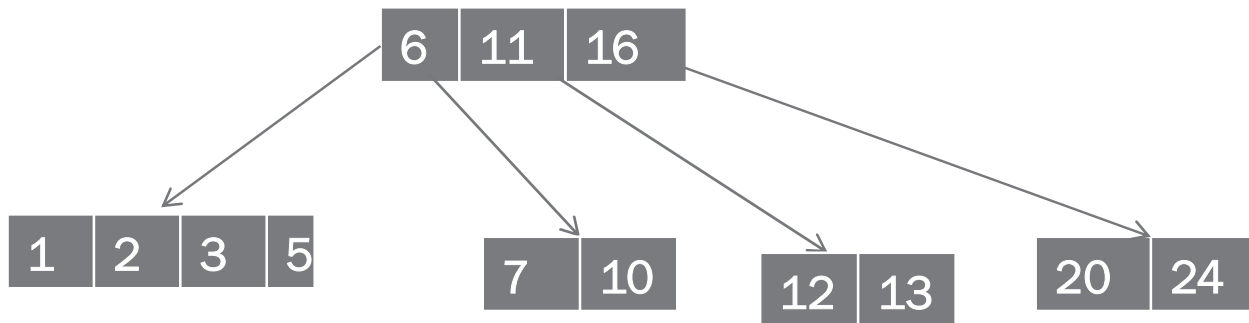
343

PROBLEM

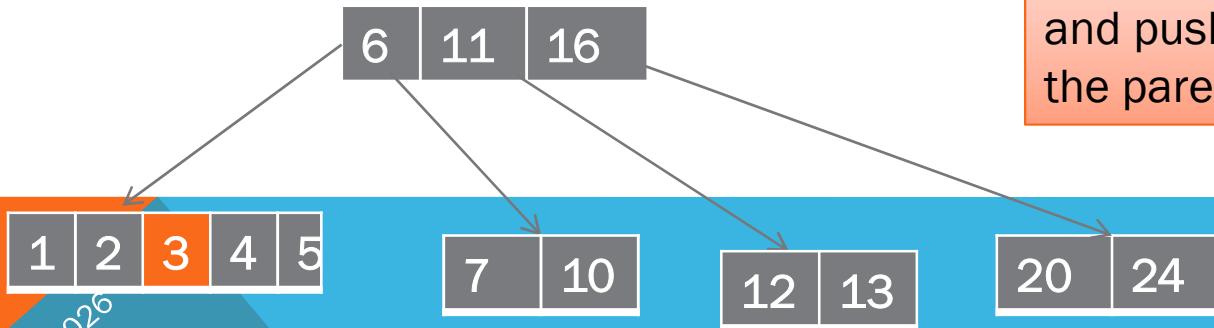
Construct 5 order tree from the following data

1,7,6,2,11,5,10,13,12,20,16,24,3,4,18,19,14,25

Insert 3:



Insert 4:



Split the node at median and push the median to the parent node

Over flow occurs so take median and shift one step upwards

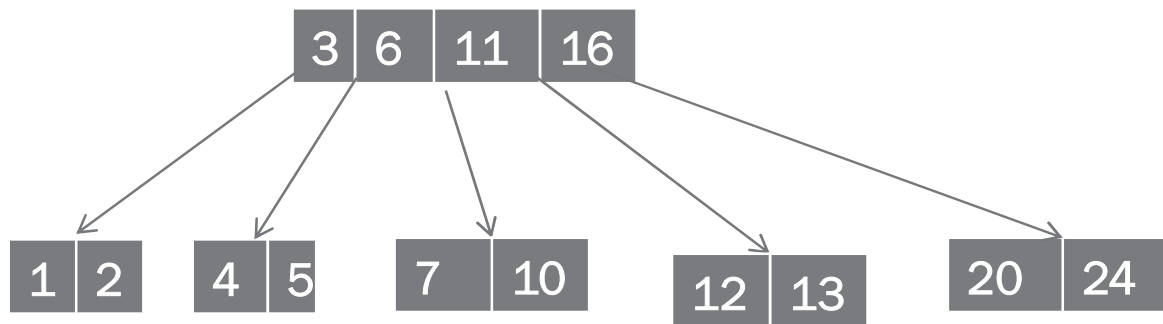
PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

PROBLEM

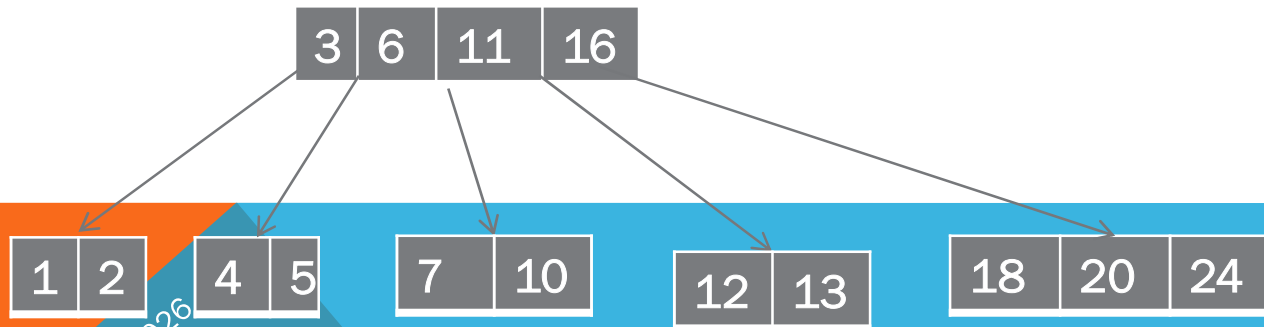
Construct 5 order tree from the following data

1,7,6,2,11,5,10,13,12,20,16,24,3,4,18,19,14,25

Insert 4:



Insert 18:



Tuesday, August 18, 2026

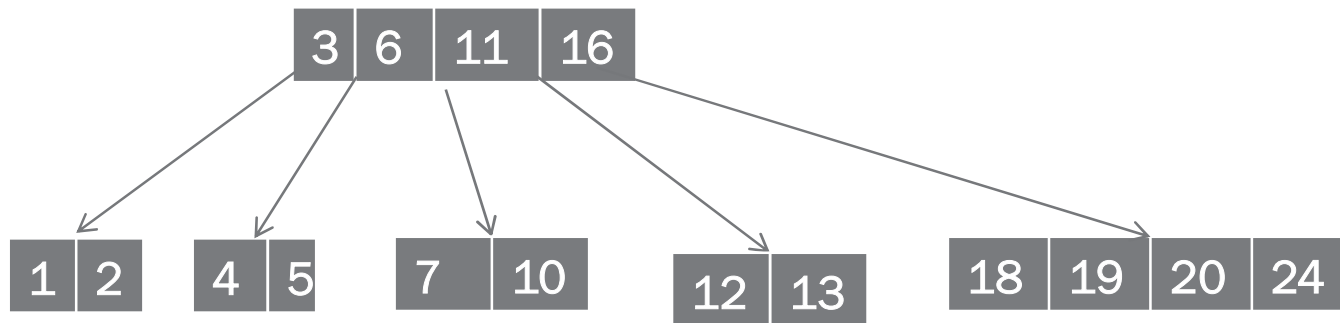
PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

PROBLEM

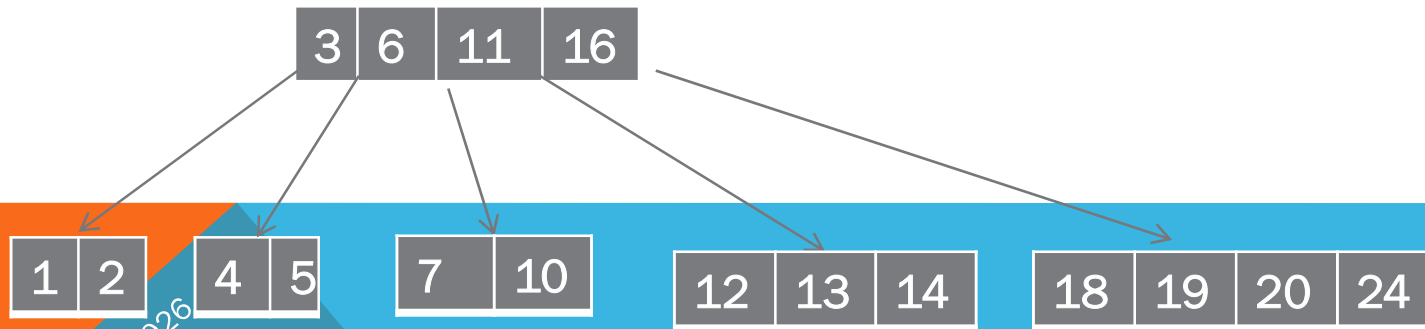
Construct 5 order tree from the following data

1,7,6,2,11,5,10,13,12,20,16,24,3,4,18,19,14,25

Insert 19:



Insert 14:



Tuesday, August 18, 2026

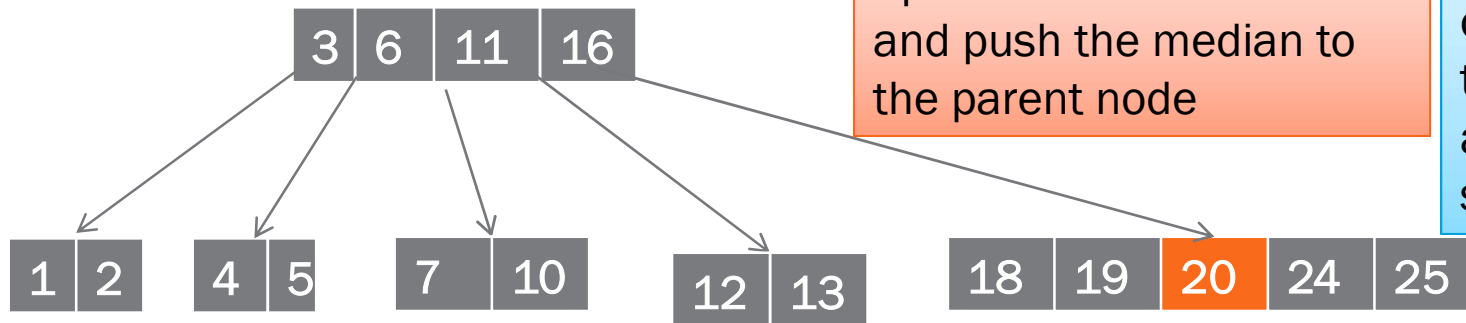
PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

PROBLEM

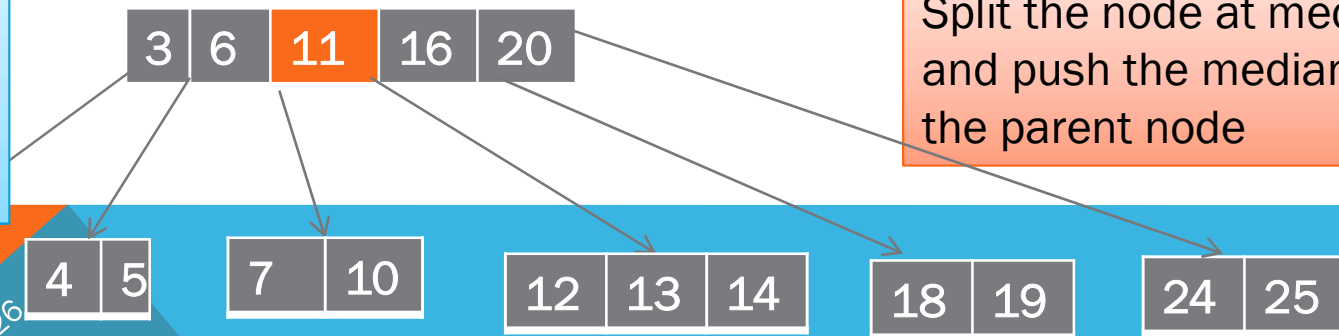
Construct 5 order tree from the following data

1,7,6,2,11,5,10,13,12,20,16,24,3,4,18,19,14,25

Insert 25:



Over flow occurs so take median and shift one step upwards



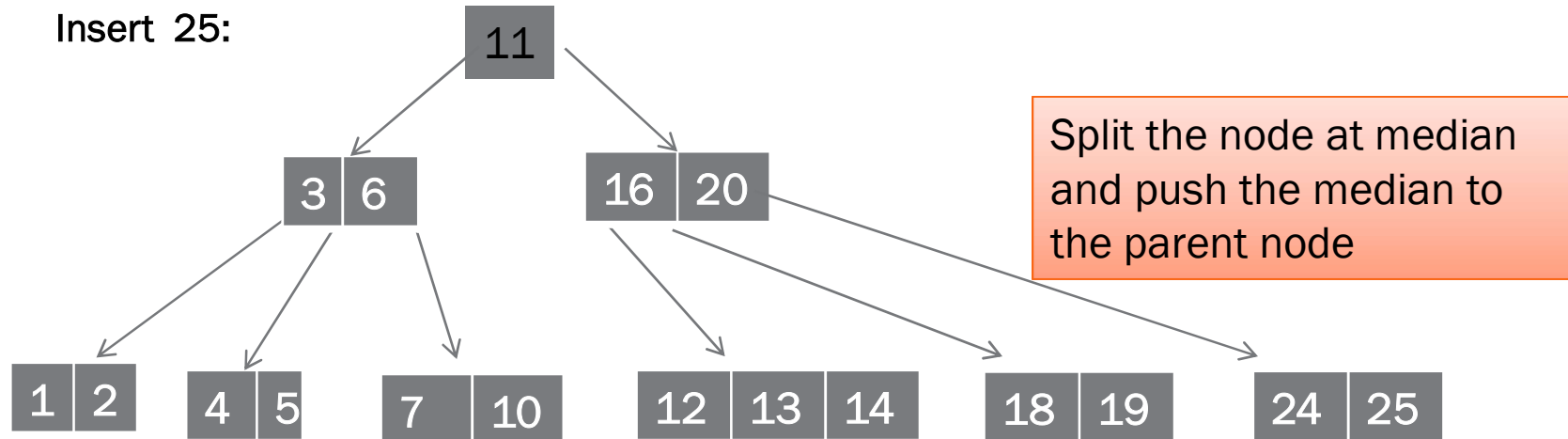
PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

PROBLEM

Construct 5 order tree from the following data

1,7,6,2,11,5,10,13,12,20,16,24,3,4,18,19,14,25

Insert 25:



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

348

DELETION FROM B-TREE

1. Deletion is also performed at the **leaf nodes**
2. The node which is to be deleted can either be a **leaf node or an internal node**
 - A. locate the leaf node
 - B. If there are **more than $m/2$ keys** in the **leaf node** then **delete** the desired key from the node
3. If the leaf node **doesn't contains $m/2$ keys** then complete the keys by taking the elements from **right or left siblings**
 - A. If the **left sibling contains more than $m/2$ elements** then push its **largest element** up to its **parent** and move the **intervening element** down to the node where the key is deleted
 - B. . If the **right sibling contains more than $m/2$ elements** then push its **smallest element** up to its **parent** and move the **intervening element** down to the node where the key is deleted

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

349

DELETION FROM B-TREE

4. If **neither of the sibling contains more than $m/2$ elements** then create a leaf node by joining two leaf nodes and the intervening element of the parent node
5. If **parent is leaf** with **less than $m/2$ nodes** then apply the above process on the parent too.
6. If the node which is to be deleted is an **internal node** then replace the node with its **in-order successor or in-order predecessor**. Since, successor or predecessor will always be on leaf node hence the process will be similar as the node is being deleted from leaf node

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

350

DELETION FROM B-TREE

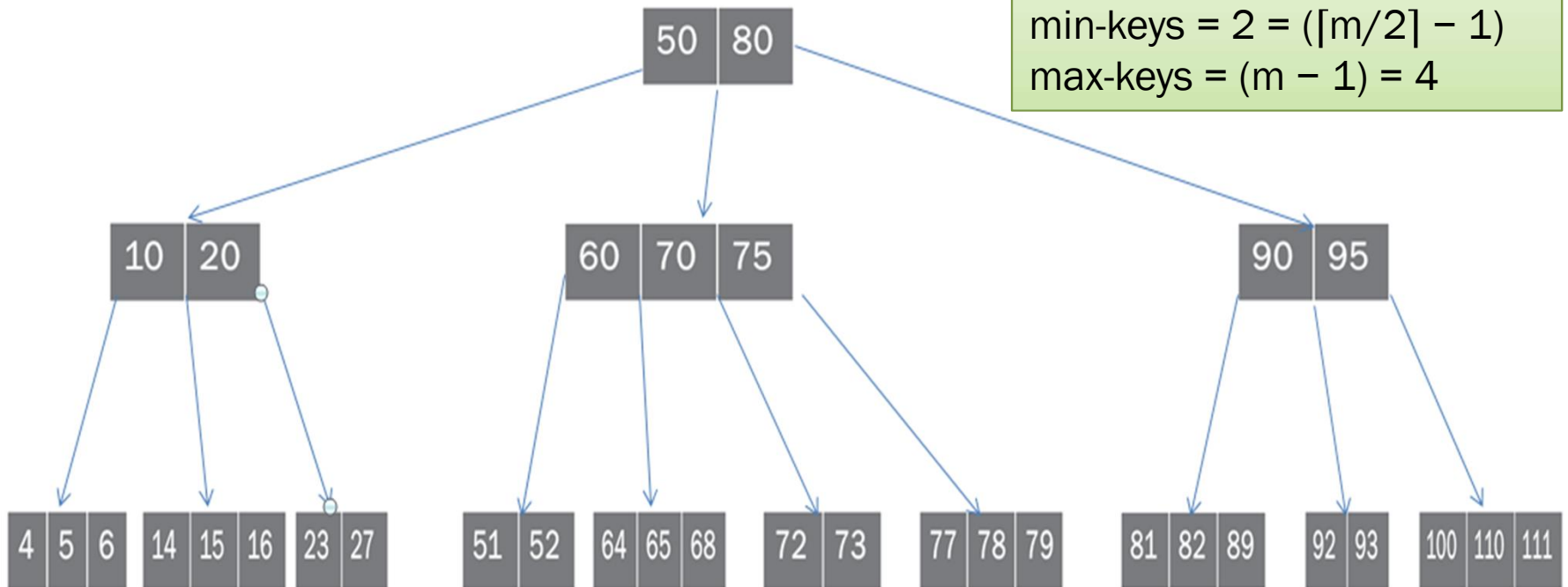
order (m) = 5

min-children = $\lceil m/2 \rceil = 3$

max-children = m = 5

min-keys = 2 = $(\lceil m/2 \rceil - 1)$

max-keys = (m - 1) = 4



Tuesday, August 18, 2026

PREPARED BY:

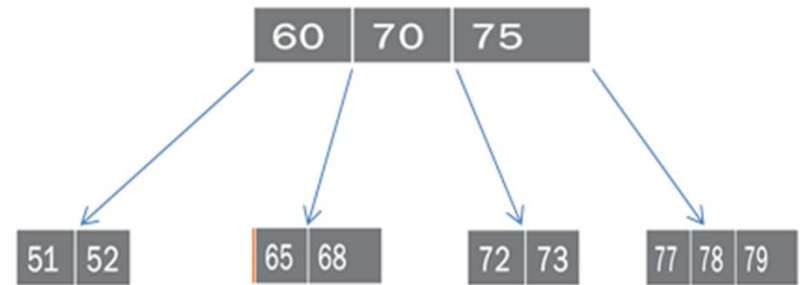
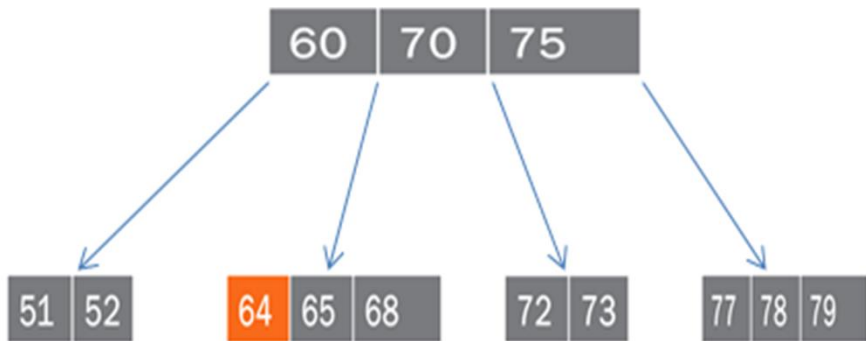
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

351

DELETE NODE 64

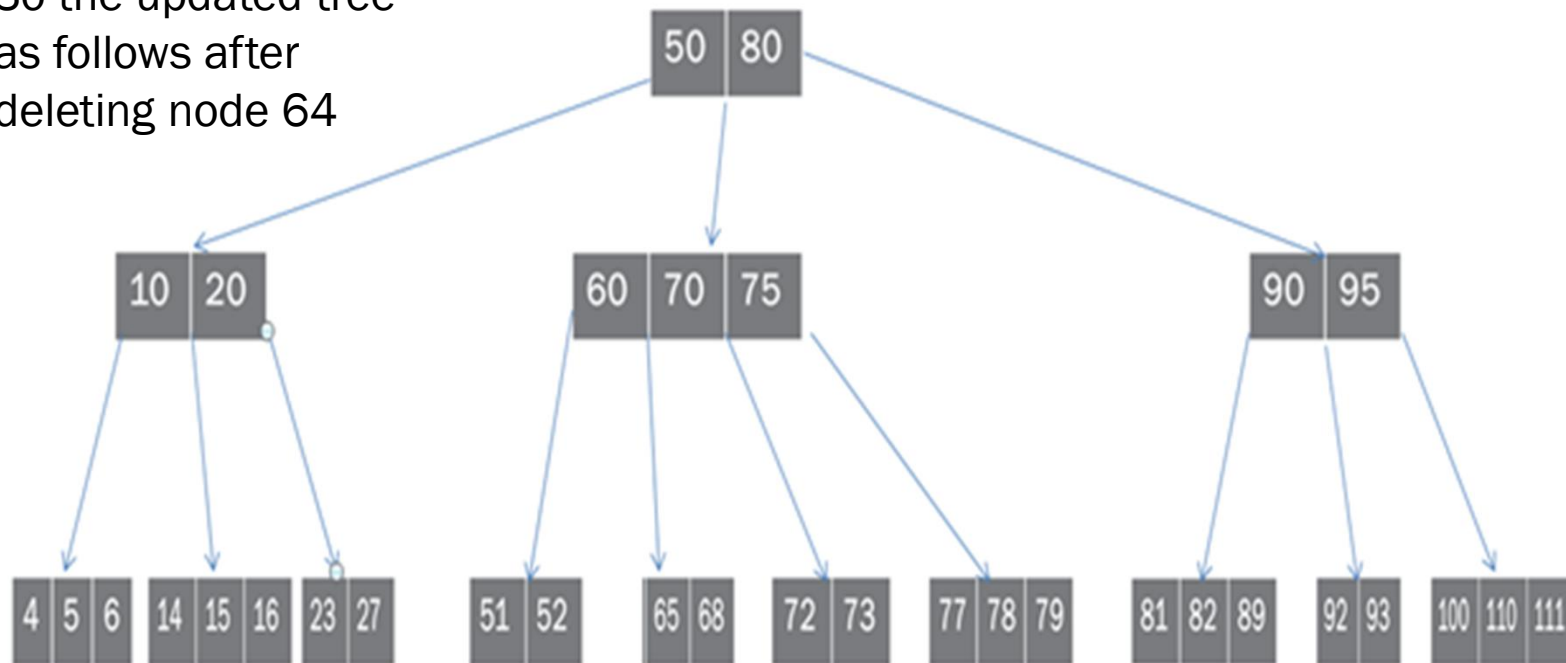
We can directly delete node 64 as
it has minimum keys and it is leaf node

The middle sub tree of above tree will be



DELETE NODE 64

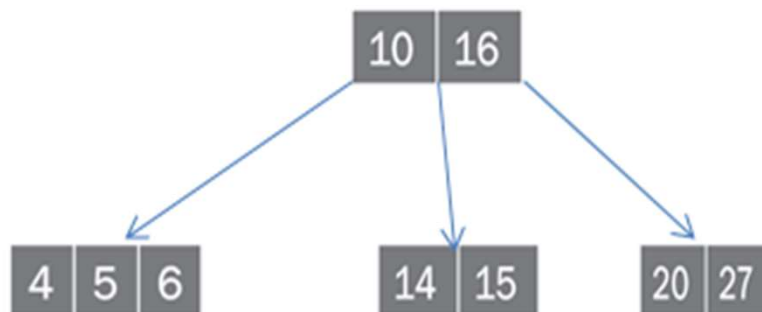
So the updated tree
as follows after
deleting node 64



DELETE NODE 23

According to case 3(a) borrow node from immediate left sibling with the help of parent node, as it don't have right sibling

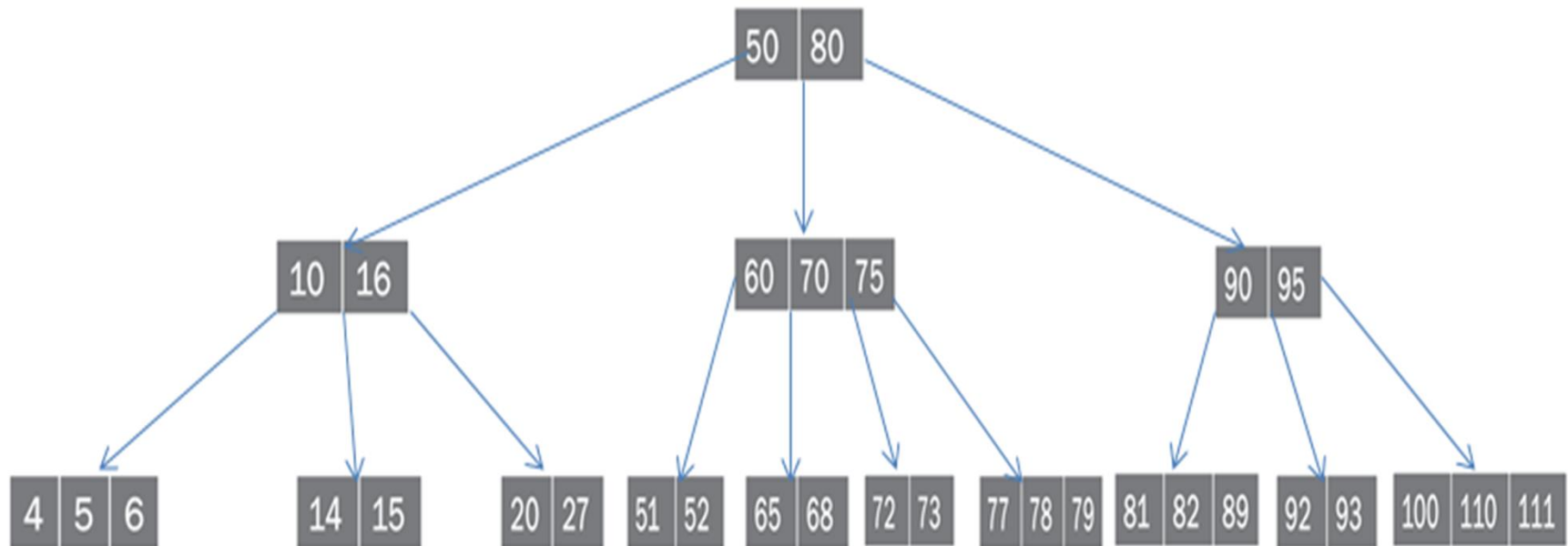
Now the part of that sub tree will be look as below



16 will go up to the parent and 20(highest element) will come down to its right child

DELETE NODE 23

After deleting node 64 and 23 the tree will look like this



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

355

DELETE NODE 72

According to case 3(b) borrow node from immediate right sibling with the help of parent node as it don't have minimum keys in left sibling

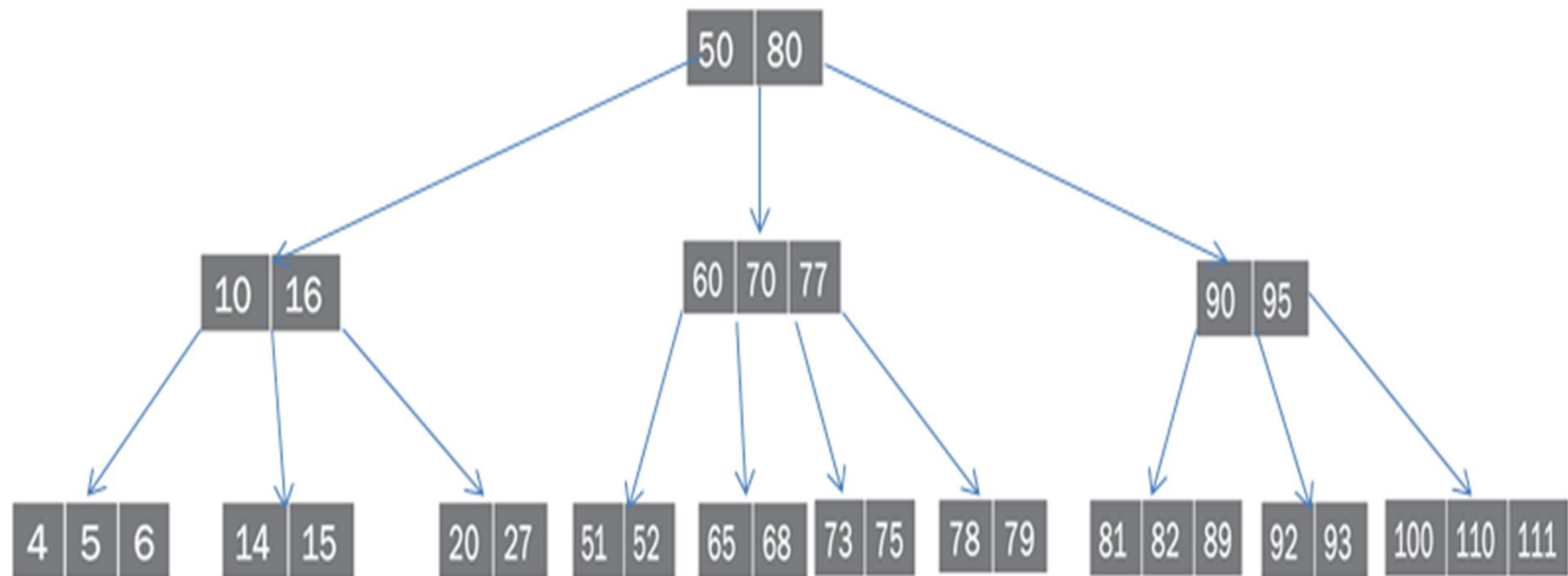
Now the middle sub tree will be updated as follows after deleting 72



77 will go its parent and 75 will come down to its leaf node and 72 is deleted

DELETE NODE 72

After deleting node 64, 23 72 the tree will look like this



DELETE NODE 65

According to case 4 create a new leaf node by joining two leaf nodes and by bringing intervening element from parent node

Now the middle sub-tree as updated as below after deleting 65



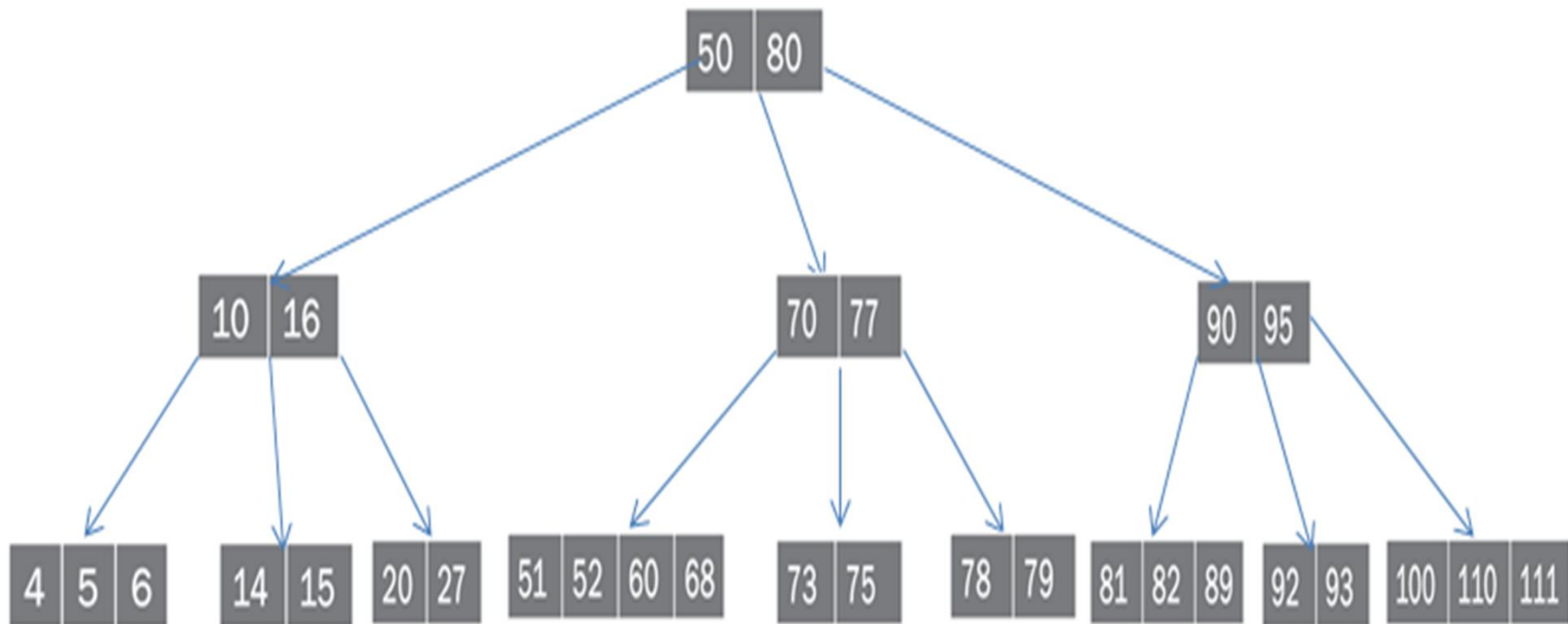
Combine leaf nodes 51,52 and 65,68 and bring intervening element 60 to the leaf node and delete 65

Note: you can combine 65,68 and 73,75 by bringing intervening element 70 to the leaf node

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

DELETE NODE 65

After deleting node 64, 23, 72, 65 the tree will look like this



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

359

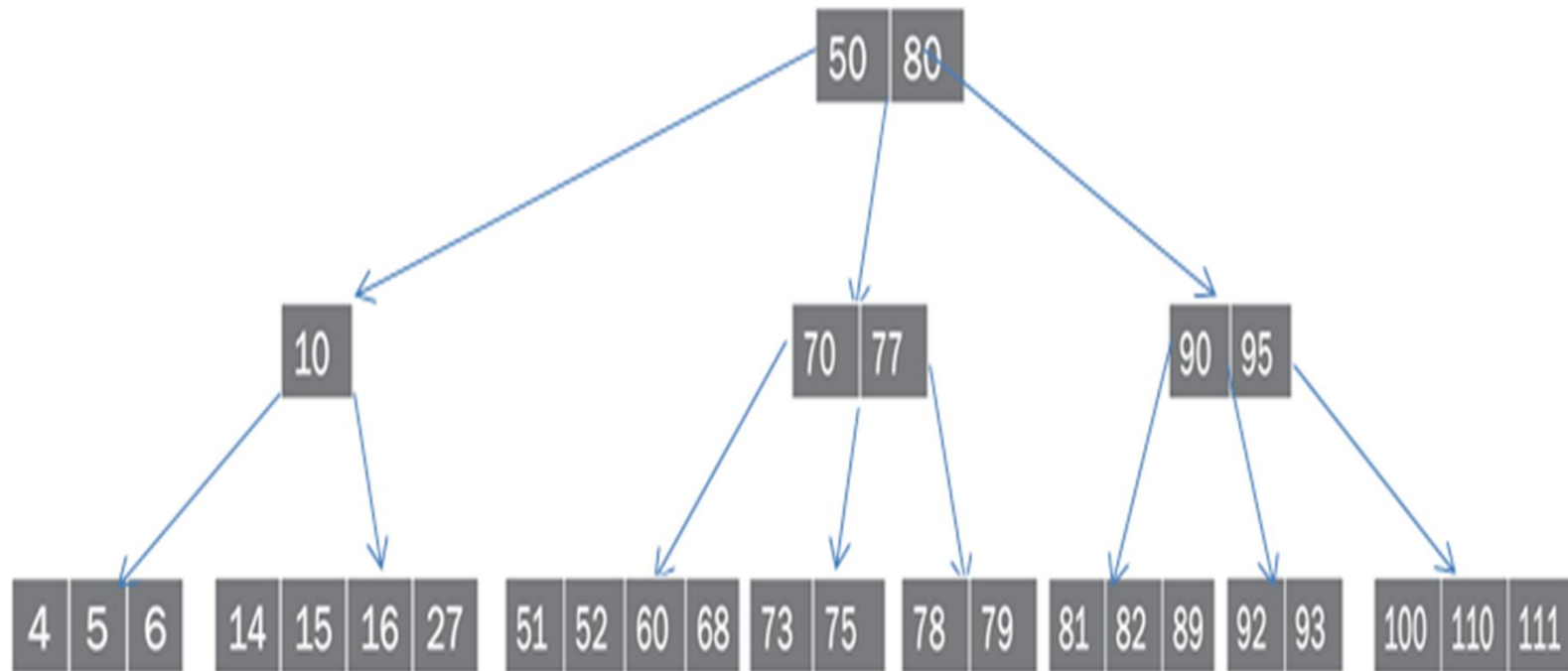
DELETE NODE 20

According to case 4 after deleting 20 the updated sub tree is as below



Join node 14,15 and 20,27 by bringing intervening element 16 from its parent and then delete 20

DELETE NODE 20

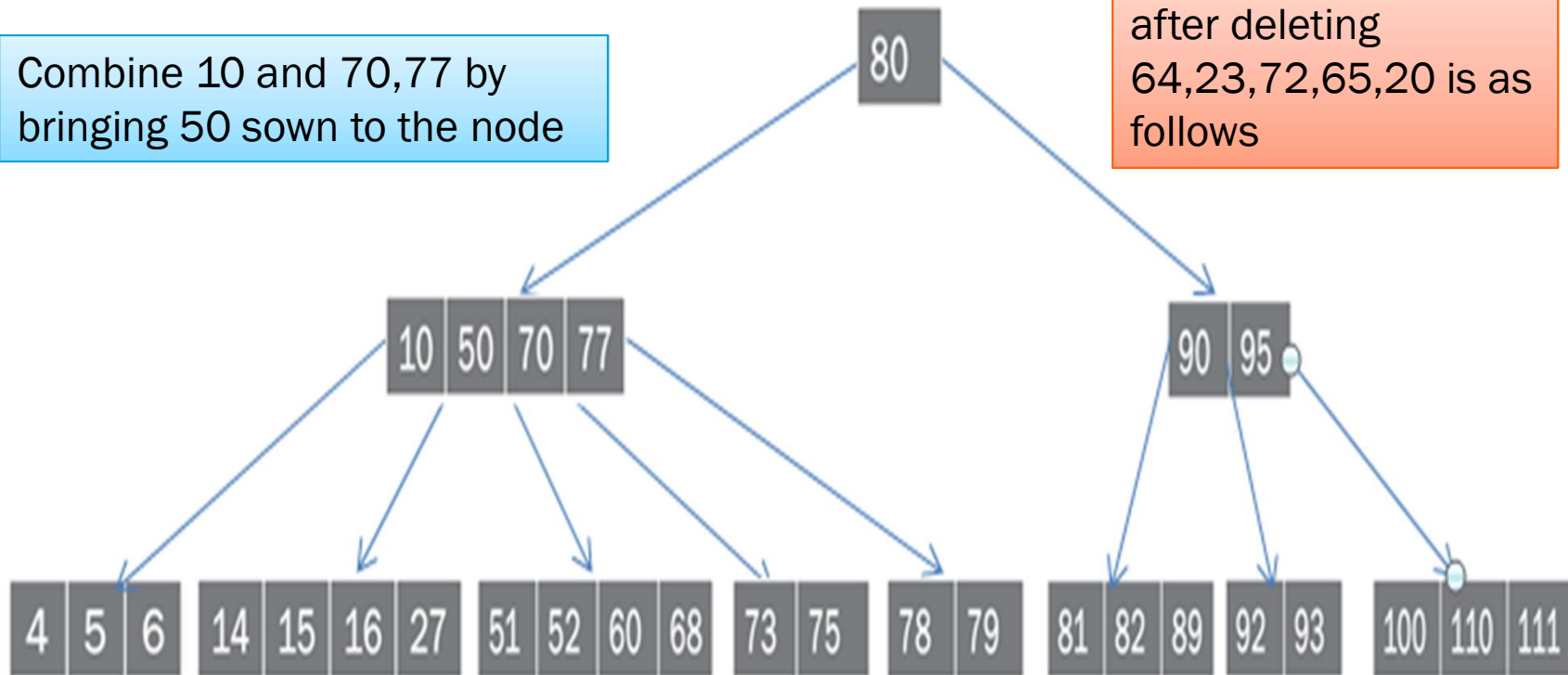


But the problem here internal node 10 is not having minimum keys. So again according to case 4 at node 10 repeat the process and updated sub tree as follows

DELETE NODE 20

Combine 10 and 70,77 by bringing 50 down to the node

So the updated tree after deleting 64,23,72,65,20 is as follows



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

362

DELETE NODE 70

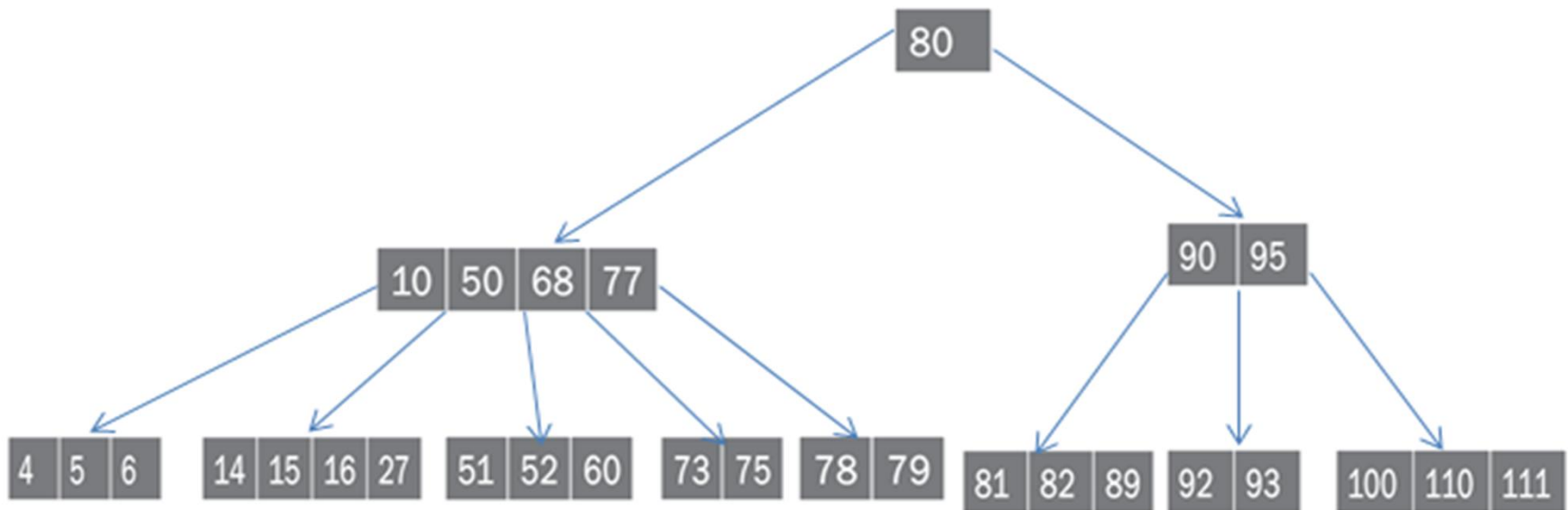
Now 70 is internal node then according to case 6 the updated sub tree as follows



Replace 70 with inorder predecessor 68 and delete 70

DELETE NODE 70

So the updated tree after deleting 64,23,72,65,20, 70 is as follows



Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

364

DELETE NODE 95

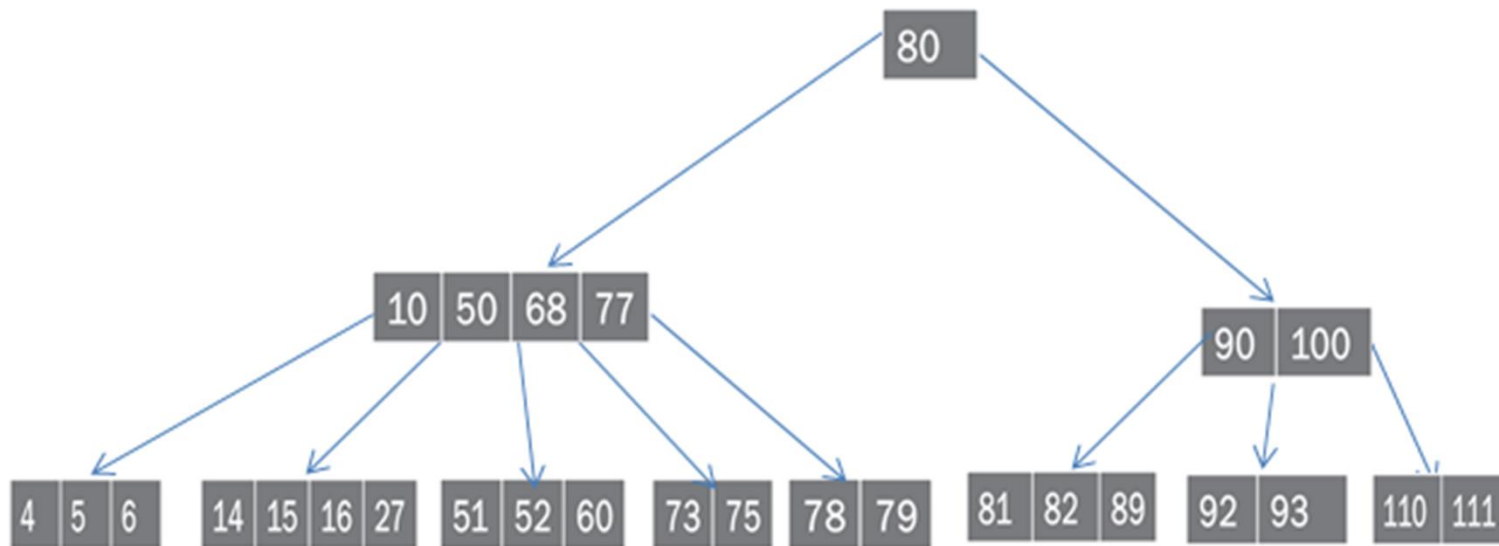
Now 95 is the internal node then according to case 6 the updated right sub tree as follows



Replace 95 with inorder successor i.e with 100 and 95 is deleted

DELETE NODE 95

So the updated tree after deleting 64,23,72,65,20, 70,95 is as follows



DELETE NODE 77

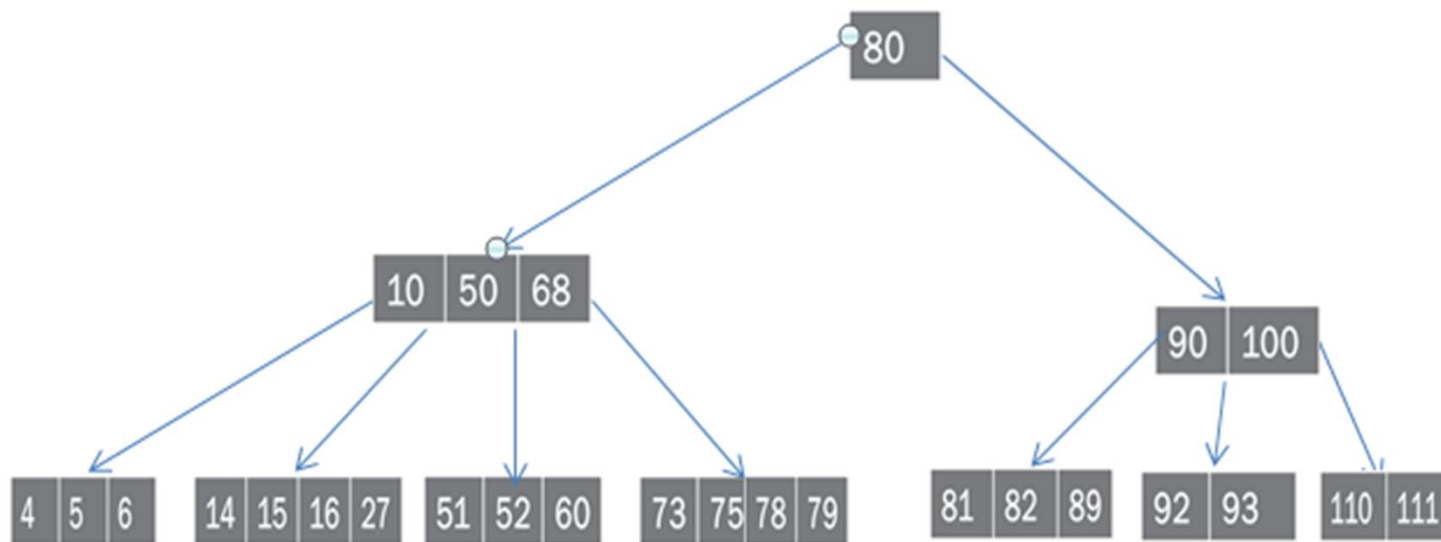
Now 77 is internal node then according to case 6 the updated right sub tree is as follows

Here, 77 is having minimum keys neither in inorder successor node or in inorder predecessor so combine leaf nodes 73,75, and 78,79 by bringing intervening element 77 and then delete node 77 from leaf node



DELETE NODE 77

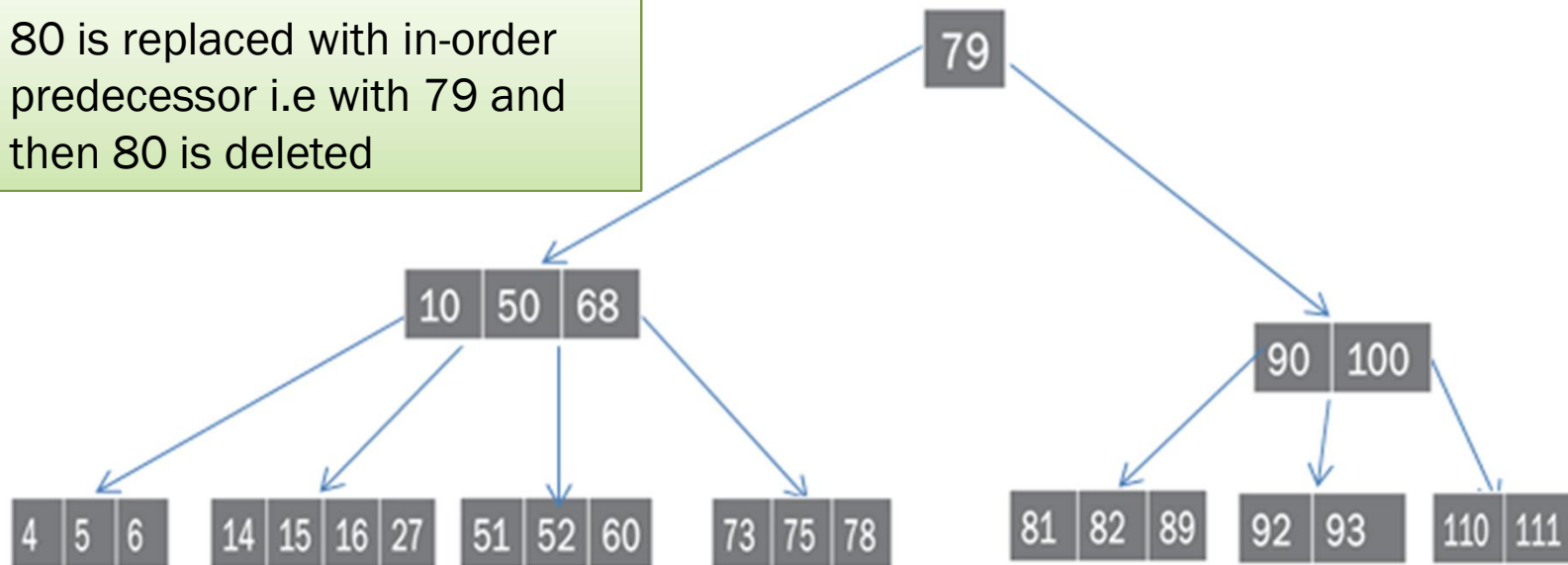
So the updated tree after deleting 64,23,72,65,20, 70,95, 77 is as follows



DELETE NODE 80

Node 80 is an internal node so according to case 6 updated tree as follows

80 is replaced with in-order predecessor i.e with 79 and then 80 is deleted



DELETE NODE 100

Node 100 is an internal node then according to case 6 updated right sub tree is as follows

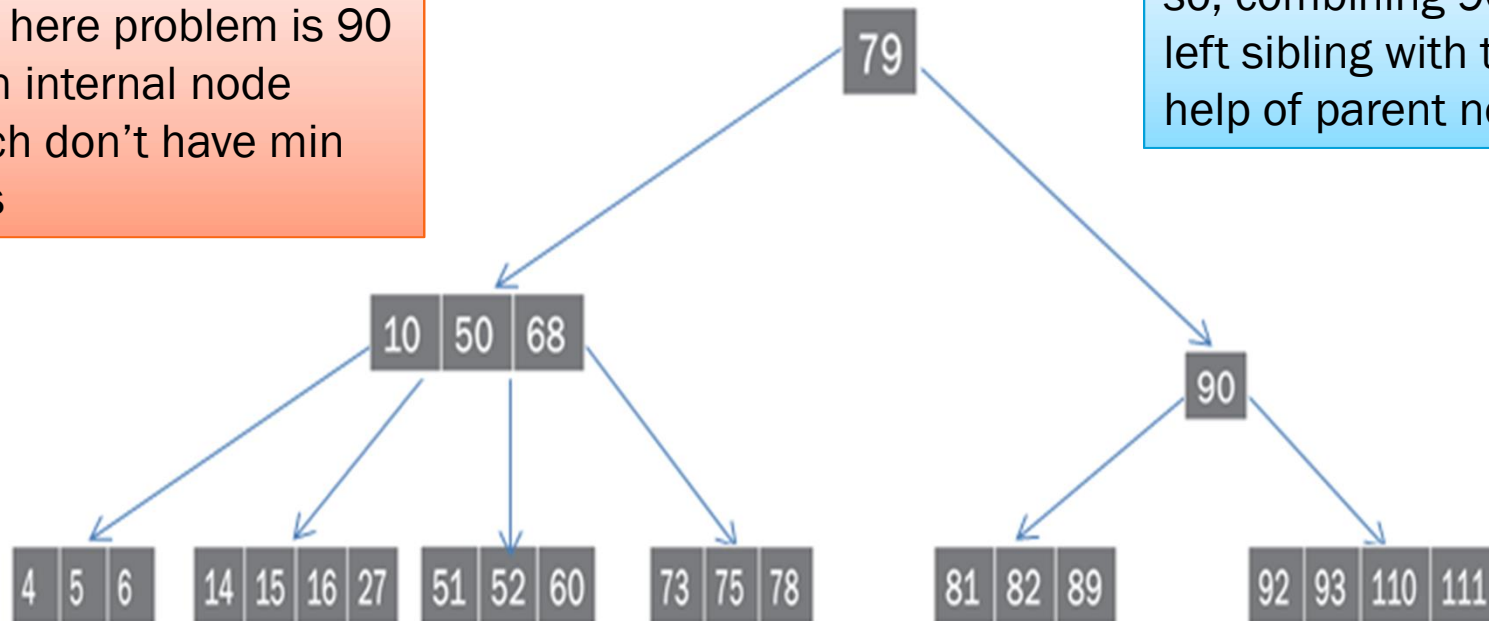


100 is deleted by combining leaf nodes 91,93 and 110,111 by bringing intervening element 100 down to its leaf

DELETE NODE 100

Updated tree as follows

But, here problem is 90 is an internal node which don't have min keys



so, combining 90 with left sibling with the help of parent node

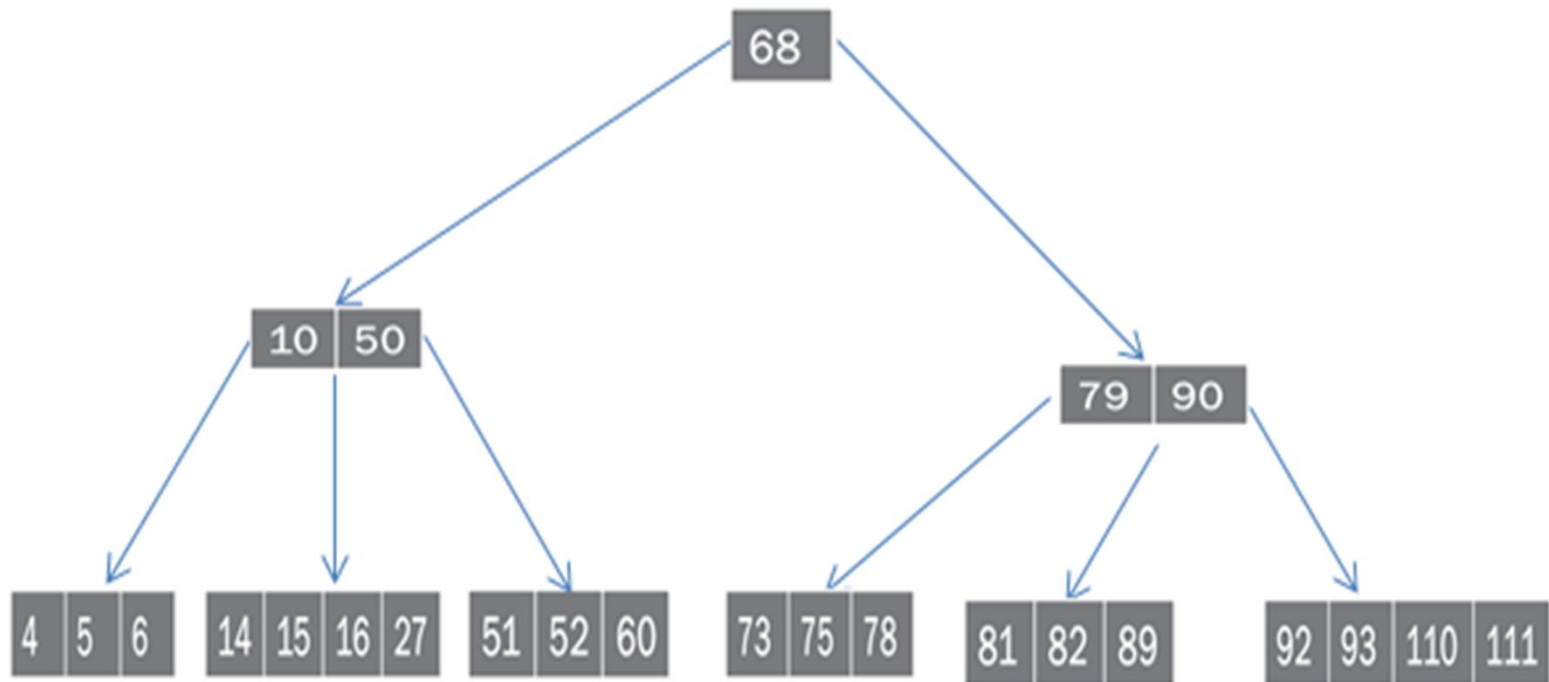
Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

371

DELETE NODE 100

So updated tree after deleting node 100 is as follows

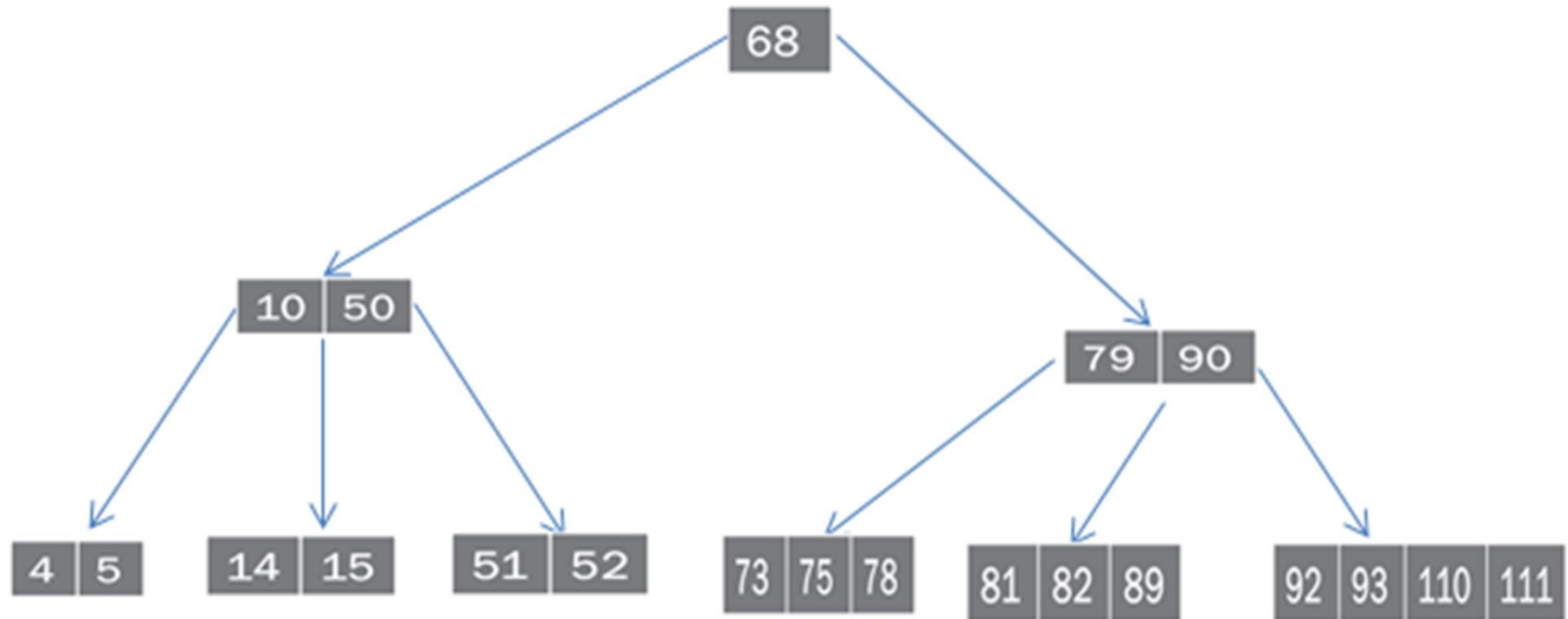


Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

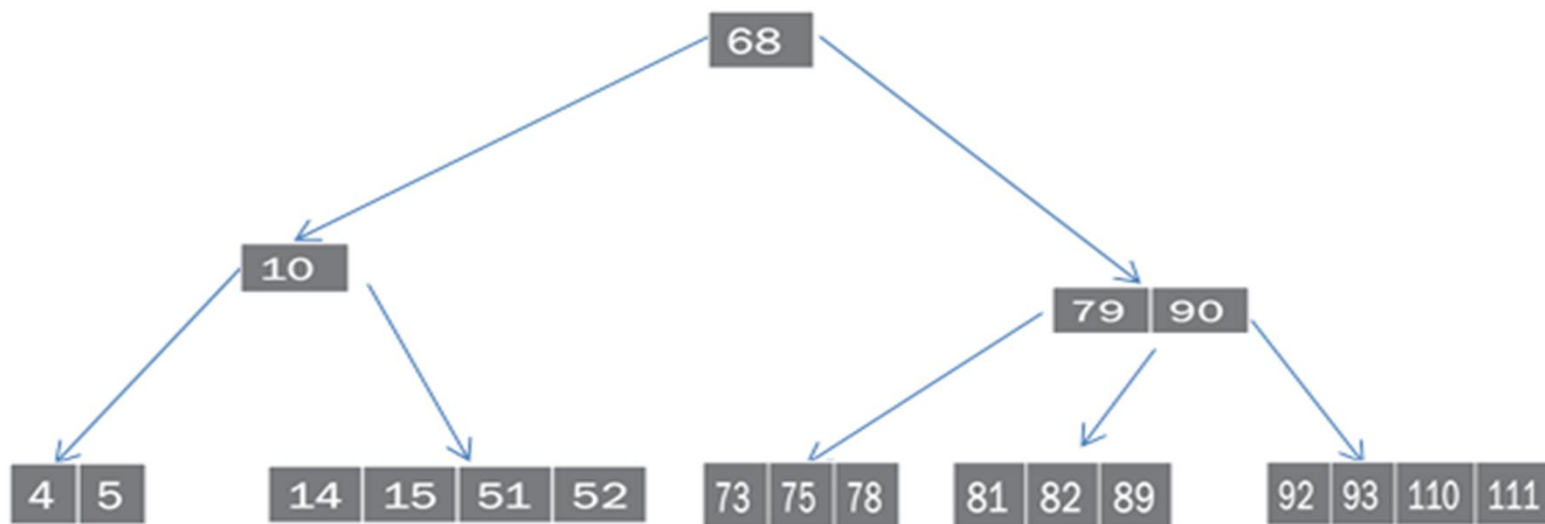
DELETE NODE 6, 27, 60, 16

The updated tree as follows after deleting 6,27,60,16



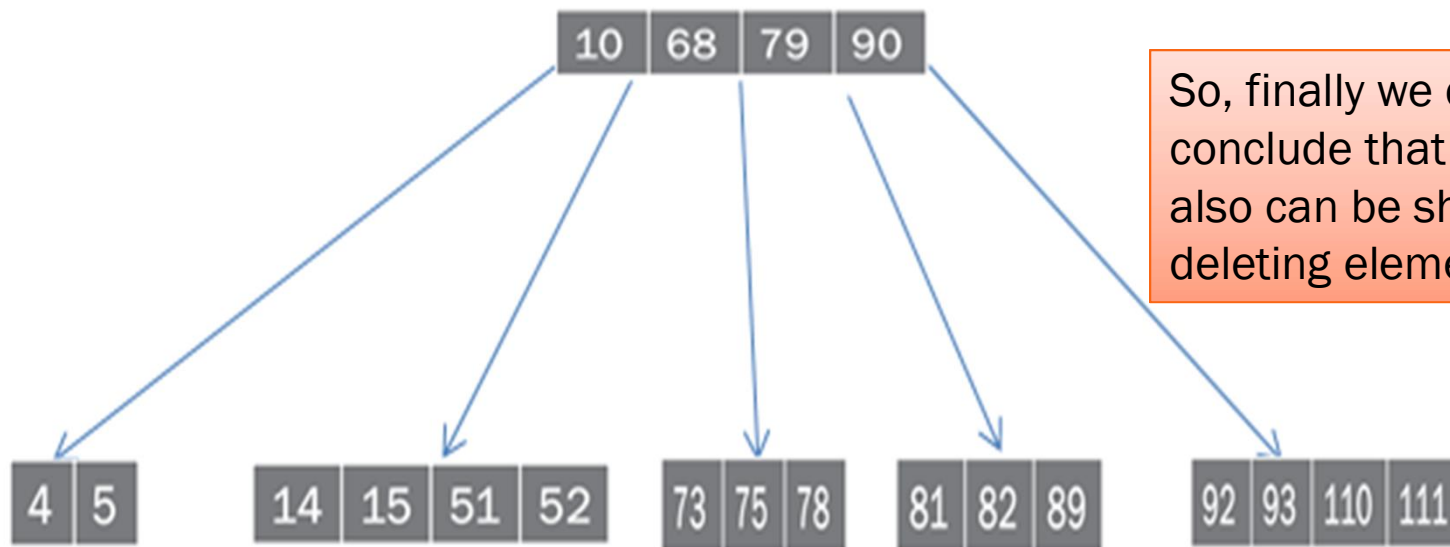
DELETE NODE 50

Now, neither in-order successor or predecessor have min keys so combine 14,15 and 51,52 by bringing intervening element 50 to the leaf node and then delete node 50. the updated tree as follows



DELETE NODE 50

But, the problem is 10 is not having min keys so combine 10 with its right sibling by bringing intervening element 68 to its leaf node then updated tree as follows



So, finally we can conclude that a tree also can be shrink by deleting elements

APPLICATIONS OF B-TREE

B-Tree is widely used in systems that need fast searching, insertion, deletion, and efficient disk access.

1. Database Management Systems (DBMS)

Used for indexing large databases.

Enables fast retrieval of records.

Examples: MySQL, Oracle, PostgreSQL, SQL Server.

2. File Systems

Used to organize files and directories efficiently.

Helps in quick file searching and updating.

Examples:

NTFS (Windows)

HFS+ (macOS)

Ext4 (Linux uses B-Trees in some indexing features)

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

APPLICATIONS OF B-TREE

3. Database Indexing

B-Trees create indexes on database columns.

Reduces the time required to execute queries.

4. Multilevel Indexing

Used in secondary storage to maintain multiple levels of indexes.

Improves search performance for very large files.

5. Disk-Based Storage Systems

Designed to minimize disk I/O operations.

Efficient for hard disks and SSDs where accessing data from disk is slower than accessing RAM.

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

377

APPLICATIONS OF B-TREE

6. Search Engines

Used to maintain indexes of web pages and documents for faster searching.

7. Operating Systems

Used for directory management, file indexing, and storage allocation.

8. Cloud Storage Systems

Used in distributed storage systems to efficiently organize and retrieve massive amounts of data.

9. Data Warehouses

Used for indexing huge datasets in business intelligence and analytics applications.

10. Key-Value Stores

Used in some NoSQL databases and storage engines to maintain sorted keys efficiently.

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

ADVANTAGES OF USING B-TREE

- Fast Search: $O(\log n)$
- Fast Insertion: $O(\log n)$
- Fast Deletion: $O(\log n)$
- Self-balancing tree
- Minimizes disk accesses
- Suitable for storing millions of records
- Supports efficient range queries

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

379

QUESTIONS ON B-TREE

1. Explain the insertion algorithm of a B-Tree with an example.
2. Explain the deletion algorithm of a B-Tree.
3. Write the properties of a B-Tree.
4. Compare BST and B-Tree.
5. Construct a B-Tree of order 3 by inserting: 8, 9, 10, 11, 15, 16, 17, 18, 20, 23. Delete the key 16 from the B-Tree and show the resulting tree.
6. Explain the steps involved in deleting a key from a B-Tree. Illustrate with an example B-Tree of order 2 containing keys: 8, 9, 10, 11, 15, 16, 17, 18, 20, 23. Delete the key 10.
7. State the main advantage of using B-trees over AVL trees in disk storage

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

380

THANK YOU

Tuesday, August 18, 2026

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

381